The Backend Engineer's Library

SBOMs and Software Transparency

Volume C03

Contents

Why SBOMs: Transparency and the Regulatory Landscape

SPDX in Depth

CycloneDX in Depth

SBOM Generation: Tools, Techniques, and Accuracy

SBOM Distribution, Storage, and Querying at Scale

VEX and Vulnerability Correlation

SBOM Quality, Completeness, and Limitations

SBOMs for Services: Containers, Serverless, and SaaS

Operationalizing SBOMs in the Enterprise

Chapter 1 — Why SBOMs: Transparency and the Regulatory Landscape

What this chapter covers. Book 2 ended on a decoupling insight: your software's *inventory* changes on one clock (when you change dependencies) and the *vulnerability data* changes on another (constantly, against code you shipped months ago), and the architecture that exploits this decoupling is "generate the inventory once, re-match it against new advisories forever." The artifact that makes that inventory portable, machine-readable, and durable is the **Software Bill of Materials** — the SBOM. This chapter is the motivation for the entire book. It defines what an SBOM actually is (and is not), takes apart the "ingredients list" analogy that everyone reaches for, explains why the industry converged on SBOMs after a specific pair of incidents, catalogs the problems SBOMs genuinely solve, is blunt about the ones they do not, and maps the regulatory and standards landscape that is now dragging SBOM production from a nice-to-have into a compliance obligation. We stay at the level of *concepts and forces* here; the wire formats (SPDX in Chapter 2, CycloneDX in Chapter 3), the generation mechanics and their accuracy limits (Chapter 4), the storage-and-query problem at fleet scale (Chapter 5), VEX (Chapter 6), and an honest accounting of quality and limitations (Chapter 7) are each their own chapter.

Learning goals — after this chapter you should be able to:

- Define an SBOM precisely as a **machine-readable component inventory with relationships and metadata**, and explain why the dependency *graph* — not a flat list — is the real deliverable.
- Enumerate the **core data** an SBOM carries: components with unique identifiers (purl, CPE, SWID), dependency relationships, licenses, hashes, and provenance pointers, and distinguish direct from transitive.
- Distinguish the **six SBOM types by lifecycle stage** (Design, Source, Build, Analyzed, Deployed, Runtime) per CISA's taxonomy, and explain why *where and when* you generate an SBOM determines what it can and cannot see.

- Articulate the **killer use case** — rapid impact assessment when a new vulnerability drops (the Log4Shell problem) — and the other problems SBOMs address: vulnerability management at scale, license compliance, vendor and procurement risk, incident response, and M&A due diligence.
- State honestly what an SBOM is **not**: not a security assessment, not an exploitability verdict, not a malware detector, and only ever as good as its generation.
- Place the **regulatory drivers** in accurate chronological order — US EO 14028, NTIA minimum elements, OMB memoranda, the EU Cyber Resilience Act, FDA premarket requirements — and name the **standards** they lean on (SPDX/ISO 5962, CycloneDX/ ECMA-424, SWID/ISO 19770-2).

A word on boundaries. This chapter assumes Book 2's vulnerability data model (CVE, NVD, OSV, GHSA, purl versus CPE) and its SCA pipeline (Book 2, Chapter 6 — Software Composition Analysis in Depth). It treats the SBOM as the *inventory substrate* that pipeline consumes and leaves the formats, generation, and scaling to the chapters that follow. It is deliberately a chapter about *why* and *what*, not *how*.

What an SBOM is

Strip away the acronym and an SBOM is a **formal, machine-readable record of the components that make up a piece of software, the relationships among those components, and enough metadata to identify each one unambiguously.** Three words in that sentence do the load-bearing work.

Formal means it conforms to a published schema, not a spreadsheet someone maintains by hand. *Machine-readable* means a program — an SCA scanner, a policy engine, an inventory database — can parse it without heuristics. And *relationships* means it is not merely a list: it encodes which component depends on which, so you can reconstruct the graph.

The universally reached-for analogy is the **ingredients list** on packaged food, and it is worth examining because it is both genuinely useful and quietly misleading. The useful part: like an ingredients label, an SBOM lets a consumer of the product see what is inside *without having to reverse-engineer it*, and it lets them react to a recall — "this lot contains

an ingredient just found contaminated" — by checking the label rather than lab-testing the food. That is exactly the SBOM value proposition: transparency you can act on mechanically.

Where the analogy breaks down matters, and Chapter 7 returns to each of these:

- **Software ingredients have ingredients.** A food label is flat; software is a graph that can run ten or more levels deep. Your service depends on a web framework that depends on an HTTP client that depends on a TLS library that depends on a crypto primitive. The interesting component — the one with the CVE — is almost always *transitive*, buried several edges from anything you chose directly.
- **The label can be wrong in ways food labels usually are not.** An ingredients list is produced by the manufacturer who literally combined the ingredients. An SBOM is often produced *after the fact* by a tool inspecting a build output, and that tool can miss things (a statically-linked C library, a shaded Java class, a vendored copy of a package) or hallucinate things. Accuracy is a property of the *generation method*, not of the format.
- **"Contains X" is not "poisoned by X."** A recall on an ingredient means every product containing it is suspect. A CVE in a component does *not* mean every product containing that component is exploitable — the vulnerable code path may be unreachable, disabled, or compiled out. Bridging "contains" to "is actually at risk" is the entire subject of reachability (Book 2, Chapter 7) and VEX (Chapter 6 of this book).

Hold the analogy loosely, then. It sells the *why* well and describes the *reality* poorly.

The core data

What is actually inside an SBOM? At minimum, a set of **components**, each carrying enough to identify it, plus the **relationships** among them. In practice a good SBOM carries:

- **Component identity.** A human name and version, a *supplier* (who produced it), and — critically — one or more **unique identifiers** that a machine can match on. The two that matter most are **purl** (Package URL, e.g. `pkg:maven/org.apache.logging.log4j/log4j-`

`core@2.14.1`), which encodes ecosystem, name, and version in a way that maps cleanly to package registries and to OSV, and **CPE** (Common Platform Enumeration, NIST's identifier, e.g. `cpe:2.3:a:apache:log4j:2.14.1:*:*:*:*:*:*:*`), which is what NVD keys its vulnerability records on. A component may also carry a **SWID** tag or a raw hash as an identifier. Book 2, Chapter 5 explained why purl and CPE produce *different* match results; an SBOM that carries both is more useful precisely because downstream tooling keys on different identifiers.

- **Dependency relationships.** Edges: "component A depends on component B," "package P contains file F." This is what turns the list into a graph.
- **License information.** An SPDX license expression (e.g. `Apache-2.0`, `MIT OR GPL-3.0-only`) per component — the raw material for the compliance use case (Book 1, Chapter 8).
- **Cryptographic hashes.** SHA-256 (and often others) of the component artifact, which let a consumer verify that the thing described is the thing they received, and let a scanner match binaries by digest rather than by fragile name-guessing.
- **Provenance pointers.** Where the component came from (a download URL, a VCS reference, a source repository), and increasingly a pointer to a *build attestation* or signature (Book 5) so the SBOM can be tied back to a verifiable build.
- **Document metadata.** Who or what authored the SBOM, a timestamp, the tool and version that generated it, and a unique document identity.

Two of these deserve emphasis because they are where beginners under-invest.

Direct versus transitive. Your `package.json` or `go.mod` lists your *direct* dependencies — the ones you chose. The lockfile expands those into the *transitive* closure — everything they pulled in. A flat SBOM listing "we use these 800 packages" is strictly less useful than one that says "we directly depend on these 40, and here is the graph by which they pull in the other 760." When a CVE lands in a transitive package, the graph tells you *which of your direct choices to bump* to remediate — often a single version change several edges up resolves the vulnerable leaf. Without the graph you are left guessing, or upgrading blindly.

The graph is the value, not the list. This is the single most important framing in the chapter. The flat set of components answers "do we contain X?" The graph answers the follow-up questions that actually drive remediation: *why* do we contain X, *through what path*, *which team's service introduced it*, and *what is the smallest change that removes it*. Later chapters (especially Chapter 5) treat the SBOM fleet as a queryable graph database precisely because the graph structure is what makes it answer questions instead of just listing facts.

```
flowchart TD
    DOC["SBOM document
author · timestamp · tool · doc-id"]
    DOC --> ROOT["Root component
your-service:v3.2.0"]

    ROOT -->|depends on| A["web-framework 4.1.0
purl · CPE · Apache-2.0 · sha256"]
    ROOT -->|depends on| B["json-lib 2.9.0
purl · MIT · sha256"]

    A -->|depends on| C["http-client 3.0.1
purl · Apache-2.0"]
    C -->|depends on| D["tls-lib 1.4.2
purl · CPE · sha256"]
    D -->|depends on| E["log4j-core 2.14.1
purl · CPE · ← the CVE lives here"]

    B -->|depends on| E

    classDef vuln fill:#7f1d1d,stroke:#f87171,color:#fff;
    class E vuln;
```

Notice two things in that diagram. First, the vulnerable component (`log4j-core`) is reached by *two different paths* — through the framework and through the JSON library — which is exactly the situation that makes flat lists useless and graphs indispensable. Second, every leaf carries identifiers and a hash; the metadata is per-component, not a single blob for the whole document.

Types of SBOM: where and when you generate it

Here is a distinction that is routinely missed and that governs everything about SBOM accuracy. An SBOM is not a single kind of object. The same software produces *different* SBOMs depending on **which lifecycle stage you observe it at**, because different stages expose different information.

CISA's community work formalized six types, and they are worth internalizing because "the SBOM was wrong" is very often really "you generated the wrong *type* for the question you were asking."

```
flowchart LR
    D["Design SBOM
intended components
(planning / RFC)"]
    S["Source SBOM
declared deps
(manifests + lockfiles)"]
    B["Build SBOM
what the build
actually pulled in"]
    A["Analyzed SBOM
binary / image
inspection post-build"]
    DE["Deployed SBOM
what is installed
on the running host"]
    R["Runtime SBOM
what is actually
loaded / executed"]

    D --> S --> B --> A --> DE --> R

    D -.->|"earlier = cheaper,
more speculative"| D
    R -.->|"later = truer to reality,
harder to attribute to source"| R
```

- **Design SBOM.** Produced before the software exists, from an architecture or an RFC: "we intend to use these components." Speculative, but useful for procurement and early risk review.
- **Source SBOM.** Derived from the source repository — the manifests and lockfiles (`go.mod` / `go.sum` , `package-lock.json` , `poetry.lock`). It captures *declared* dependencies and their resolved versions. Cheap and accurate about *intent*, but blind to anything the build injects that is not in a manifest (a base image's OS packages, a vendored C library, a generated file).
- **Build SBOM.** Produced by or alongside the build, from the build graph itself: what the build *actually resolved and compiled in*. This is generally the highest-fidelity SBOM you can get about a released artifact, because it sees what the build system saw — including build-time-only dependencies and the exact resolution the build chose.

- **Analyzed SBOM.** Produced *after* the build by inspecting the finished artifact — a container image, a binary, a firmware blob — without access to the build's internal state. This is what a tool like Syft does when you point it at an image. It can find things the source never declared (OS packages baked into a base image), but it must *infer* identity from filenames, package databases, and hashes, so it is prone to both misses (a statically-linked library leaves few fingerprints) and misattributions.
- **Deployed SBOM.** What is actually installed in a running environment — the artifact *plus* its configuration and the host packages around it. Answers "what is on this machine," which can differ from "what we built" once operators patch, sidecar, or layer things on.
- **Runtime SBOM.** What is actually *loaded and executing* — observed by instrumenting the running process. It is the only type that can distinguish "present on disk" from "actually loaded into the process," which is the beginning (only the beginning) of the reachability question.

The general rule: **earlier stages are cheaper and more speculative; later stages are truer to production reality but harder to attribute back to a source change you can make.** A Source SBOM tells you what to fix in a pull request but might miss the OS package with the CVE. An Analyzed SBOM of the shipped image catches that OS package but cannot tell you which line of which manifest to edit. Mature programs generate *more than one type* and reconcile them. Chapter 4 is about the generation mechanics that produce each type and the accuracy trade-offs among them; Chapter 7 is about what every type still misses.

Why SBOMs: the problems they solve

The killer use case: rapid impact assessment

The single incident that moved SBOMs from standards-committee jargon into board-level mandates was **Log4Shell** — CVE-2021-44228, a remote-code-execution flaw in the ubiquitous `log4j-core` Java logging library, disclosed on 9–10 December 2021 (Book 1, Chapter 5). The vulnerability itself was severe, but what seared it into the industry's memory was the *response*. For most organizations, the first question after disclosure was embarrassingly simple and catastrophically hard to answer:

> *Which of our systems contain `log4j-core`, at what version, and where?*

Very few could answer it. Log4j is almost never a *direct* dependency — it arrives transitively, often shaded or repackaged inside other JARs, sometimes several edges deep inside a framework nobody remembers adding. Organizations spent **days to weeks** hand- auditing: grepping source trees, asking every team, unpacking JARs, chasing down vendors. Because the vulnerable code could be buried inside a fat JAR under a renamed package path, even "search for log4j" produced false negatives. The remediation effort was gated not by the difficulty of patching but by the difficulty of *finding*.

An SBOM inverts this. If every artifact you build carries an SBOM, and those SBOMs feed a central inventory (Chapter 5), then "which systems contain `log4j-core` and at what version" is a **query**, not an investigation — and it returns in seconds because the work of enumerating components was done *once, at build time*, and stored.

```
flowchart TB
    CVE["New CVE drops
e.g. Log4Shell (CVE-2021-44228)"]

    subgraph without["WITHOUT an SBOM inventory"]
        W1["Email every team:
'do you use log4j?'"] --> W2["Each team greps repos,
unpacks JARs, guesses"]
        W2 --> W3["Chase vendors for
their components"]
        W3 --> W4["Reconcile partial,
inconsistent answers"]
        W4 --> W5["Impact known:
days to weeks · gaps remain"]
    end

    subgraph with["WITH a per-artifact SBOM inventory"]
        S1["Query the inventory:
component = log4j-core"] --> S2["Return every service,
version, and build that
contains it — with graph paths"]
        S2 --> S3["Impact known:
minutes · complete + auditable"]
    end

    CVE --> W1
    CVE --> S1
```

This is the killer use case, and it is worth being precise about *why* it works: the inventory and the vulnerability data change on **different clocks**. You build your SBOM when your dependencies change (occasionally). The world discovers new vulnerabilities constantly, against code you shipped long ago. Decoupling the two — enumerate once, re-match forever — is the architectural heart of both SCA (Book 2, Chapter 6) and every SBOM program.

Vulnerability management at scale

The Log4Shell scenario generalizes. Any organization running hundreds of services faces a steady rain of new advisories against components it already ships. The scalable answer is **"scan the SBOM," not "re-scan the software."** Generate the component inventory once per build, store it, and every time a new advisory lands, re-match the *stored* inventory against the *updated* vulnerability database. You do not need to rebuild or re-fetch a five-year-old release to learn it is newly vulnerable; you need its SBOM and today's OSV feed. This decoupling is what makes vulnerability management tractable across a fleet, and it is why the SBOM is infrastructure rather than paperwork.

License compliance and legal risk

Every component carries a license, and licenses carry obligations — attribution, source-disclosure (copyleft), or outright incompatibility with how you ship. Discovering a strong-copyleft library deep in a proprietary product *after* release is an expensive surprise. Because a good SBOM records a license expression per component, license scanning becomes the same "match the inventory against a policy" operation as vulnerability scanning — one substrate, two consumers. Book 1, Chapter 8 covers the open-source licensing model this rests on.

Supply chain transparency and trust

SBOMs cut two ways. For software you **ship**, an SBOM is how you let *your* customers see what is inside — increasingly a contractual and regulatory expectation. For software you **buy or ingest**, a vendor-supplied SBOM is how you assess *their* risk without reverse-engineering their product: you can run it through your own scanners and policies before you deploy it. This is the core of third-party and vendor risk management (Book 8, Chapter 3). The transparency is only as good as the

SBOM's accuracy — a point Chapter 7 hammers — but even an imperfect SBOM beats a black box.

Incident response and forensics

When an incident hits — a compromised dependency, a newly-weaponized flaw, a suspicious build — responders need to reconstruct *what was actually running* at a point in time. Historical SBOMs, tied to specific build and deploy versions, are the forensic record that answers "which versions of which service contained the affected component, and when did we deploy them." Without that record, incident scoping degrades into the same frantic audit Log4Shell exposed. Book 8, Chapter 6 treats SBOM-driven incident response in depth.

M&A due diligence and procurement

Finally, SBOMs have become a due-diligence instrument. An acquirer evaluating a target's codebase, or a procurement team evaluating a vendor, can use SBOMs to assess license exposure, vulnerability debt, and dependency risk *before* signing — turning a qualitative "trust us, it's clean" into a reviewable artifact. Design-stage SBOMs feed this even before a product ships.

Why SBOMs are not a silver bullet

The industry's biggest SBOM mistake is treating the artifact as a *solution* rather than as *necessary infrastructure*. An SBOM is an **inventory**. Inventories enable security work; they are not themselves security. Chapter 7 is the full accounting; here is the honest preview, because setting expectations now prevents the disillusionment that follows a program built on the wrong premise.

- **An SBOM is not a security assessment.** It tells you what components are present. It does *not* tell you whether you are exploitable. "Contains `log4j-core` 2.14.1" and "is vulnerable to Log4Shell in a way an attacker can reach" are different claims; bridging them requires reachability analysis (Book 2, Chapter 7) and, for communicating the result, VEX (Chapter 6). A fleet-wide "contains" query without exploitability triage produces a mountain of findings, most of which are not actually risks — the alert- fatigue problem Book 2, Chapter 6 dissected.

- **An SBOM does not find malicious code.** It records the components you *have*, drawn from package metadata and build inputs. A backdoor smuggled into a component's build (the xz-utils implant, Book 1, Chapter 5) sits *inside* a component the SBOM faithfully lists as present and legitimate. SBOMs are about *known* components and *known* vulnerabilities; they are structurally blind to a novel implant. Conflating "I have an SBOM" with "I would catch malware" is a category error.
- **An SBOM is only as good as its generation.** Every accuracy caveat from the "types" section applies. An Analyzed SBOM of a container might miss a statically-linked library entirely; a Source SBOM might miss everything the base image contributes; a hand-maintained SBOM is stale the moment a dependency bumps. Incomplete and inaccurate SBOMs are the *common* case, not the exception, and a confidently wrong inventory can be worse than none because it manufactures false assurance. NTIA's own minimum-elements work explicitly includes "known unknowns" — the expectation that an SBOM should be able to *declare where it is incomplete* — precisely because completeness cannot be assumed.

None of this is an argument against SBOMs. It is an argument for treating them as the *substrate* on which reachability, VEX, provenance, and policy are built — not as the finish line. The rest of this book is largely about turning the substrate into something that answers real questions.

The regulatory and standards landscape

For most of the 2010s, SBOMs were a niche practice pushed by a handful of NTIA working- group participants. Two things changed that: a landmark supply-chain incident, and the regulation it triggered. What follows is the landscape as it stands, with dates stated precisely where they are firm and hedged where they are not. **Get the dates right** — this is a domain where a wrong year undermines everything, and where regulations are still phasing in as this is written (mid-2026).

```
flowchart LR
    SW["Dec 2020
SolarWinds disclosed"] --> EO["May 2021
US EO 14028"]
    EO --> NTIA["Jul 2021
NTIA Minimum
Elements for an SBOM"]
    NTIA --> M22["Sep 2022
OMB M-22-18
(self-attestation)"]
    M22 --> FDA["Dec 2022 / 2023
FDA §524B +
premarket guidance"]
    FDA --> M23["Jun 2023
OMB M-23-16
(extends M-22-18)"]
    M23 --> CISATYPES["2023
CISA 'Types of SBOM'"]
    CISATYPES --> CRA["Dec 2024
EU CRA in force
(obligations to ~2027)"]
    CRA --> REFRESH["2025
CISA minimum-elements
refresh (draft)"]
```

United States: the executive-order lineage

The proximate cause was **SolarWinds** — the compromise of the Orion build pipeline, disclosed in December 2020 (Book 1, Chapter 3), which pushed a trojanized update to thousands of organizations including US federal agencies. The federal response was **Executive Order 14028, "Improving the Nation's Cybersecurity," signed 12 May 2021.** EO 14028 is broad — it covers logging, zero trust, and incident response — but for our purposes its supply-chain section is what matters: it directed NIST to define secure software development practices and directed the production of guidance on providing a **Software Bill of Materials** to purchasers, explicitly naming the SBOM as a mechanism for software transparency to the federal government.

The EO delegated the definition of *what an SBOM must contain* to the Department of Commerce / NTIA, which published **"The Minimum Elements for a Software Bill of Materials (SBOM)" on 12 July**

2021. This document is the reference point every subsequent US requirement cites, and it is worth knowing its three-part structure:

1. **Data Fields** — the baseline information for each component: **Supplier Name, Component Name, Version of the Component, Other Unique Identifiers, Dependency Relationship, Author of the SBOM Data, and Timestamp.** (Notice this maps almost exactly onto the "core data" we described earlier — that is not a coincidence; the formats were designed to carry these fields.)
2. **Automation Support** — the SBOM must be produced in a machine-readable, automatable format. NTIA names three: **SPDX, CycloneDX, and SWID tags.** This is the moment the regulation *pins itself to the standards*, which is why Chapters 2 and 3 exist.
3. **Practices and Processes** — operational expectations: frequency (regenerate on each build or component change), depth (how far down the dependency tree), handling of **known unknowns** (declaring incompleteness), distribution and delivery, access control, and accommodation of mistakes. This third pillar is the one organizations most often ignore, and it is exactly the part that separates a checkbox SBOM from a useful one.

The minimum elements are a *floor*, deliberately modest — the goal in 2021 was to get adoption moving, not to demand perfection. Later chapters will show how far above this floor a genuinely operable program has to reach.

Enforcement flows through the **Office of Management and Budget (OMB)**, which translates EO direction into concrete obligations for federal agencies and, through them, their software vendors. Two memoranda matter:

- **OMB M-22-18** (14 September 2022), "*Enhancing the Security of the Software Supply Chain through Secure Software Development Practices,*" requires that software producers selling to the federal government **self-attest** to following NIST's Secure Software Development Framework (**SSDF, NIST SP 800-218** — Book 1, Chapter 7). The memo makes an SBOM an artifact an agency *may require* as part of that assurance rather than a blanket mandate for every purchase, but it firmly establishes attestation-plus-SBOM as the federal procurement posture.

- **OMB M-23-16** (9 June 2023) updates and extends M-22-18 — principally adjusting timelines and clarifying scope after industry feedback. The practical effect is the same direction of travel: self-attestation to SSDF, with SBOMs as supporting evidence.

CISA (the Cybersecurity and Infrastructure Security Agency) carries the ongoing community work. Beyond convening the SBOM working groups, CISA published the **"Types of Software Bill of Materials (SBOM)"** document (2023) that gives us the Design/Source/ Build/Analyzed/ Deployed/Runtime taxonomy used earlier in this chapter. As of this writing CISA has been running a **refresh of the minimum-elements guidance** — a draft updating the 2021 NTIA baseline circulated for public comment in 2025, reflecting several years of implementation experience (for instance, tightening expectations around component identifiers and completeness). Treat the specifics of that refresh as *in flux*: the direction is toward a stricter, more prescriptive baseline, but the exact finalized fields should be checked against the published version rather than assumed.

European Union: the Cyber Resilience Act

The EU's contribution is structurally different from the US approach. Where the US routed requirements through *federal procurement* (comply if you want to sell to the government), the EU's **Cyber Resilience Act (CRA) — Regulation (EU) 2024/2847** — is *product regulation*: it applies to essentially any "product with digital elements" placed on the EU market, government customer or not, backed by CE-marking and market-surveillance enforcement.

The dates, stated carefully: the CRA **entered into force in December 2024** (twenty days after its publication in the Official Journal in late November 2024). Its obligations **phase in over roughly the following three years**, with the bulk of the substantive manufacturer obligations applying from **late 2027** and certain earlier obligations (notably vulnerability and incident reporting) applying sooner. Because the phase-in is still ahead as of mid-2026, describe the CRA as *in force but not yet fully applicable* — that is the accurate state.

On SBOMs specifically: the CRA makes the SBOM an **explicit expectation**. Manufacturers must, among their cybersecurity obligations, **identify and document the components in their products — including by drawing up a software bill of materials in**

a commonly used, machine-readable format — and must handle vulnerabilities across the product's support period. A nuance worth getting right: the CRA (in its core text) requires the SBOM to be *produced and maintained* and made available to market-surveillance authorities on request, and to cover *at least the top-level dependencies* of the product; it does **not** by default require handing a full SBOM to every end customer. Implementing standards and guidance (developed under bodies such as ENISA and the European standards organizations) are expected to sharpen the format and depth expectations over the phase-in period, so the operational detail here is still settling.

The strategic point: the CRA globalizes the pressure. A US company selling connected products into Europe inherits SBOM obligations regardless of any US mandate, which is a large part of why SBOM production is becoming table stakes rather than a niche compliance task.

Sector-specific mandates

Two sectors moved faster and more concretely than the horizontal rules, and both are worth knowing because they are *already enforced*, not phasing in.

- **Medical devices (US FDA).** The Consolidated Appropriations Act, 2023 (the "omnibus," signed December 2022) added **section 524B to the Food, Drug, and Cosmetic Act**, requiring cybersecurity information for "cyber devices" in premarket submissions — **including an SBOM.** The FDA finalized its guidance, "*Cybersecurity in Medical Devices: Quality System Considerations and Content of Premarket Submissions,*" in **September 2023**, and began exercising **"refuse to accept" (RTA)** authority for non-compliant cyber-device submissions from **1 October 2023**. This is the sharpest teeth in the landscape: no compliant SBOM (among the other cyber requirements), no market authorization. If you build software that ends up in a regulated medical device, the SBOM is not optional and not future-tense.
- **Defense and automotive.** The US Department of Defense has folded SBOM expectations into its acquisition and software-assurance guidance, aligned with the same NTIA baseline. In automotive, the security-process standards (notably **ISO/SAE 21434** for road-vehicle cybersecurity, alongside UN Regulation No. 155) drive component-inventory and vulnerability-management practices

that SBOMs directly support, even where "SBOM" is not the literal word in the standard. Treat these as *converging on the same substrate* rather than as separate demands.

The standards the regulations lean on

Every regulation above stops short of inventing a file format — deliberately. They point at **existing, standardized SBOM formats**, which is what makes cross-jurisdiction compliance tractable: produce one good SBOM in a recognized format and it satisfies many masters. The three named in NTIA's automation-support pillar, each now backed by a formal standards body:

Format	Steward	Standardization	Origin / character
SPDX	Linux Foundation	**ISO/IEC 5962:2021** (standardized SPDX 2.2.1); SPDX **3.0** released 2024	Started as a *license-compliance* format; broad, expressive, relationship-rich. Deep dive in **Chapter 2**.
CycloneDX	OWASP	**ECMA-424** (1st edition, 2024, covering CycloneDX **1.6**)	Started as a *security/BOM* format; compact, security-focused, native VEX and VDR support. Deep dive in **Chapter 3**.
SWID tags	ISO	**ISO/IEC 19770-2:2015**	Software *identification* tags from IT asset management; narrower, used more as an identifier source than a full SBOM.

A note on the identifiers these formats carry, which we met earlier: **purl** (Package URL) and **CPE** (NIST's Common Platform Enumeration, current version 2.3) are not SBOM formats — they are *component-identity schemes* that the formats embed. Book 2, Chapter 5 is the reference for why they matter and why they disagree; here it is enough to know that a well-formed SBOM carries them so downstream tooling can match reliably.

The takeaway for a practitioner: the regulations converge on **"emit a machine-readable SBOM in SPDX or CycloneDX, carrying the NTIA minimum fields, per artifact, kept current."** That sentence is the compliance kernel underneath all the memoranda and articles.

Distributed-systems lens

Everything above becomes concrete — and much harder — the moment you stop imagining "an SBOM" and start imagining **an organization's SBOMs**. For a company running hundreds of services across dozens of teams with high deploy frequency, the SBOM story is not a document; it is a **data pipeline and an inventory system**.

The first thing to internalize is that **a single flat SBOM of "the company" is meaningless.** There is no such artifact and there should not be. Software is versioned and deployed per-artifact; its component inventory is a property of *a specific build of a specific service*, and it changes every time that service's dependencies change. So the unit of SBOM generation is **per-artifact, per-build**: every image, binary, or package your CI produces emits its own SBOM at build time, tagged with the exact version it describes. Ten services deploying twenty times a day produce a *stream* of SBOMs, not a document you maintain.

The second thing is that those per-build SBOMs are only valuable if they flow into a **central, queryable inventory** — a fleet-wide store you can ask questions of. This is the architecture that turns the Log4Shell capability from aspiration into reality:

```
flowchart TB
    subgraph CI["CI/CD — per artifact, per build"]
        B1["build svc-A v3.2 → SBOM"]
        B2["build svc-B v1.9 → SBOM"]
        B3["build svc-C v7.0 → SBOM"]
    end

    B1 --> STORE[("Central SBOM inventory
graph-queryable · versioned")]
    B2 --> STORE
    B3 --> STORE

    DEPLOY["Deploy tracking:
which build runs where"] --> STORE

    STORE --> Q1["'Where is log4j-core
across everything we run?'"]
    STORE --> Q2["'Which shipped versions
carry GPL code?'"]
    STORE --> Q3["Feed SCA / VEX / policy
(Book 2 Ch 6, Ch 6 here)"]
```

Three engineering realities fall out of this picture, each of which later chapters develop:

- **Generation must be automated and in-pipeline, not a manual step.** At fleet scale nobody hand-writes SBOMs; they are emitted by the build (Chapter 4). The generation must be cheap enough to run on every build and accurate enough to trust — and, as the "types" discussion warned, you often want *both* a Source/Build SBOM (accurate about intent, attributable to a PR) *and* an Analyzed SBOM of the final image (catches what the base image contributed).
- **Storage and query is a real systems problem.** Millions of SBOM documents, each a graph, versioned over time, is a substantial data-management challenge — dedup, indexing by component, joining against a moving vulnerability feed, and answering the "where is X" query in seconds. This is not a file share; it is a database. Chapter 5 is entirely about it, and it is where the "SBOM as graph, not list" framing pays off.
- **Deploy tracking closes the loop.** An SBOM inventory tells you what *builds* contain a component. To answer "where is it *running*," you must join that against deployment state — which build is live in which cluster, region, and namespace. The inventory plus the deploy

map is what makes impact assessment *complete* rather than merely *approximate*.

Finally, the lens that reorganizes the whole book: **SBOMs for what you SHIP versus SBOMs for what you RUN.** These are two consumers of the same substrate with different goals.

- The SBOM for what you **ship** is *outbound* — it goes to customers, auditors, and regulators (the CRA, the FDA, a procurement questionnaire). It must be complete, well-formed, and defensible, because someone external will judge it. Its consumer is a *third party assessing you*.
- The SBOM for what you **run** is *inbound and operational* — it feeds *your own* vulnerability management, the Log4Shell query, your policy gates. Its consumer is *your own security and platform teams*, and it must cover not just your first-party code but everything you deploy, including third-party and vendor software running in your estate.

Same generation machinery, same formats, same inventory — but the outbound case is driven by *compliance and trust* and the inbound case by *operational risk reduction*. A program that builds only one and calls it done will find the other stakeholder unserved: ship-only SBOMs satisfy the auditor while your own responders still cannot answer "where is X," and run-only SBOMs let you respond fast while your customers' procurement teams still treat you as a black box. The rest of this book builds the machinery that serves both from one substrate.

Key takeaways

- An **SBOM is a formal, machine-readable inventory of a software's components, their relationships, and identifying metadata.** The "ingredients list" analogy sells the transparency value but hides three truths: software ingredients nest into a deep graph, the label is only as accurate as the tool that generated it, and "contains X" is not "exploitable via X."
- The **dependency graph — not the flat list — is the real deliverable.** The interesting component is almost always transitive, and the graph is what tells you *which direct choice to change* to remediate it.
- **Where and when you generate an SBOM determines what it can see.** CISA's six types (Design, Source, Build, Analyzed,

Deployed, Runtime) trade earlier-and-speculative against later-and-truer; mature programs generate more than one type and reconcile them.

- The **killer use case is rapid impact assessment.** Log4Shell (CVE-2021-44228, December 2021) turned "which systems contain this component?" from a weeks-long audit into a seconds-long query — *if* you have a per-artifact SBOM inventory. This works because inventory and vulnerability data change on decoupled clocks: enumerate once, re-match forever.
- SBOMs also underwrite **vulnerability management at scale, license compliance, vendor and procurement risk, incident-response forensics, and M&A due diligence** — different consumers of one substrate.
- An **SBOM is necessary infrastructure, not a security solution.** It is not an exploitability verdict (needs reachability and VEX), does not detect malicious code, and is only as good as its generation. Incomplete and inaccurate SBOMs are the common case.
- The **regulatory drivers, in order:** SolarWinds (Dec 2020) → US **EO 14028** (May 2021) → **NTIA Minimum Elements** (July 2021, defining the baseline data fields, format support, and practices) → OMB **M-22-18** (2022) and **M-23-16** (2023) tying procurement to SSDF self-attestation → **EU CRA** (Regulation 2024/2847, in force Dec 2024, obligations phasing to ~2027, SBOM an explicit expectation) → **FDA §524B** and 2023 premarket guidance with RTA enforcement from Oct 2023. A **CISA minimum-elements refresh** was in draft in 2025 — treat its specifics as still settling.
- Regulations lean on **standardized formats**: **SPDX** (ISO/IEC 5962:2021; 3.0 in 2024), **CycloneDX** (OWASP; ECMA-424 in 2024), and **SWID** (ISO/IEC 19770-2:2015), carrying identity schemes **purl** and **CPE**. Deep dives follow in Chapters 2 and 3.
- **At scale, SBOMs are a pipeline, not a document:** generated per-artifact per-build, fed into a central queryable graph inventory, joined against deploy state to answer "where is X across everything we run." And they serve two masters — **what you ship** (outbound, compliance and trust) and **what you run** (inbound, your own vulnerability management) — from the same substrate.

Further reading

- **Executive Order 14028**, "Improving the Nation's Cybersecurity," The White House, 12 May 2021.
- **NTIA**, "The Minimum Elements for a Software Bill of Materials (SBOM)," US Department of Commerce, 12 July 2021.
- **OMB M-22-18**, "Enhancing the Security of the Software Supply Chain through Secure Software Development Practices," 14 September 2022; and **OMB M-23-16** (9 June 2023).
- **CISA**, "Types of Software Bill of Materials (SBOM)," 2023; and CISA's SBOM resource hub (`cisa.gov/sbom`) for the ongoing minimum-elements refresh and community documents.
- **Regulation (EU) 2024/2847** (the Cyber Resilience Act), Official Journal of the European Union, 2024; and ENISA guidance on CRA implementation.
- **FDA**, "Cybersecurity in Medical Devices: Quality System Considerations and Content of Premarket Submissions," final guidance, September 2023; and FD&C Act §524B.
- **NIST SP 800-218**, "Secure Software Development Framework (SSDF) Version 1.1," 2022.
- **ISO/IEC 5962:2021** (SPDX 2.2.1) and the **SPDX 3.0** specification, Linux Foundation.
- **ECMA-424**, "CycloneDX Bill of Materials Specification," Ecma International, 2024; and the OWASP CycloneDX project documentation.
- **ISO/IEC 19770-2:2015** (SWID tags); the **Package URL (purl)** specification; and **NIST CPE 2.3**.
- **CVE-2021-44228** (Log4Shell) — the Apache Log4j security advisory and the CISA log4j guidance, for the incident that motivated the killer use case.

Chapter 2 — SPDX in Depth

What this chapter covers. Chapter 1 argued *why* SBOMs exist and *what* they must carry: a machine-readable inventory of components, their relationships, and enough metadata to identify each one unambiguously.

This chapter is the first of two that take apart the wire formats those requirements actually travel in. It is the definitive technical treatment of **SPDX** — the Software Package Data Exchange — the older of the two dominant formats, an ISO standard, and the one most likely to land on your desk when a government or enterprise customer asks for an SBOM. We treat SPDX at the level a systems engineer needs: the object model, the exact field names, the relationship graph, the serializations on the wire, the license-expression grammar that SPDX contributed to the whole industry, and the 2.x-versus- 3.0 fork that every organization now has to reason about. By the end you should be able to read an SPDX document and reconstruct the dependency graph in your head, produce a valid one, and make a defensible call about which version your program emits.

Learning goals — after this chapter you should be able to:

- Place SPDX historically and legally: a **2010 Linux Foundation license-compliance project** that became a full SBOM format and then **ISO/IEC 5962:2021** (standardizing SPDX 2.2.1), with **SPDX 3.0** released in 2024 as a ground-up redesign.
- Navigate the **SPDX 2.2/2.3 object model** — Document Creation Information, Packages, Files, Snippets, Relationships, Other Licensing Info, Annotations — and name the load-bearing fields (`SPDXID`, `documentNamespace`, `PackageLicenseConcluded` vs `PackageLicenseDeclared`, `ExternalRef`, checksums).
- Read and write the **relationship graph** — `DESCRIBES`, `CONTAINS`, `DEPENDS_ON`, `GENERATED_FROM` — and understand why SPDX's relationship model is more expressive than a flat dependency list.
- Choose among SPDX's **serializations** (tag-value, JSON, YAML, RDF/XML) and read a correct SPDX 2.3 JSON document without a reference open.
- Write **SPDX license expressions** correctly: the `AND` / `OR` / `WITH` grammar, the `+` operator, `LicenseRef-`, and `NOASSERTION` / `NONE` — a mini-standard reused far outside SPDX.
- Explain what **SPDX 3.0** changed — the core-plus-profiles class model, the element/ relationship graph, JSON-LD serialization, and the Security/Build/AI/Dataset profiles — and why 2.3 remains the production reality.
- Weigh SPDX's **strengths and weaknesses** honestly, setting up a fair comparison with CycloneDX in Chapter 3.

This chapter assumes Chapter 1's vocabulary (purl, CPE, SWID, the NTIA minimum elements, the six SBOM types) and Book 2's identifier model. It stays inside the format; *generating* SPDX from a build is Chapter 4, *storing and querying* thousands of them is Chapter 5, and the head-to-head with CycloneDX is Chapter 3.

Where SPDX came from

SPDX did not start life as an SBOM format, and understanding that origin explains most of its shape. The project began in **2010** under the **Linux Foundation** as a working group with a narrow, painful problem: **open-source license compliance**. Companies shipping products built on open-source code needed to communicate, machine-to-machine, exactly which licenses applied to which files and packages — the "attribution and copyleft" obligations that Book 1, Chapter 8 covers. Doing this by hand, in prose notices and spreadsheets, did not scale across a supply chain. SPDX was the interchange format for that problem: a way to say "this package is under Apache-2.0, this file was concluded to be MIT despite a declared license of GPL, here is the copyright text" in a form a program could consume.

That heritage is why, even today, SPDX is unusually rigorous about *licensing* — the **SPDX License List**, the **license-expression grammar**, and the concluded-versus-declared distinction are all more developed than in any competing format, because licensing was the original mission, not a bolt-on.

Over the 2010s the format grew. The same machinery that described a package's license could describe its supplier, its download location, its checksums, its files, and — crucially — its **relationships** to other packages. By the SPDX 2.x line, the format had become a general-purpose software bill of materials: still license-fluent, but now able to carry the full component inventory and dependency graph that Chapter 1 described. The industry's convergence on SBOMs (Log4Shell, EO 14028, the NTIA minimum elements) met a format that was already sitting there, mature and standardized.

The standardization milestone matters for procurement conversations, so state it precisely: **SPDX 2.2.1 was published as ISO/IEC 5962:2021.** When a contract says "provide an SBOM in an ISO-standard format," SPDX is the format it is usually pointing at. Note the exact mapping: the

ISO standard corresponds to **2.2.1**, a maintenance revision of 2.2. The widely deployed line then moved on to **SPDX 2.3** (2022), which added fields and refinements while staying backward-compatible with 2.2. Most tooling today emits **2.3**.

Then, in **2024**, the project shipped **SPDX 3.0** — not an incremental update but a ground-up redesign of the data model, described in its own section below. The result is a genuine fork in the road: **2.3 is what production tooling emits and what your customers can consume today; 3.0 is where the standard is going but is still early in adoption.** A practitioner needs both in their head, so this chapter covers both, with the weight on 2.3 because that is what you will actually handle in 2026.

```
flowchart LR
    A["2010
LF working group
license compliance"] --> B["2011-2020
SPDX 1.x - 2.1
grows into full SBOM"]
    B --> C["2020
SPDX 2.2"]
    C --> D["2021
SPDX 2.2.1
= ISO/IEC 5962:2021"]
    D --> E["2022
SPDX 2.3
widely deployed line"]
    E --> F["2024
SPDX 3.0
redesign: core + profiles"]
```

The SPDX 2.2/2.3 object model

An SPDX 2.x document is a single logical object with a fixed top-level shape. Whatever the serialization — tag-value, JSON, YAML, RDF — the same conceptual pieces are present. There are seven kinds of thing in a 2.x document:

1. **Document Creation Information** — metadata about the document itself.
2. **Packages** — the components: libraries, applications, containers, archives.

3. **Files** — individual files, when the document describes contents at file granularity.
4. **Snippets** — sub-file regions (byte or line ranges), for when a fragment of a file has a different license or origin than the file around it.
5. **Relationships** — the edges connecting all of the above into a graph.
6. **Other Licensing Information** — definitions for license identifiers not on the SPDX License List (the `LicenseRef-` entries).
7. **Annotations** — free-form review comments attached to any element.

```
flowchart TD
    DOC["SPDX Document
Creation Information
spdxVersion - dataLicense - SPDXID
documentNamespace - creators - created"]

    DOC -->|DESCRIBES| PKGROOT["Package (root)
SPDXRef-Package-app"]
    DOC --> PKGS["Packages[]
each: SPDXID, name, versionInfo,
supplier, downloadLocation,
licenseConcluded / licenseDeclared,
checksums, externalRefs"]
    DOC --> FILES["Files[]
SPDXID, fileName,
checksums, licenseInfoInFile"]
    DOC --> SNIP["Snippets[]
byte/line ranges
within a File"]
    DOC --> OLI["Other Licensing Info[]
LicenseRef- definitions"]
    DOC --> ANN["Annotations[]
review comments"]

    PKGS -->|CONTAINS| FILES
    FILES --> SNIP
    PKGS -.->|DEPENDS_ON| PKGS

    REL["Relationships[]
the graph edges"] --- DOC
```

Document Creation Information

Every SPDX document opens with a header identifying the document, not its contents. These fields are mandatory and a validator will reject a

document missing them. In tag-value form the field names are literal; in JSON they map to camelCase keys. The essentials:

- **`SPDXVersion`** (JSON: `spdxVersion`) — the format version, e.g. `SPDX-2.3`. This is the first thing a parser branches on.
- **`DataLicense`** (JSON: `dataLicense`) — the license of the SBOM *data itself*, and it is fixed: SPDX **requires** **`CC0-1.0`** here. This is deliberate. The metadata about your software must be freely shareable so that consumers, scanners, and regulators can pass it around without a licensing entanglement of its own. Do not put your product's license here; this field is about the document, not the software.
- **`SPDXID`** — the identifier of the document element itself, always `SPDXRef-DOCUMENT`.
- **`DocumentName`** (JSON: `name`) — a human-readable name for the document.
- **`DocumentNamespace`** (JSON: `documentNamespace`) — a globally unique URI for *this document instance*, typically a URL under your control plus a UUID, e.g. `https://acme.example/spdx/checkout-service-3.2.0-6f0a...`. This is the anchor that makes `SPDXRef-` identifiers globally unique: an `SPDXID` is only unique *within* a document, and the namespace plus the `SPDXID` together form a globally unique reference. At fleet scale (Chapter 5) this matters enormously — it is how you tell two `SPDXRef-Package-libc` entries in two different documents apart.
- **`Creator`** (JSON: `creationInfo.creators`) — who or what produced the document. SPDX distinguishes three creator types, and you can list several: `Tool: syft-1.18.1`, `Organization: Acme Corp`, `Person: Jane Roe`. The **tool** creator is what tells a downstream consumer *how* the SBOM was generated — which, per Chapter 1's "an SBOM is only as good as its generation," is essential provenance.
- **`Created`** (JSON: `creationInfo.created`) — an ISO-8601 UTC timestamp. Together with the namespace, this is what makes an SBOM a point-in-time forensic record.

An optional but important sub-field is **`licenseListVersion`** — which version of the SPDX License List the concluded/declared identifiers were validated against. Because the license list evolves (identifiers get added,

a few get deprecated), recording the list version lets a consumer interpret the expressions correctly.

Packages

The **Package** is the workhorse element — in most SBOMs, packages are nearly the whole document. A package is any unit of software you want to inventory: an application, a library, a container image, an OS package, a source archive. The fields that carry the Chapter 1 "core data" (using the tag-value names, with JSON keys noted):

- **`SPDXID`** — a document-local identifier of the form `SPDXRef-<something>`. The suffix is arbitrary but must match `[a-zA-Z0-9.-]+` and be unique in the document; tools typically generate `SPDXRef-Package-<name>-<hash>`.
- **`PackageName`** (JSON: `name`) and **`PackageVersion`** (JSON: `versionInfo`) — the human identity.
- **`PackageSupplier`** (JSON: `supplier`) — who supplied it, as `Organization: ...`, `Person: ...`, or `NOASSERTION`. Distinct from **`PackageOriginator`** (`originator`), which is who *originally created* it; supplier is who *you got it from*. This is one of the NTIA minimum "Supplier Name" fields.
- **`PackageDownloadLocation`** (JSON: `downloadLocation`) — where the package can be obtained: a URL, a VCS locator (`git+https://...@<commit>`), `NONE`, or `NOASSERTION`. This field is **mandatory** — a package with no download location must still say `NOASSERTION` explicitly, which encodes the "known unknowns" discipline Chapter 1 stressed.
- **`PackageLicenseConcluded`** (JSON: `licenseConcluded`) and **`PackageLicenseDeclared`** (JSON: `licenseDeclared`) — the two-license model, dissected below.
- **`PackageChecksum`** (JSON: `checksums`) — one or more digests, each an `{algorithm, checksumValue}` pair. `SHA256` is standard; `SHA1` appears for git-object compatibility, `SHA512` and `MD5` are also permitted. Checksums are what let a consumer verify the described artifact is the received artifact, and what let a scanner match by digest rather than by fragile name-guessing.
- **`FilesAnalyzed`** (JSON: `filesAnalyzed`) — a boolean that changes the document's meaning. When `true`, the document is asserting it

enumerated the package's files and the package's license was derived from them, which requires a **`PackageVerificationCode`** (a hash over the set of contained files). When `false` — the common case for a dependency you did not unpack — the document is describing the package as an opaque unit and the verification code is omitted. Getting this flag wrong is a frequent validation failure.

- **`ExternalRef`** (JSON: `externalRefs`) — the crucial carrier of machine identifiers, covered in its own subsection.

PackageLicenseConcluded versus PackageLicenseDeclared

This distinction is the clearest fingerprint of SPDX's license-compliance heritage, and it is genuinely useful, so understand it precisely.

- **`PackageLicenseDeclared`** is the license the package **claims about itself** — what the upstream author put in the `LICENSE` file, the `package.json` `"license"` field, the Maven POM, the gem spec. It is the *declared intent* of the producer.
- **`PackageLicenseConcluded`** is the license the **SBOM author concludes actually applies** after analysis. This can differ from the declared license: a scanner might find a GPL header inside a package that declares MIT, a legal reviewer might resolve an ambiguous dual-license down to one, or an automated tool that does not do license analysis might set concluded to `NOASSERTION` while faithfully copying the declared field.

The two fields exist because **the producer's claim and the consumer's determination are different facts**, and a compliance process needs both: the declared license is evidence, the concluded license is a judgment. A tool like Syft, which reads package metadata but does not perform deep license analysis, will typically populate `licenseDeclared` from the manifest and set `licenseConcluded` to `NOASSERTION` — correctly declining to assert a conclusion it did not make. A dedicated license scanner (FOSSology, ScanCode) is what fills in a meaningful concluded license. When you consume an SBOM, know which field you are reading: treating a tool's `NOASSERTION` concluded license as "no license" is a misread.

ExternalRefs: how SPDX carries coordinates and security identifiers

A package's `PackageName` and `PackageVersion` are human strings; they are not reliable machine keys. The **`ExternalRef`** mechanism is how SPDX attaches the *stable, matchable identifiers* that make an SBOM queryable — the purl, CPE, and SWID that Chapter 1 and Book 2 established as the identity substrate. Each external reference has three parts:

- **`referenceCategory`** — one of `PACKAGE-MANAGER`, `SECURITY`, `PERSISTENT-ID`, or `OTHER`.
- **`referenceType`** — the specific scheme: `purl`, `cpe23Type` (and legacy `cpe22Type`), `swid`, `gitoid`, `advisory`, and others.
- **`referenceLocator`** — the actual value.

The two you will see most:

- **purl** under `PACKAGE-MANAGER`: `pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1`. This is the identifier that maps cleanly to package registries and to OSV, and it is what most modern matching keys on.
- **CPE** under `SECURITY`, type `cpe23Type`: `cpe:2.3:a:apache:log4j:2.14.1:*:*:*:*:*:*:*`. This is NIST's identifier, the one NVD keys its vulnerability records on.

Carrying **both** is what makes an SPDX package robust for downstream security work, because — as Book 2, Chapter 5 explained at length — purl and CPE produce *different* match results against different databases, and a package that offers only one identifier is invisible to tooling keyed on the other. The `SECURITY` category can also carry an `advisory` reference pointing at a specific vulnerability disclosure, though SPDX 2.x has no native vulnerability *model* — a limitation we return to when comparing with CycloneDX.

Files, Snippets, and Annotations

Below the package level, SPDX can describe individual **Files** — each with an `SPDXID` (`SPDXRef-File-...`), a `fileName`, checksums, and `licenseInfoInFile` (the licenses actually found in that file). File-level detail is where SPDX's compliance heritage shines and where it gets verbose: a source-scanned SPDX document for a large project can enumerate tens of thousands of files. Most *dependency* SBOMs set `filesAnalyzed: false` on packages and skip file enumeration entirely;

file-level SPDX is characteristic of source-code license scans, not of build-time dependency inventories.

Snippets go finer still: a snippet identifies a byte range or line range *within* a file that has a different license or origin than its surroundings — the classic case being a chunk of GPL code copy-pasted into an otherwise-permissive file. Snippets are powerful for provenance forensics and almost never present in machine-generated dependency SBOMs.

Annotations are free-text review notes (`REVIEW` or `OTHER`) with an annotator and timestamp, attachable to any element. They are how a human reviewer records "checked this license conclusion on 2024-03-01" inside the document.

The relationship graph

If Chapter 1's central claim is "the graph is the value, not the list," then **Relationships** are where SPDX earns that claim. A relationship is a typed, directed edge:

```
Relationship: <spdxElementId> <RELATIONSHIP_TYPE> <relatedSpdxElement>
```

read as "`spdxElementId` *is/has* `RELATIONSHIP_TYPE` *toward* `relatedSpdxElement`." SPDX 2.3 defines a large vocabulary of relationship types — several dozen — which is exactly why the model is more expressive than a flat parent-child dependency tree. The ones you must know:

- **`DESCRIBES`** — the document points at its top-level subject(s). Every well-formed SPDX document has a `SPDXRef-DOCUMENT DESCRIBES SPDXRef-Package-<root>` relationship (or the shorthand `documentDescribes` array). This answers "what is this SBOM *about*," which is not otherwise obvious in a document that may list hundreds of packages. Without it, a consumer cannot tell the root application from its 500th transitive dependency.
- **`CONTAINS`** — a containment edge: a package contains a file, or a container image contains an OS package. This models physical/archival inclusion, distinct from a build dependency.
- **`DEPENDS_ON`** (and its inverse `DEPENDENCY_OF`) — the dependency edge that encodes the graph Chapter 1 cares about. `A DEPENDS_ON B`

means A needs B. This is what a "what depends on X" query traverses.

- **`GENERATED_FROM`** (and inverse `GENERATES`) — a provenance edge: a compiled artifact was generated from a source file. `libfoo.so GENERATED_FROM foo.c`.
- Others you will meet: **`BUILD_DEPENDENCY_OF`**, **`DEV_DEPENDENCY_OF`**, **`OPTIONAL_DEPENDENCY_OF`**, **`RUNTIME_DEPENDENCY_OF`** (dependency scoping), **`PATCH_APPLIED`**, **`COPY_OF`**, **`DYNAMIC_LINK`** / **`STATIC_LINK`**, and **`VARIANT_OF`**. The linking relationships matter for license reasoning (static linking of copyleft code has different obligations than dynamic).

Because relationships are first-class objects rather than nested structure, the *same* package element can participate in many relationships — a diamond dependency where one library is reached by two paths is represented naturally, with two `DEPENDS_ON` edges pointing at one package element. This is the structural advantage over formats that nest dependencies as a tree: SPDX represents a true DAG.

```
flowchart TD
    DOC["SPDXRef-DOCUMENT"]
    APP["SPDXRef-Package-app
checkout-service 3.2.0"]
    WEB["SPDXRef-Package-web
web-framework 4.1.0"]
    JSONLIB["SPDXRef-Package-json
json-lib 2.9.0"]
    TLS["SPDXRef-Package-tls
tls-lib 1.4.2"]
    LOG4J["SPDXRef-Package-log4j
log4j-core 2.14.1"]

    DOC -->|DESCRIBES| APP
    APP -->|DEPENDS_ON| WEB
    APP -->|DEPENDS_ON| JSONLIB
    WEB -->|DEPENDS_ON| TLS
    WEB -->|DEPENDS_ON| LOG4J
    JSONLIB -->|DEPENDS_ON| LOG4J

    classDef vuln fill:#7f1d1d,stroke:#f87171,color:#fff;
    class LOG4J vuln;
```

That diagram is a literal picture of a relationships array: one `DESCRIBES` edge and five `DEPENDS_ON` edges, with `log4j-core` reached by two paths

— the exact situation that makes the graph indispensable and a flat list useless.

Serializations

SPDX 2.x defines the *same logical model* in several concrete file formats. All five are "real" SPDX; they differ only in encoding, and a conformant tool can round-trip between them.

```
flowchart LR
    MODEL["SPDX 2.3
logical model
(Document / Packages /
Files / Relationships)"]
    MODEL --> TV[".spdx
tag-value
original, human-writable"]
    MODEL --> JSON[".spdx.json
JSON
dominant in practice"]
    MODEL --> YAML[".spdx.yaml
YAML"]
    MODEL --> RDF[".spdx.rdf / .rdf.xml
RDF/XML
original semantic-web form"]
```

- **Tag-value (`.spdx`)** is the original format: line-oriented `Tag: Value` pairs, grouped by element. It is the most human-readable and the form the SPDX spec uses in its examples. The literal tag names (`PackageLicenseConcluded`, `SPDXID`, `Relationship`) come from here.
- **JSON (`.spdx.json`)** is what the ecosystem has consolidated on. It is what Syft, the official SPDX tools, and virtually all modern tooling emit and consume by default, because JSON is trivial to parse and store in the kind of central inventory Chapter 5 builds. When someone says "an SPDX file" in 2026 without qualification, assume `.spdx.json`.
- **YAML (`.spdx.yaml`)** is a straightforward re-encoding of the JSON model — same keys, more human-editable, occasionally used in CI config contexts.
- **RDF/XML (`.spdx.rdf`)** is the original machine format, reflecting SPDX's early semantic-web design. It is verbose and rarely produced

by hand today, but it is why SPDX has an underlying ontology at all — a lineage that becomes central in 3.0's JSON-LD.

A correct SPDX 2.3 JSON document

Here is a small but complete and valid SPDX 2.3 JSON document describing the graph above: `checkout-service 3.2.0` depending on `web-framework 4.1.0` and `json-lib 2.9.0`, with the `log4j-core` leaf reached by two paths. Trimmed to two dependency packages for space, it shows every structural piece — creation info, packages with the key fields, external refs, and the relationships array.

```
{
  "spdxVersion": "SPDX-2.3",
  "dataLicense": "CC0-1.0",
  "SPDXID": "SPDXRef-DOCUMENT",
  "name": "checkout-service-3.2.0",
  "documentNamespace": "https://acme.example/spdx/checkout-
service-3.2.0-6f0a2e1c",
  "creationInfo": {
    "created": "2026-07-31T14:12:03Z",
    "creators": [
      "Tool: syft-1.18.1",
      "Organization: Acme Corp"
    ],
    "licenseListVersion": "3.25"
  },
  "documentDescribes": ["SPDXRef-Package-app"],
  "packages": [
    {
      "SPDXID": "SPDXRef-Package-app",
      "name": "checkout-service",
      "versionInfo": "3.2.0",
      "supplier": "Organization: Acme Corp",
      "downloadLocation": "git+https://git.acme.example/
checkout.git@v3.2.0",
      "filesAnalyzed": false,
      "licenseConcluded": "Apache-2.0",
      "licenseDeclared": "Apache-2.0",
      "copyrightText": "Copyright 2026 Acme Corp",
      "checksums": [
        { "algorithm": "SHA256",
          "checksumValue": "3b1c9d0e5a7f4c2b8e6d1a0f9c3b5e7d2a4f6c8b0e1
d3a5f7c9b1e3d5a7f9c1b" }
      ],
      "externalRefs": [
        { "referenceCategory": "PACKAGE-MANAGER",
          "referenceType": "purl",
          "referenceLocator": "pkg:generic/checkout-service@3.2.0" }
      ]
    },
    {
      "SPDXID": "SPDXRef-Package-log4j",
      "name": "log4j-core",
      "versionInfo": "2.14.1",
      "supplier": "Organization: Apache Software Foundation",
      "downloadLocation": "https://repo1.maven.org/maven2/org/apache/
logging/log4j/log4j-core/2.14.1/log4j-core-2.14.1.jar",
      "filesAnalyzed": false,
      "licenseConcluded": "Apache-2.0",
      "licenseDeclared": "Apache-2.0",
      "copyrightText": "NOASSERTION",
```

```
      "checksums": [
        { "algorithm": "SHA256",
          "checksumValue": "8b2fed4b8f42c6d1e2c1e2c1e2c1e2c1e2c1e2c1e2c1e2c
1e2c1e2c1e2c1e2c1e2c1" }
      ],
      "externalRefs": [
        { "referenceCategory": "PACKAGE-MANAGER",
          "referenceType": "purl",
          "referenceLocator": "pkg:maven/org.apache.logging.log4j/
log4j-core@2.14.1" },
        { "referenceCategory": "SECURITY",
          "referenceType": "cpe23Type",
          "referenceLocator": "cpe:2.3:a:apache:log4j:2.14.1:*:*:*:*:*:
*:*" }
      ]
    }
  ],
  "relationships": [
    { "spdxElementId": "SPDXRef-DOCUMENT",
      "relationshipType": "DESCRIBES",
      "relatedSpdxElement": "SPDXRef-Package-app" },
    { "spdxElementId": "SPDXRef-Package-app",
      "relationshipType": "DEPENDS_ON",
      "relatedSpdxElement": "SPDXRef-Package-log4j" }
  ]
}
```

A few things to notice, because they are exactly what trips people up when they first read SPDX JSON:

- The **document element** (`SPDXRef-DOCUMENT`) and the **root package** (`SPDXRef-Package-app`) are different things, joined by the `DESCRIBES` relationship. The document is *about* the package.
- `dataLicense` is `CC0-1.0` and has nothing to do with the software's Apache-2.0 license — that lives on the packages.
- `log4j-core` carries `NOASSERTION` for `copyrightText` — a faithful "we didn't determine this" — while still carrying a confident purl and CPE. Partial knowledge is expressed field by field.
- The `relationships` array is a flat list of edges; the graph structure is entirely in those edges, not in nesting. To reconstruct the DAG you index packages by `SPDXID` and walk the relationships.

SPDX license expressions

The **SPDX License List** and the **SPDX License Expression** grammar are arguably SPDX's most widely reused contribution — you will find SPDX license identifiers in `package.json`, Cargo manifests, Linux kernel `SPDX-License-Identifier:` header comments, and CycloneDX documents. They are worth learning as a standard in their own right, independent of the SBOM format.

The **License List** is a curated registry of open-source and other licenses, each assigned a short, stable **identifier**: `Apache-2.0`, `MIT`, `GPL-2.0-only`, `GPL-2.0-or-later`, `BSD-3-Clause`, `MPL-2.0`, `LGPL-3.0-or-later`, and hundreds more, plus a companion list of **license exceptions** (`Classpath-exception-2.0`, `GCC-exception-3.1`, `LLVM-exception`). The list is versioned — hence the `licenseListVersion` field — and the identifiers are chosen to be unambiguous, which is the whole point: `GPL-2.0-only` and `GPL-2.0-or-later` are distinct identifiers precisely because "GPLv2" alone is ambiguous about the "or later" clause, and that ambiguity has real legal consequences.

A **license expression** composes identifiers with a small grammar:

- A bare **identifier** is the simplest expression: `MIT`.
- The `+` **operator** means "or any later version" for a license family that supports it: `Apache-2.0+`. (For GPL-family licenses the modern practice is the explicit `-or-later` identifier rather than `+`.)
- `WITH` attaches a license exception: `GPL-2.0-or-later WITH Classpath-exception-2.0`. The left side must be a license identifier, the right an exception identifier.
- `AND` means all listed licenses apply simultaneously (you must comply with every one): `Apache-2.0 AND MIT`.
- `OR` means a choice is offered (you may pick one): `MIT OR GPL-3.0-only` — a dual-licensed package.
- **Parentheses** group: `(MIT OR Apache-2.0) AND BSD-3-Clause`.

Operator precedence is fixed and matters: `WITH` **binds tightest, then** `AND`**, then** `OR`. So `Apache-2.0 AND MIT OR GPL-3.0-only` parses as `(Apache-2.0 AND MIT) OR GPL-3.0-only`, which is a materially different obligation than `Apache-2.0 AND (MIT OR GPL-3.0-only)` — when in doubt, parenthesize. A compliance engine that gets precedence wrong will compute the wrong obligation set.

For licenses **not on the list** — a proprietary EULA, a bespoke or unrecognized open-source license — SPDX provides `LicenseRef-` identifiers. You mint a document-local id such as `LicenseRef-Acme-Proprietary`, reference it in an expression exactly like a list identifier (`LicenseRef-Acme-Proprietary AND MIT`), and **define** it in the document's *Other Licensing Information* section with its full license text (`extractedText`). This keeps expressions uniform whether or not a license is standardized.

Two sentinel values complete the vocabulary and are not the same thing:

- `NONE` — an affirmative assertion that there is *no* license (public domain, or the field genuinely does not apply).
- `NOASSERTION` — the author is *not making a claim* (did not determine it, or could not). This is the "known unknown" from Chapter 1, and conflating it with `NONE` is a real bug: "no license" and "we don't know the license" are opposite operational situations.

```
{
  "licenseConcluded": "(MIT OR Apache-2.0) AND BSD-3-Clause",
  "licenseDeclared": "GPL-2.0-or-later WITH Classpath-exception-2.0"
}
```

SPDX 3.0: the redesign

SPDX 3.0, released in **2024**, is not a bigger 2.3 — it is a **new data model** built to carry things the 2.x model was never designed for. Understanding *why* it exists makes the changes legible: by the early 2020s, the software supply chain had grown demands SPDX 2.x could only awkwardly serve. Security teams wanted a native place for vulnerability and VEX data. Build teams wanted first-class provenance. The AI/ML community wanted to describe models and training datasets as supply-chain artifacts (Book 7, Chapter 7). Stuffing all of that into a license-compliance-shaped 2.x model — which had no vulnerability class at all — meant abuse of `ExternalRef` and annotations. 3.0 answers this with **extensibility as the organizing principle**.

The core-plus-profiles model

The central idea is a **class model** split into a shared **Core** and a set of domain **Profiles**. Everything is an **Element** — a base class with identity (`spdxId`), a name, creation info, and the ability to participate in

relationships. **Relationship** is itself an Element, with `from`, `to`, and a `relationshipType`, so the whole document is a graph of typed elements and typed relationships rather than the 2.x fixed sections. This is a genuine graph model, closer to RDF (which is no accident — 3.0 serializes as JSON-LD).

On top of Core sit **profiles**, each adding classes and properties for a domain:

- **Software** — the SBOM classes proper: `Package`, `File`, `Snippet`, `Sbom`. This is the 2.x functionality, re-homed.
- **Security** — vulnerability and assessment classes: a `Vulnerability` element, and relationship types expressing **VEX-style** statements (affected / not affected / fixed / under investigation) and CVSS/EPSS assessments. This is the native security model 2.x lacked.
- **Licensing** — the license-expression and `LicenseRef` machinery, now formalized as classes.
- **Build** — build **provenance**: a `Build` element capturing how an artifact was produced, aligning conceptually with SLSA and in-toto provenance (Book 4, Book 5).
- **AI** — classes describing AI/ML **models** (intended use, energy, safety-relevant metadata).
- **Dataset** — classes describing **training and evaluation datasets** (collection process, size, sensitivity) — together with the AI profile, this is what makes SPDX 3.0 a candidate format for the ML supply chain (Book 7, Chapter 7).
- **Lite** — a deliberately minimal profile for constrained producers (a heritage of the Japanese "SPDX Lite" work) who need a small mandatory field set.

```
flowchart TD
    CORE["Core model
Element - Relationship
CreationInfo - Agent
Bundle - SpdxDocument"]
    CORE --> SW["Software profile
Package - File - Snippet - Sbom"]
    CORE --> SEC["Security profile
Vulnerability - VexAffected
CvssAssessment"]
    CORE --> LIC["Licensing profile
License expressions
CustomLicense"]
    CORE --> BLD["Build profile
Build provenance"]
    CORE --> AI["AI profile
AIPackage / model metadata"]
    CORE --> DS["Dataset profile
training / eval datasets"]
    CORE --> LITE["Lite profile
minimal field set"]
```

A document declares which profiles it conforms to, and a consumer can process just the profiles it understands — a security tool reads Core plus Security and ignores the Dataset classes it does not care about. This is the extensibility payoff: new domains become new profiles without re-versioning the whole format.

Serialization and the migration reality

SPDX 3.0's primary serialization is **JSON-LD** — JSON with an `@context` that maps the plain keys to the formal SPDX ontology IRIs. This makes a 3.0 document simultaneously ordinary JSON (any JSON tool reads it) and linked data (RDF tools can consume it and merge graphs across documents). Structurally a 3.0 document is a `@graph` array of elements, each with a `type`, an `spdxId` (now a full IRI/URN, not a document-local `SPDXRef-`), and profile-prefixed properties (e.g. `software_packageVersion`, `software_downloadLocation`).

```
{
  "@context": "https://spdx.org/rdf/3.0.1/spdx-context.jsonld",
  "@graph": [
    {
      "type": "CreationInfo",
      "@id": "_:creationinfo",
      "specVersion": "3.0.1",
      "created": "2026-07-31T14:12:03Z",
      "createdBy": ["https://acme.example/agents/syft"]
    },
    {
      "type": "software_Package",
      "spdxId": "urn:acme:spdx:pkg:log4j-core-2.14.1",
      "name": "log4j-core",
      "software_packageVersion": "2.14.1",
      "creationInfo": "_:creationinfo"
    },
    {
      "type": "Relationship",
      "spdxId": "urn:acme:spdx:rel:app-depends-log4j",
      "from": "urn:acme:spdx:pkg:checkout-service-3.2.0",
      "relationshipType": "dependsOn",
      "to": ["urn:acme:spdx:pkg:log4j-core-2.14.1"],
      "creationInfo": "_:creationinfo"
    }
  ]
}
```

The blunt fact for a practitioner: **3.0 is not backward-compatible with 2.x.** The element graph, the IRI identifiers, the JSON-LD envelope, and the profile-prefixed property names all differ. A parser written for 2.3 will not read a 3.0 document and vice versa. Migration is a real conversion, not a version bump, and the ecosystem is doing it gradually — the SPDX project maintains tooling to convert between the versions, but every producer and consumer in a chain has to move. As of this writing (mid-2026), **adoption is early**: most SBOM tooling — Syft, the common CI plugins, most enterprise consumers — still emits and expects **2.3**. Treat 3.0 as strategically important and technically ready, but do not assume your customer's ingestion pipeline can parse it yet.

Producing, consuming, and validating SPDX

Producing. In practice almost nobody hand-writes SPDX; it is generated. The dominant open-source generator is **Syft** (Anchore), which inspects a directory, container image, or archive and emits SPDX JSON (`syft`

`<image> -o spdx-json > sbom.spdx.json`). The reference **SPDX tools** exist for several languages, and many build systems have SPDX-emitting plugins. The generation mechanics — and their accuracy limits, which are the whole ballgame — are Chapter 4; here it is enough to know that the JSON above is the *shape* those tools produce.

```
# Generate an SPDX 2.3 JSON SBOM from a container image
syft registry.acme.example/checkout-service:3.2.0 -o spdx-json > checko
ut.spdx.json
```

Consuming and validating. SPDX ships an **online validator** and libraries in the major languages: **`tools-golang`** (Go) and **`spdx-tools`** (Python) are the reference implementations, with equivalents elsewhere. Validation checks structural conformance — mandatory fields present, `SPDXID`s unique and well-formed, relationships referencing existing elements, `dataLicense` equal to `CC0-1.0`, license expressions parseable. Validating on ingest is not optional at fleet scale: a malformed SBOM that a producer's pipeline emitted for months is exactly the kind of silent corruption that turns the Log4Shell query into a false negative.

Beyond structural validity there is **NTIA-minimum-elements conformance** (Chapter 1): does the document actually carry Supplier, Component Name, Version, a unique identifier, dependency relationships, author, and timestamp for each component? A structurally valid SPDX document can still fail this — e.g. every package saying `supplier: NOASSERTION`. Conformance checkers (the NTIA "sbom conformance" style tooling) test the minimum-elements bar specifically, which is a different and stricter question than "does it parse."

Reading a real-ish SPDX document

Put the pieces together on the example above as a consumer would. Given `checkout.spdx.json`, here is what you extract and how:

- **What is this SBOM about?** Follow the `DESCRIBES` relationship (or `documentDescribes`): `SPDXRef-Package-app` — `checkout-service 3.2.0`. That is the root; everything else is a component of it.
- **What is the dependency graph?** Index every package by `SPDXID`, then walk the `relationships` array collecting `DEPENDS_ON` edges. From this document, `checkout-service → log4j-core`. In the full

graph, `log4j-core` is reached through both `web-framework` and `json-lib` — two `DEPENDS_ON` edges into one package element.

- **What are the licenses?** Read `licenseConcluded` per package, falling back to `licenseDeclared` and noting any `NOASSERTION`. Here everything concludes to `Apache-2.0`; a copyleft or dual-license would surface in the expression and drive the compliance workflow.
- **What are the machine identifiers?** Collect `externalRefs`. `log4j-core` carries `pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1` (purl, for OSV/registry matching) and `cpe:2.3:a:apache:log4j:2.14.1:*:*:*:*:*:*:*` (CPE, for NVD matching). Feed both to your SCA matcher — this is the seam where the SBOM meets Book 2's vulnerability data.

That four-step read — *root, graph, licenses, identifiers* — is the whole job of consuming an SPDX document, and it is identical whether the file has two packages or two thousand.

SPDX against your needs: strengths and weaknesses

SPDX is not the only format, and Chapter 3 gives CycloneDX a full and fair treatment. To set that comparison up honestly, here is where SPDX is strong and where it is weak.

Strengths.

- **ISO standard, mature, broadly accepted.** ISO/IEC 5962:2021 status carries real weight in government and enterprise procurement; "provide an ISO-standard SBOM" points at SPDX. It has been in production for over a decade.
- **License rigor.** The License List and expression grammar are the industry reference for license identity — reused far outside SPDX. If license compliance is a first-order concern, SPDX's concluded-versus-declared model and expression precision are unmatched.
- **Relationship expressiveness.** The typed-relationship model represents true DAGs, linking semantics, build/dev/runtime scoping, patch and generation provenance — richer than a plain dependency tree.
- **Broad tooling and government mandate fit.** Named in the NTIA automation-support pillar; first-class support across the major SBOM tools.

Weaknesses and criticisms.

- **Verbosity.** SPDX documents, especially file-level ones, are large. The tag-value and RDF/XML forms in particular are heavy; even JSON SPDX tends to be bulkier than the equivalent CycloneDX. At fleet scale this is a real storage and parsing cost.
- **Historically weaker on security.** This is the sharpest, and it is fair: **SPDX 2.x has no native vulnerability model.** You cannot express "this component is affected by CVE-X, and here is the VEX status" in core 2.x — you bolt it on via `ExternalRef` advisories or annotations. Native vulnerability and VEX support is exactly the ground **CycloneDX led on**, and it is a major reason security teams often prefer CycloneDX. SPDX 3.0's Security profile is the answer, but it is not yet the deployed reality.
- **Complexity.** The generality that makes SPDX expressive also makes it harder to produce minimally and correctly. The number of fields, the concluded/declared subtlety, the `filesAnalyzed`/verification-code coupling, and the 2.x-versus-3.0 fork are all real cognitive load compared with a leaner format.

The honest summary: **SPDX is the license-and-compliance-first, ISO-blessed, relationship-rich choice; CycloneDX (Chapter 3) is the security-and-VEX-first, compact choice.** Neither is strictly better; they were optimized for different original problems, and Chapter 3 makes the trade-off concrete before Chapter 5 shows how to normalize *both* into one internal model.

Distributed-systems lens

Everything in this chapter changes character once you multiply it by a fleet. A single SPDX document is a file you read; **thousands of SPDX documents flowing from CI on every build** are a data-engineering problem, and several properties of the format determine how well it survives that scale.

JSON is the ingestion format, and stable identifiers are the join keys. The reason `.spdx.json` won over tag-value and RDF/XML is operational: a central inventory (Chapter 5) ingests JSON trivially, and the `documentNamespace` plus `SPDXID` give every element a globally unique address to store and reference. Even more important are the **purl and CPE in `externalRefs`** — those, not the human `PackageName`, are the keys

you index and join on. A fleet inventory that indexes packages by purl can answer "where is `pkg:maven/.../log4j-core@2.14.1` across every service" as a lookup; one that tries to match on name strings drowns in `log4j` versus `log4j-core` versus `Log4j` disambiguation. **The value of an SBOM at scale is proportional to the quality of the machine identifiers it carries** — which is why an SPDX package that omits its purl is nearly worthless to the inventory even if it is otherwise complete.

The relationship graph is what makes "what depends on X" answerable. Load the `DEPENDS_ON` edges from every document into a graph store and the transitive-dependents query — "everything, across the fleet, that transitively pulls in this component" — is a graph traversal, not a scan. This is the direct fleet-scale payoff of SPDX's relationship model over a flat list, and it is the mechanism behind Chapter 1's Log4Shell capability. The `DESCRIBES` edge matters here too: it is how the inventory attributes each document to its root artifact, so a hit deep in the graph can be traced back to *which service's build* introduced it.

Choosing 2.3 versus 3.0 is an org-level decision with a clear near-term answer. Emit **2.3** today, because that is what your producers generate and your consumers — internal and customer-facing — can parse. Track **3.0** deliberately: pilot it where its profiles buy you something concrete (the Security profile for native VEX, the AI/Dataset profiles if you ship ML), and design your ingestion so that adding a 3.0 parser later does not require re-architecting the store. Because 3.0 is not wire-compatible with 2.x, the migration is a project, not a flag; plan for a period where your pipeline reads *both*.

Normalize into one internal model. The strategic conclusion, developed fully in Chapter 5: do not let SPDX-versus-CycloneDX or 2.3-versus-3.0 leak into every downstream consumer. Ingest every format at the edge, extract the invariant substrate — components with purls and CPEs, typed dependency edges, license expressions, checksums, document provenance — and store *that*. Your policy engines, SCA matchers, and impact-assessment queries then run against one clean internal graph, and the SPDX-specific concerns of this chapter (the `filesAnalyzed` flag, the concluded/declared split, the tag-value versus JSON encoding) become an *ingestion-adapter* detail rather than a fleet-wide concern. SPDX's job, at scale, is to be a faithful and identifier-rich *source* for that internal model.

Key takeaways

- **SPDX began in 2010 as a Linux Foundation license-compliance format** and grew into a full SBOM format; **SPDX 2.2.1 is ISO/IEC 5962:2021**, the production line is **2.3**, and **SPDX 3.0 (2024)** is a ground-up redesign. Emit 2.3 today; track 3.0.
- A **2.x document has seven parts**: Document Creation Information, Packages, Files, Snippets, Relationships, Other Licensing Info, Annotations. The `documentNamespace` plus document-local `SPDXID`s give every element a globally unique identity; `dataLicense` is always `CC0-1.0` (the license of the *data*, not the software).
- **`PackageLicenseConcluded` versus `PackageLicenseDeclared`** is SPDX's signature distinction: declared is what the package claims about itself, concluded is what the SBOM author determined — and a tool that does not analyze licenses correctly sets concluded to `NOASSERTION`.
- **`ExternalRef` carries the machine identifiers** — purl under `PACKAGE-MANAGER`, CPE under `SECURITY` — and carrying **both** is what makes a package robust for downstream security matching, because purl and CPE key different databases.
- **Relationships are first-class typed edges** (`DESCRIBES`, `CONTAINS`, `DEPENDS_ON`, `GENERATED_FROM`, and dozens more), which lets SPDX represent a true dependency DAG — the graph, not the list, is the deliverable.
- SPDX serializes as **tag-value, JSON, YAML, and RDF/XML**; **`.spdx.json` is the production format**. The graph lives in a flat `relationships` array of edges, reconstructed by indexing packages on `SPDXID`.
- **SPDX license expressions** — `AND` / `OR` / `WITH`, the `+` operator, `LicenseRef-`, and the `NONE` / `NOASSERTION` sentinels, with precedence `WITH > AND > OR` — are a widely reused mini-standard; `NOASSERTION` ("unknown") and `NONE` ("no license") are opposites, not synonyms.
- **SPDX 3.0** replaces the fixed sections with a **Core-plus-Profiles class model** (Software, Security, Licensing, Build, AI, Dataset, Lite), an element/relationship graph, and **JSON-LD** serialization. It adds the native **Security (VEX-ish)** and **Build (provenance)** models 2.x lacked and the **AI/Dataset** profiles for the ML supply chain — but it is **not backward-compatible with 2.x**, and adoption is early.

- **Strengths:** ISO standing, license rigor, relationship expressiveness, broad tooling. **Weaknesses:** verbosity, historically **no native vulnerability/VEX model in 2.x** (the ground CycloneDX led on), and complexity. This sets up Chapter 3's comparison.
- **At fleet scale**, SPDX's value is its JSON serialization (easy ingest), its stable purl/CPE/`SPDXID` identifiers (the join keys for a central store), and its relationship graph (the "what depends on X" traversal). The right architecture normalizes SPDX *and* CycloneDX into one internal model (Chapter 5), leaving format specifics at the ingestion edge.

Further reading

- **SPDX Specification 2.3**, Linux Foundation / SPDX project — the authoritative reference for the production format (`spdx.github.io/spdx-spec/v2.3/`).
- **ISO/IEC 5962:2021**, "Information technology — SPDX Specification V2.2.1" — the ISO standardization of SPDX.
- **SPDX 3.0 Specification**, Linux Foundation / SPDX project — the core model and profile documents (`spdx.github.io/spdx-spec/v3.0.1/`).
- **SPDX License List** and the **SPDX License Expressions** appendix — the license identifier registry and the expression grammar (`spdx.org/licenses/`).
- `tools-golang` and `spdx-tools` (Python) — the reference parser/validator libraries; and the SPDX **online validator**.
- **Syft** (Anchore) documentation — the dominant open-source SPDX generator; deeper treatment in Book 3, Chapter 4 — Generating SBOMs.
- **NTIA**, "The Minimum Elements for a Software Bill of Materials (SBOM)," 2021 — the conformance bar an SPDX document must actually meet (Book 3, Chapter 1).

- **Package URL (purl)** specification and **NIST CPE 2.3** — the identity schemes SPDX carries in `ExternalRef` (Book 2, Chapter 5).

Chapter 3 — CycloneDX in Depth

What this chapter covers. Chapter 2 dissected SPDX — a format whose bones were set by the license-compliance world and which grew, through ISO standardization and the 3.0 rewrite, into a general-purpose graph model. This chapter is the counterpart: **CycloneDX**, the other format you will actually encounter in production, and the one most likely to be sitting in your artifact registry if a modern scanner produced it. CycloneDX comes from a different lineage — application security and vulnerability management, not legal compliance — and that heritage is visible in every design decision. It ships a native vulnerability model, a native VEX, and an entire *family* of bill-of-materials types (the "xBOM" vision) that reach past software into services, hardware, cryptography, and machine-learning models. We cover the object model precisely, the differentiators that matter, and then do the thing every engineer actually wants: an honest, non-partisan SPDX-versus-CycloneDX comparison, because in a real fleet you do not pick one — you *consume both* and normalize.

Learning goals — after this chapter you should be able to:

- Trace CycloneDX's **governance and heritage** (OWASP, Dependency-Track lineage, ECMA-424 in 2024, spec 1.6) and explain how a security-first origin shaped the format differently from SPDX's compliance-first origin.
- Read and write a **CycloneDX 1.6 document**: `bomFormat`, `specVersion`, `serialNumber`, `version`, `metadata`, `components` (with `bom-ref`, `purl`, `hashes`, `licenses`, `scope`), and the `dependencies` graph (`dependsOn` / `provides`).
- Use CycloneDX's **differentiators**: the native `vulnerabilities` array (ratings, CWEs, advisories, and the `analysis` /VEX block), `services` (SaaSBOM), `compositions` (completeness assertions), `formulation` (build provenance), and `evidence` (identification provenance and reachability support).
- Explain the **xBOM family** — SBOM, VEX, VDR, SaaSBOM, HBOM, ML-BOM, CBOM, OBOM, MBOM — and why CycloneDX treats them as one schema with different populated sections.

- Make a **defensible format choice** (or, more often, a defensible normalization strategy) using a fair trade-off table, and wire up a produce/consume pipeline with `syft`, `cdxgen`, the `cyclonedx-cli`, and OWASP Dependency-Track.

A word on boundaries. This chapter assumes Chapter 2's SPDX model, Chapter 1's motivation and the identity schemes (purl, CPE) from Book 2, Chapter 6 — Software Composition Analysis in Depth. It goes deep on *what CycloneDX can express*; the mechanics of getting an accurate BOM out of a build live in Chapter 4 (SBOM Generation), the VEX correlation pipeline in Chapter 6, and fleet-scale ingestion/normalization in Chapter 5. Signing is sketched here and treated fully in Book 5 (Signing and Attestation).

Origin, heritage, and governance

CycloneDX began around 2017 inside the **OWASP** ecosystem, closely tied to **OWASP Dependency-Track** — a continuous component-analysis platform that needed a compact, machine-friendly BOM to ingest and monitor. That origin is the single most important fact about the format. SPDX was born to answer "*what are the licenses of everything in this release, and can we ship it?*" CycloneDX was born to answer "*what components are in this running application, and which of them have known vulnerabilities right now?*" The formats have since grown toward each other, but the center of gravity never moved: SPDX is a license/compliance format that learned security; CycloneDX is a security format that learned compliance.

Concretely, the security heritage shows up as first-class schema real estate that SPDX did not have natively for most of its life: an embedded `vulnerabilities` array, an embedded VEX `analysis` block, a `services` model for describing the runtime call graph, and `evidence` structures that record *how confidently* a component was identified (the input to reachability analysis). None of that is bolted on — it is in the core schema.

Governance and cadence

CycloneDX is governed as an OWASP flagship project with a working group and a public GitHub-based process. Two properties matter to an engineer choosing a format:

- **Standardization.** In June 2024 CycloneDX was ratified as **ECMA-424** ("CycloneDX Bill of Materials Specification"), its first-edition text corresponding to the 1.6 specification. This is the counterpart to SPDX's ISO/IEC 5962 — both formats now carry a formal standards imprimatur, though through different bodies (ECMA International for CycloneDX, ISO/IEC for SPDX). ECMA's process is generally faster and lighter-weight than ISO's, which is consistent with the rest of the CycloneDX story.
- **Cadence.** CycloneDX moves quickly and is unashamedly developer-centric. The version history is a good map of its ambitions: **1.4** (January 2022) introduced the native `vulnerabilities` array and VEX; **1.5** (June 2023) added `formulation` (build provenance), `annotations`, `lifecycles`, richer `compositions` and `evidence`, and the `machine-learning-model` and `data` component types (ML-BOM); **1.6** (April 2024) added the `cryptographic-asset` component type with `cryptoProperties` (CBOM), a `declarations` section for attestations (CycloneDX Attestations / CDXA), the `provides` dependency edge, `standards` definitions, and a rename of `manufacture` to `manufacturer`. That is a lot of surface area added in roughly two years — the trade-off for velocity is that tooling and downstream consumers lag the newest sections, so a 1.6 CBOM or attestation block may be produced before much of the ecosystem can *do* anything with it.

Throughout this chapter, "CycloneDX 1.6" means the ECMA-424 first edition unless noted.

The CycloneDX object model

A CycloneDX BOM is a single document — JSON, XML, or Protocol Buffers — with a small set of top-level arrays hanging off a thin header. The mental model is: a **header** that identifies the document, a **metadata** block describing *what this BOM is about*, and then parallel arrays (`components`, `services`, `dependencies`, `vulnerabilities`, `compositions`, `formulation`, …) that are cross-linked by an internal identifier called

`bom-ref`. Unlike SPDX 2.x, where nearly every relationship is a generic `(elementA, relationshipType, elementB)` triple, CycloneDX uses *purpose-built* structures for each kind of relationship. That is the format's defining ergonomic choice, and we will return to it in the comparison.

```
flowchart TD
  BOM["BOM root
bomFormat, specVersion,
serialNumber, version"]
  META["metadata
timestamp, tools, authors,
component (subject),
lifecycles, manufacturer, supplier"]
  COMP["components[]
type, name, version,
bom-ref, purl, cpe,
hashes, licenses, scope,
evidence, nested components"]
  SVC["services[]
bom-ref, provider,
endpoints, data flows,
trust boundary"]
  DEP["dependencies[]
ref, dependsOn[], provides[]"]
  VULN["vulnerabilities[]
id, ratings, cwes,
advisories, analysis (VEX),
affects[]"]
  COMPOS["compositions[]
aggregate: complete /
incomplete / unknown"]
  FORM["formulation[]
build workflows,
tasks, provenance"]

  BOM --> META
  BOM --> COMP
  BOM --> SVC
  BOM --> DEP
  BOM --> VULN
  BOM --> COMPOS
  BOM --> FORM
  DEP -. "ref / dependsOn
point at bom-ref" .-> COMP
  DEP -. "and at services" .-> SVC
  VULN -. "affects[].ref
points at bom-ref" .-> COMP
```

The header and `metadata`

Four fields identify the document itself:

- **`bomFormat`** — always the literal string `"CycloneDX"`. This is how a consumer tells a CycloneDX file from an SPDX one without sniffing structure.
- **`specVersion`** — the schema version, e.g. `"1.6"`. It governs which fields are legal.
- **`serialNumber`** — a URN of the form `urn:uuid:<uuid>`. It is the *stable identity of this BOM document* across revisions.
- **`version`** — an integer, the **revision of the BOM**, starting at 1. If you regenerate a BOM for the same subject and correct an error, you keep the `serialNumber` and bump `version`. This lets consumers reason about "the latest BOM for artifact X."

Then **`metadata`** describes what the BOM is *about* and how it was made:

- **`timestamp`** — when the BOM was created (RFC 3339).
- **`lifecycles`** — new-ish; the lifecycle phase(s) this BOM reflects (`design`, `pre-build`, `build`, `post-build`, `operations`, `discovery`, `decommission`). This is CycloneDX's answer to CISA's SBOM-types taxonomy from Chapter 1: a *build*-phase BOM and an *operations*-phase BOM describe the same software but see different things.
- **`tools`** — what generated the BOM. In 1.5+ this is an object with `components[]` and `services[]` (the tools themselves modeled as components), superseding the flat 1.4-era `tool` array.
- **`authors`** — the people/contacts responsible.
- **`component`** — the *subject* of the BOM: the single component this BOM describes (your application, your container image). Everything in the top-level `components` array is, conceptually, *inside* this subject.
- **`manufacturer`** and **`supplier`** — the organization that made, and the organization that supplied, the subject. (`manufacturer` was `manufacture` before 1.6.)

`components`

The `components` array is the inventory. Each component carries an identity and metadata; the fields you will use constantly:

- **`type`** — and this is where CycloneDX's breadth first shows. The enumeration in 1.6 is `application`, `framework`, `library`, `container`, `platform`, `operating-system`, `device`, `device-driver`, `firmware`, `file`, `machine-learning-model`, `data`, and `cryptographic-asset`. A format that can type a component as a machine-learning model or a cryptographic asset is telling you it intends to describe more than a Maven jar.
- **`name`**, **`version`**, **`group`** — the human identity (`group` is the Maven groupId / npm scope).
- **`bom-ref`** — the *internal* identifier used by every other section to point at this component. It must be unique within the document. Conventionally people use the purl as the `bom-ref` (`pkg:npm/lodash@4.17.21`) because it is both unique and meaningful, but it is an opaque string as far as the schema cares.
- **`purl`** — the Package URL: the portable, ecosystem-aware identity (`pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1`). This is the field that makes the component *matchable* against vulnerability databases (OSV, GHSA). See Book 2, Chapter 6.
- **`cpe`** — the NVD Common Platform Enumeration string, for matching against NVD/CVE data that is keyed on CPE rather than purl.
- **`hashes`** — an array of `{alg, content}` (SHA-256, SHA-512, SHA3, BLAKE2b/3, …). The cryptographic pin from an artifact to its bytes.
- **`licenses`** — an array of either `{license: {id | name, ...}}` (an SPDX license ID or a free name) or `{expression: "..."}` (an SPDX license expression). In 1.6 a license entry can carry an `acknowledgement` of `declared` or `concluded` — the same declared-vs-concluded distinction SPDX makes.
- **`supplier`**, **`publisher`** — provenance of the component.
- **`scope`** — `required`, `optional`, or `excluded`. A build tool can mark a dependency that is present but not actually used (`excluded`) or only needed in some configuration (`optional`) — directly useful when triaging whether a vulnerable component is even in the shipped path.
- **`components`** — **nested components**. A component can contain sub-components. This is how CycloneDX models "this container image

contains these OS packages contains these files," and it is an alternative to expressing everything through the dependency graph.

- `evidence` — provenance of the *identification itself* (covered under differentiators).

Here is a minimal but real and correct CycloneDX 1.6 document: an application with two library dependencies and a dependency graph.

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.6",
  "serialNumber": "urn:uuid:3e671687-395b-41f5-a30f-a58921a69b79",
  "version": 1,
  "metadata": {
    "timestamp": "2026-07-31T09:15:00Z",
    "lifecycles": [
      { "phase": "build" }
    ],
    "tools": {
      "components": [
        {
          "type": "application",
          "group": "@cyclonedx",
          "name": "cdxgen",
          "version": "10.9.5"
        }
      ]
    },
    "authors": [
      { "name": "Platform Security", "email": "seceng@example.com" }
    ],
    "component": {
      "type": "application",
      "bom-ref": "pkg:generic/checkout-service@2.4.0",
      "name": "checkout-service",
      "version": "2.4.0",
      "purl": "pkg:generic/checkout-service@2.4.0",
      "supplier": { "name": "Example, Inc." }
    },
    "manufacturer": { "name": "Example, Inc." }
  },
  "components": [
    {
      "type": "library",
      "bom-ref": "pkg:maven/com.squareup.okhttp3/okhttp@4.12.0",
      "group": "com.squareup.okhttp3",
      "name": "okhttp",
      "version": "4.12.0",
      "purl": "pkg:maven/com.squareup.okhttp3/okhttp@4.12.0",
      "scope": "required",
      "hashes": [
        { "alg": "SHA-256",
          "content": "b2a5c1d0f0a3e4b5c6d7e8f901122334455667788 99aabbcc
ddeeff001122334" }
      ],
      "licenses": [
        { "license": { "id": "Apache-2.0", "acknowledgement": "declared
" } }
```

```
      ]
    },
    {
      "type": "library",
      "bom-ref": "pkg:maven/com.squareup.okio/okio@3.9.0",
      "group": "com.squareup.okio",
      "name": "okio",
      "version": "3.9.0",
      "purl": "pkg:maven/com.squareup.okio/okio@3.9.0",
      "scope": "required",
      "licenses": [
        { "license": { "id": "Apache-2.0" } }
      ]
    }
  ],
  "dependencies": [
    {
      "ref": "pkg:generic/checkout-service@2.4.0",
      "dependsOn": [ "pkg:maven/com.squareup.okhttp3/okhttp@4.12.0" ]
    },
    {
      "ref": "pkg:maven/com.squareup.okhttp3/okhttp@4.12.0",
      "dependsOn": [ "pkg:maven/com.squareup.okio/okio@3.9.0" ]
    },
    {
      "ref": "pkg:maven/com.squareup.okio/okio@3.9.0",
      "dependsOn": []
    }
  ]
}
```

Note what is *not* here: no per-relationship triples, no separate "describes" element. The subject is `metadata.component`, and the graph is expressed once in `dependencies`.

`dependencies` — the graph

CycloneDX's dependency graph is deliberately flat and purpose-built. Each entry has:

- **`ref`** — the `bom-ref` of a component (or service).
- **`dependsOn`** — an array of `bom-ref`s that `ref` directly depends on.
- **`provides`** — added in 1.6: an array of `bom-ref`s that `ref` *provides* an implementation of. This models the interface/implementation relationship — e.g. a concrete SLF4J binding *provides* the SLF4J API — which the pure `dependsOn` edge could not express cleanly.

The graph is a set of adjacency lists. To reconstruct the full transitive tree you walk `dependsOn` from the subject. Leaf components appear with an empty `dependsOn` (or are simply referenced without their own entry). Compared with SPDX, where you would encode the same tree as a pile of `DEPENDS_ON` relationship objects each naming two SPDXIDs, the CycloneDX form is terser and reads like the graph it represents — but it is also *less expressive*: SPDX 2.x defines dozens of relationship types (`DYNAMIC_LINK`, `BUILD_DEPENDENCY_OF`, `GENERATED_FROM`, `PATCH_APPLIED`, …), while CycloneDX gives you `dependsOn` and `provides` and expects you to encode nuance elsewhere (in `scope`, in `pedigree`, in `properties`). That is a real trade: CycloneDX optimizes for the graph you query most (runtime dependency) at the cost of the long tail of relationship semantics.

```
flowchart LR
  APP["checkout-service@2.4.0
(metadata.component)"]
  OKHTTP["okhttp@4.12.0
bom-ref"]
  OKIO["okio@3.9.0
bom-ref"]
  APP -->|dependsOn| OKHTTP
  OKHTTP -->|dependsOn| OKIO
```

Serializations

CycloneDX defines three wire formats from a single logical model:

- **JSON** — the dominant format in practice, with a published JSON Schema (`bom-1.6.schema.json`). Almost every modern tool emits and ingests JSON first.
- **XML** — the original serialization (CycloneDX predates its own JSON support), still widely used and the only one for which the classic **XML Signature** enveloped-signing path applies.
- **Protocol Buffers** — a `.proto` schema for high-throughput, size-sensitive pipelines. Useful when you are shipping millions of BOMs through a message bus, but the protobuf representation historically trails the newest schema features, so treat it as a transport optimization rather than the canonical form.

The three are meant to be losslessly convertible for the features each supports; `cyclonedx-cli convert` moves between them.

CycloneDX's differentiators

Everything above has a rough SPDX analogue. What follows mostly does not — this is where choosing CycloneDX buys you something concrete.

The native `vulnerabilities` model

Since 1.4, a CycloneDX BOM can carry a top-level `vulnerabilities` array. This is the feature that most cleanly separates it from SPDX, whose vulnerability story historically lived in a separate profile or an external document. Each vulnerability entry can hold:

- **`id`** and **`source`** — the identifier (`CVE-2021-44228`, a GHSA ID, an internal ID) and where it came from (`{name: "NVD", url: "..."}`).
- **`ratings`** — an array of severity ratings, each with a `source`, a numeric `score`, a `severity` (`critical` / `high` / `medium` / `low` / `info` / `none` / `unknown`), a `method` (`CVSSv31`, `CVSSv4`, `OWASP`, `SSVC`, …), and a `vector`. Multiple ratings from different sources can coexist — you are not forced to collapse NVD and a vendor score into one number.
- **`cwes`** — an array of CWE integers (e.g. `502` for deserialization).
- **`advisories`**, **`references`** — links to the advisory and related records.
- **`description`**, **`detail`**, **`recommendation`**, **`workaround`**.
- **`affects`** — an array binding the vulnerability to *components in this BOM* by `bom-ref`, optionally with affected `versions`/ranges and a per-range `status`. This is the join back into the inventory.
- **`analysis`** — the VEX block (next).

Because the vulnerability is bound to the component graph by `bom-ref` in the *same* document, a scanner's output and the inventory it scanned travel together. That is the whole appeal of the model for a scan-and-triage pipeline.

```
flowchart TD
  subgraph BOM["One CycloneDX document"]
    C["components[]
okhttp, okio, log4j-core…"]
    V["vulnerabilities[]
CVE-2021-44228"]
    A["analysis (VEX)
state: not_affected
justification: code_not_reachable"]
  end
  V -->|"affects[].ref → bom-ref"| C
  V --> A
```

CycloneDX VEX and the `analysis` block

The **`analysis`** object inside a vulnerability entry is CycloneDX's inline **VEX** (Vulnerability Exploitability eXchange). It answers the question a bare CVE match cannot: *does this vulnerability actually affect this product?* Its fields:

- **`state`** — `resolved`, `resolved_with_pedigree`, `exploitable`, `in_triage`, `false_positive`, or `not_affected`.
- **`justification`** — when the state is `not_affected`, *why*: `code_not_present`, `code_not_reachable`, `requires_configuration`, `requires_dependency`, `requires_environment`, `protected_by_compiler`, `protected_at_runtime`, `protected_at_perimeter`, or `protected_by_mitigating_control`.
- **`response`** — planned responses: `can_not_fix`, `will_not_fix`, `update`, `rollback`, `workaround_available`.
- **`detail`** — free text for the human reasoning.

Crucially, CycloneDX VEX can live **inline** (the `analysis` block inside a full SBOM) *or* as a **standalone BOM** that carries only `vulnerabilities` (with `analysis`) and points at the components of an SBOM published elsewhere. That flexibility — one schema, two deployment modes — is why Chapter 6 treats CycloneDX VEX as a first-class option alongside the OpenVEX and CSAF/VEX profiles. Here is the shape:

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.6",
  "serialNumber": "urn:uuid:c2b8a0e4-6d21-4a7f-9b3e-8f0d6a1c2e34",
  "version": 1,
  "metadata": { "timestamp": "2026-07-31T10:00:00Z" },
  "vulnerabilities": [
    {
      "bom-ref": "vuln-log4shell-checkout",
      "id": "CVE-2021-44228",
      "source": { "name": "NVD", "url": "https://nvd.nist.gov/vuln/
detail/CVE-2021-44228" },
      "ratings": [
        {
          "source": { "name": "NVD" },
          "score": 10.0,
          "severity": "critical",
          "method": "CVSSv31",
          "vector": "CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H"
        }
      ],
      "cwes": [ 502, 917 ],
      "description": "Apache Log4j2 JNDI features do not protect
against attacker-controlled LDAP endpoints.",
      "advisories": [
        { "title": "Apache Log4j Security", "url": "https://
logging.apache.org/log4j/2.x/security.html" }
      ],
      "analysis": {
        "state": "not_affected",
        "justification": "code_not_reachable",
        "response": [ "will_not_fix" ],
        "detail":
"log4j-core is on the classpath transitively but JNDI lookup is never
invoked; message-lookup is disabled via
log4j2.formatMsgNoLookups=true."
      },
      "affects": [
        {
          "ref": "pkg:maven/org.apache.logging.log4j/log4j-
core@2.14.1",
          "versions": [
            { "range": "vers:maven/>=2.0.0|<2.15.0", "status": "affecte
d" }
          ]
        }
      ]
    }
  ]
}
```

Note the `affects[].ref` points at a `bom-ref` that would exist in the *separate* SBOM this VEX annotates — the two documents are joined by shared `bom-ref` values, which is exactly how a fleet correlation engine reconnects them (Chapter 5, Chapter 6).

The xBOM family

CycloneDX's most strategic bet is that a "bill of materials" is not specific to software. The same schema — same header, same `bom-ref` linking, same `dependencies` graph — can enumerate *anything* whose composition and provenance you want to track. This is the **xBOM** vision, and the specification's breadth of component types and top-level sections exists to serve it.

```
flowchart TD
  ROOT["CycloneDX schema
(one model)"]
  ROOT --> SBOM["SBOM
software components"]
  ROOT --> VEX["VEX / VDR
vulnerabilities + analysis"]
  ROOT --> SAAS["SaaSBOM
services, endpoints, data flows"]
  ROOT --> ML["ML-BOM
models, datasets, model cards"]
  ROOT --> CBOM["CBOM
cryptographic assets & algorithms"]
  ROOT --> HBOM["HBOM
hardware / device components"]
  ROOT --> OBOM["OBOM / MBOM
operations / manufacturing"]
```

The members:

- **SBOM** — the software inventory (everything above).
- **VEX** — exploitability statements (the `analysis` block), inline or standalone.
- **VDR (Vulnerability Disclosure Report)** — the inverse framing of VEX: a report of the *known* vulnerabilities affecting a product, disclosed by the supplier. Same `vulnerabilities` array, different intent — VDR discloses what *is* wrong; VEX asserts what is *not* exploitable. A supplier often issues both.
- **SaaSBOM** — the `services` model (below): external services, endpoints, trust boundaries, and data flows.

- **HBOM (Hardware BOM)** — physical/device composition, using `device`, `device-driver`, and `firmware` component types. Relevant to medical devices and IoT where the FDA and CRA regimes from Chapter 1 reach hardware.
- **ML-BOM** — machine-learning models and datasets, via the `machine-learning-model` and `data` component types and the `modelCard` structure (energy, considerations, quantitative analysis, and — critically for supply-chain risk — training-data provenance). As AI systems become software dependencies, "what model and what data went into this" becomes a supply-chain question; see Book 7, Chapter 7 for the AI/ML supply-chain treatment.
- **CBOM (Cryptography BOM)** — new in 1.6: the `cryptographic-asset` component type with `cryptoProperties` describing algorithms, key sizes, protocols, and certificates. The motivating use case is **post-quantum cryptography (PQC) migration**: to know which of your systems use RSA-2048 or ECDSA and must move to PQC algorithms, you first need an inventory of cryptographic assets. A CBOM is that inventory.
- **OBOM (Operations BOM)** and **MBOM (Manufacturing BOM)** — runtime/operational configuration and manufacturing composition, rounding out the lifecycle.

The engineering point is not that you will use all of these — most teams use SBOM + VEX and maybe SaaSBOM. It is that CycloneDX did not fork a new format for each; it extended one schema, so a single parser, a single signing story, and a single `bom-ref` linking model cover the whole family. SPDX 3.0 pursues comparable breadth through its **profiles** mechanism instead; the comparison section returns to this.

`services` — the SaaSBOM

The `services` array describes external services your software *calls* — the runtime supply chain, not the compile-time one. This is uniquely valuable for distributed systems, and it is the CycloneDX feature with no real SPDX analogue. A service entry carries:

- **`bom-ref`**, **`name`**, **`version`**, **`group`**, **`provider`** — identity and who runs it.
- **`endpoints`** — the URLs/URIs the service exposes or that you call.
- **`authenticated`** — whether calls to it are authenticated.

- `x-trust-boundary` — whether calling the service crosses a trust boundary (leaves your security domain).
- `trustZone` — a label for the zone the service sits in.
- `data` — the **data flows**: an array of classified flows, each with a `flow` direction (`inbound`, `outbound`, `bi-directional`, `unknown`), a `classification` (e.g. `PII`, `PCI`, `public`), and optional `source`/`destination` and `governance` (data-ownership) metadata.
- `services` — nested services.

That set of fields is essentially a machine-readable data-flow diagram. For a microservice, a SaaSBOM records "I call the payments API (outbound, PCI data, crosses a trust boundary, authenticated) and the internal user-profile service (bi-directional, PII, same trust zone)." Combined with the software SBOM, you get both halves of the supply chain in one document.

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.6",
  "serialNumber": "urn:uuid:9a1f0c33-77bd-4a2e-8e2b-1d4c9b0a2f55",
  "version": 1,
  "metadata": {
    "timestamp": "2026-07-31T11:20:00Z",
    "component": {
      "type": "application",
      "bom-ref": "checkout-service",
      "name": "checkout-service",
      "version": "2.4.0"
    }
  },
  "services": [
    {
      "bom-ref": "svc-payments",
      "provider": { "name": "Acme Payments" },
      "name": "payments-api",
      "endpoints": [ "https://api.acme-pay.example/v2/charges" ],
      "authenticated": true,
      "x-trust-boundary": true,
      "trustZone": "third-party-pci",
      "data": [
        {
          "flow": "outbound",
          "classification": "PCI",
          "name": "card-charge-request",
          "destination": "svc-payments"
        }
      ]
    },
    {
      "bom-ref": "svc-user-profile",
      "name": "user-profile",
      "authenticated": true,
      "x-trust-boundary": false,
      "trustZone": "internal",
      "data": [
        { "flow": "bi-directional", "classification": "PII", "name": "p
rofile-lookup" }
      ]
    }
  ],
  "dependencies": [
    { "ref": "checkout-service", "dependsOn": [ "svc-payments", "svc-
user-profile" ] }
  ]
}
```

Because services are `bom-ref`-addressable, they participate in the same `dependencies` graph as software components — the subject `dependsOn` both a library and a service. This is the diagram:

```
flowchart LR
  APP["checkout-service"]
  PAY["payments-api
trust: third-party-pci
PCI · outbound"]
  UP["user-profile
trust: internal
PII · bi-directional"]
  APP -->|"outbound, crosses boundary"| PAY
  APP <-->|"internal"| UP
  classDef ext fill:#f8d7da,stroke:#b02a37,color:#000;
  classDef int fill:#d1e7dd,stroke:#146c43,color:#000;
  class PAY ext;
  class UP int;
```

`compositions`, `formulation`, `evidence`, and annotations

Four more sections carry provenance and completeness signal — the things that separate an SBOM you can *trust* from one you merely *have*.

`compositions` encodes **completeness assertions**. Each composition has an `aggregate` value declaring how complete a slice of the BOM is: `complete` (every constituent is accounted for), `incomplete`, one of several qualified-incomplete values (`incomplete_first_party_only`, `incomplete_third_party_only`, …), `unknown`, or `not_specified`, plus the `assemblies` / `dependencies` the assertion applies to. This is how a producer *honestly signals* "I enumerated the third-party dependencies completely but the first-party file list is partial." A consumer that treats an SBOM as ground truth without reading `compositions` is over-trusting it — Chapter 7 (SBOM Quality) leans hard on this field.

`formulation` records **how the software was built** — build provenance inside the BOM. A `formula` can contain `components`, `services`, and `workflows`, where a workflow is a sequence of `tasks` / `steps` with `inputs` and `outputs` and `resourceReferences`. It overlaps conceptually with SLSA provenance and in-toto attestations (Book 4, Chapter 5; Book 5) — the distinction is that `formulation` lives *inside* the CycloneDX document as part of one signed artifact, whereas SLSA provenance is typically a separate attestation. Do not treat them as interchangeable, but do recognize they answer the same "how was this made" question.

`evidence` (on a component) records the **provenance of the identification** — how the tool decided this component is present, and how confident it is. It holds:

- `identity` — which field was inferred (`purl` , `cpe` , `name` , `version` , …), a `confidence` score in `[0,1]` , and the `methods` used, each a `{technique, confidence, value}` where `technique` is one of `source-code-analysis` , `binary-analysis` , `manifest-analysis` , `ast-fingerprint` , `hash-comparison` , `instrumentation` , `dynamic-analysis` , `filename` , `attestation` , or `other` .
- `occurrences` — *where* the component was found (file paths, locations), which supports **reachability**: a component found only in a test directory is a very different risk from one linked into the main binary.
- `callstack` — call-stack frames showing the component being invoked, the strongest possible "this code actually runs" evidence.

`evidence` is CycloneDX admitting that SBOM generation is *inference*, and giving tools a place to show their work. A downstream reachability or triage system can read `confidence` and `occurrences` to prioritize — a low-confidence `filename` -only match is a candidate for suppression; a `callstack` -backed match is not.

`annotations` attach signed, authored commentary to any `bom-ref` — reviewer notes, triage decisions, machine-generated remarks — with a timestamp and annotator, and can themselves be signed.

Signing and integrity

An SBOM you cannot verify is an SBOM you cannot trust in an automated pipeline. CycloneDX supports **enveloped** signatures (the `signature` property carried inside the document) and **detached** signatures. For JSON, signing uses the **JSON Signature Format (JSF)**; for XML, **XML Signature (XML-DSig)**. The `signature` property can appear at the document root and, granularly, on individual `compositions` and `annotations` , so you can sign a completeness assertion or a triage annotation independently.

In practice, teams increasingly wrap the BOM in a **Sigstore/cosign** signature or an **in-toto attestation** rather than using the native JSF envelope, because that plugs into the same keyless-signing and transparency-log infrastructure used for container images (Book 5,

Chapters 3–4). Both approaches are valid; the native envelope keeps the signature *in* the document, the cosign approach keeps signing uniform across all your artifacts. The integrity requirement is the constant — an unsigned SBOM in a registry is a document any build step could have tampered with.

SPDX versus CycloneDX — the honest comparison

Engineers want a verdict. The honest answer is that both formats are mature, both are standardized, both can express a modern dependency graph with purls and hashes and licenses, and the interesting differences are about *heritage, ergonomics, and native support for adjacent concerns* — not about one being "better." Here is a fair table.

Dimension	SPDX	CycloneDX
Governance	Linux Foundation; **ISO/IEC 5962:2021**	OWASP; **ECMA-424 (2024)**
Heritage	License compliance / legal	Application security / vulnerability mgmt
Current version	3.0 (2024), 2.3 widely deployed	1.6 (2024)
Relationship model	Generic typed triples (`DEPENDS_ON` , `GENERATED_FROM` , dozens more) — very expressive	Purpose-built (`dependsOn` , `provides`) — terse, graph-shaped
Native vuln model	Weak historically; separate/ evolving	**Native** `vulnerabilities` array since 1.4
Native VEX	Via external/companion mechanisms	**Native** `analysis` block; inline or standalone
Services / runtime	Not natively modeled	**`services`** (SaaSBOM) — unique
Breadth strategy	**Profiles** (Core, Software, Security, Licensing, Build, AI, Dataset, …) in 3.0	**xBOM** (SBOM, VEX, SaaSBOM, ML-BOM, CBOM, HBOM, …) in one schema

Dimension	SPDX	CycloneDX
Serializations	Tag-value, JSON, RDF/Turtle, YAML, XML	JSON, XML, Protocol Buffers
Verbosity	Higher (explicit elements + relationships)	Lower (compact, especially JSON)
License expression	Origin of SPDX license IDs/ expressions	Reuses SPDX license IDs/ expressions
Tooling center of gravity	FOSSology, Tern, license/ compliance tooling, gov procurement	Dependency-Track, cdxgen, appsec/vuln tooling
Build provenance	`Build` profile / relationships	**`formulation`** section

Reading the table:

- **Heritage predicts fit.** If your primary driver is license compliance, open-source governance, or a government procurement mandate that names SPDX, SPDX's richer relationship vocabulary and ISO pedigree are advantages. If your primary driver is vulnerability management, VEX, and describing running services, CycloneDX's native security sections save you from bolting on external documents.
- **Expressiveness versus ergonomics.** SPDX's triple model can say more (patch-applied-to, generated-from, static-vs-dynamic link) but costs verbosity and complexity. CycloneDX's purpose-built sections are easier to produce and consume for the common cases and harder to stretch to the uncommon ones.
- **Breadth, two ways.** Both formats now reach beyond software — SPDX 3.0 via composable *profiles*, CycloneDX via the *xBOM* family. SPDX's profile model is arguably cleaner architecturally (opt into exactly the semantics you need); CycloneDX's single-schema model is arguably simpler operationally (one parser, one signer). Neither is obviously superior.

The real answer: you consume both

Here is the operational reality that dissolves most of the debate: **at fleet scale you do not choose a format — you receive both and**

normalize. Your own builds might standardize on one, but the SBOMs that arrive with third-party software, base images, and vendor components will be a mix. Worse, the *same tool* frequently emits both: `syft` **produces CycloneDX and SPDX** (and its own format) from the same scan; **Trivy** does likewise.

So the correct architecture is not "pick SPDX or CycloneDX." It is: **normalize every inbound SBOM, whatever its format, into one internal component-graph model at ingestion,** and treat SPDX-vs-CycloneDX as a serialization detail at the edge. That internal model keys components by purl (falling back to CPE, then hash), stores the dependency graph, and attaches vulnerability/VEX state as a separate overlay. Chapter 5 (SBOM Distribution, Storage, and Querying at Scale) builds exactly this. The format war is a non-event once you have a normalizer; format *bugs and lossiness* — what each format silently drops on round-trip — are the real operational concern, and that is a Chapter 7 topic.

Producing and consuming CycloneDX

Producing

Three common paths, in rough order of accuracy (Chapter 4 goes deep on why the ordering exists):

- `cdxgen` — the OWASP CycloneDX generator (`@cyclonedx/cdxgen`). Multi-ecosystem, CycloneDX-native, and the reference producer for the newer sections (services, evidence, ML-BOM). Runs against a project directory or a container image:

```bash # Generate a CycloneDX 1.6 SBOM for a project directory cdxgen -t java -o bom.json --spec-version 1.6 /path/to/checkout-service

# Generate for a container image cdxgen -t docker -o image-bom.json registry.example.com/checkout-service:2.4.0 ```

- `syft` — Anchore's scanner; emits CycloneDX *and* SPDX from one analysis, which is why it anchors so many normalization pipelines:

```
bash syft registry.example.com/checkout-service:2.4.0 \ -o cyclonedx-json=cdx.json \ -o spdx-json=spdx.json
```

- **Build-plugins** — ecosystem-native plugins that read the *resolved* dependency graph from the build tool itself, generally the most
```

accurate source because they see what the build actually resolved rather than inferring from lockfiles or binaries: `cyclonedx-maven-plugin`, `cyclonedx-gradle-plugin`, `@cyclonedx/cyclonedx-npm`, `cyclonedx-py` (Python), `cyclonedx-gomod` (Go), and others. Wiring these into CI is Chapter 4's subject.

Consuming

- **OWASP Dependency-Track** — the natural home of a CycloneDX BOM. You POST the BOM to its API; it stores the component graph and *continuously* re-correlates it against vulnerability sources (OSS Index, the NVD mirror, GitHub Advisories, and others), surfacing new findings against components you shipped months ago. This is the "enumerate once, re-match forever" decoupling from Chapter 1 realized as a product, and it ingests CycloneDX VEX to suppress findings you have triaged. A Dependency-Track instance fronting a fleet of services is, in effect, a *fleet inventory + vulnerability correlation stack* (Book 2, Chapter 6).

```
bash curl -s -X POST "https://dtrack.example.com/api/v1/bom" \ -H "X-Api-Key: $DT_API_KEY" \ -F "project=$PROJECT_UUID" \ -F "bom=@cdx.json"
```

- **`cyclonedx-cli`** — the Swiss-army knife: `validate`, `convert` (between JSON/XML/ protobuf and even to/from SPDX for the overlapping subset), `merge`, `diff`, and `sign`.

```
bash # Validate against the 1.6 schema before publishing cyclonedx-cli validate --input-file cdx.json --input-version v1_6 --fail-on-errors
```

- **Libraries** — `cyclonedx-python-lib`, `cyclonedx-core-java`, `cyclonedx-javascript-library`, and `cyclonedx-go` give you typed models so your normalizer is not string-munging JSON.

Validation

Always validate before you trust or publish. Two levels: **schema validation** (does it conform to `bom-1.6.schema.json`?) via `cyclonedx-cli validate` or any JSON-Schema validator, and **semantic checks** you add yourself (every `dependsOn` ref resolves to a `bom-ref`; every `affects[].ref` resolves; purls parse). Schema-valid does not mean *complete* or *accurate* — read the `compositions` block and the per-

component `evidence` before you treat an SBOM as ground truth. That gap between "valid" and "trustworthy" is the whole of Chapter 7.

Distributed-systems lens

Four ways CycloneDX specifically earns its place in a large backend estate:

- **SaaSBOM models the runtime supply chain.** A microservice architecture's real attack surface is not just its jars — it is the graph of services it calls, the data that crosses each edge, and which edges leave your trust boundary. The `services` model with `data` flows and `x-trust-boundary` is a machine-readable version of the data-flow diagram your threat model already needs (Book 1, Chapter 9 — the runtime supply chain). Generate a SaaSBOM per service from config/traffic analysis and you can query "which services send PII across a trust boundary" across the whole fleet.
- **The native vuln/VEX model streamlines scan-and-suppress.** Because the vulnerability, its rating, and its `analysis` (VEX) live *with* the component graph and are joined by `bom-ref`, the pipeline from "scanner found CVE-X in service-Y" to "team triaged it `not_affected` with justification `code_not_reachable`" to "suppressed everywhere that component appears" is one data model, not three integrations. Chapter 6 builds this pipeline; CycloneDX is the substrate that makes it terse.
- **Dependency-Track + CycloneDX is a fleet correlation stack.** Point every service's CI at Dependency-Track, feed it CycloneDX BOMs, and you have continuous re-matching of your entire inventory against fresh advisories, with VEX-based suppression — the fleet-scale version of Book 2, Chapter 6's SCA. The `serialNumber` + `version` scheme lets it track "the current BOM for service-Y" across rebuilds.
- **Normalize both formats at ingestion.** The distributed-systems reality is heterogeneity: many teams, many tools, inbound third-party BOMs in whatever format the vendor chose. Do *not* let format sprawl reach your inventory database. Normalize CycloneDX and SPDX into one internal graph model at the edge (Chapter 5), keyed on purl, with vuln/VEX as an overlay. Then CycloneDX's ergonomic advantages help you *produce* good BOMs and its native sections help

you *consume* rich ones, without the format choice leaking into every downstream query.

Key takeaways

- **CycloneDX is a security-first format.** Its OWASP / Dependency-Track heritage put a native `vulnerabilities` array, native VEX (`analysis`), a `services` model, and `evidence` structures in the core schema. SPDX learned security later; CycloneDX started there. Standardized as **ECMA-424 (2024)**, current spec **1.6**.
- **The object model is thin header + `metadata` + parallel arrays cross-linked by `bom-ref`.** `bomFormat` / `specVersion` / `serialNumber` (a `urn:uuid`) / `version` identify the document; `metadata.component` is the subject; `components`, `services`, `dependencies`, `vulnerabilities` hang off the root and reference each other by `bom-ref`.
- **The dependency graph is purpose-built and flat:** `dependencies[]` with `ref` / `dependsOn` (and `provides`, new in 1.6). Terser than SPDX's typed-triple graph, and less expressive — CycloneDX optimizes the query you run most.
- **The differentiators are the reason to reach for it:** native vuln + VEX joined to the graph by `bom-ref`; `services` (SaaSBOM) with classified data flows and trust boundaries; `compositions` (honest completeness signaling); `formulation` (build provenance); `evidence` (identification provenance + reachability); and native signing (JSF/XML-DSig).
- **The xBOM vision is one schema, many bills:** SBOM, VEX, VDR, SaaSBOM, HBOM, ML-BOM (AI supply chain — Book 7, Chapter 7), CBOM (cryptographic assets — the PQC-migration inventory), OBOM, MBOM. SPDX pursues the same breadth via *profiles*; both are valid, different architectures.
- **The format debate is mostly moot in practice.** Both are mature and standardized; tools like `syft` emit both. Do not choose — **normalize both into one internal model at ingestion** (Chapter 5) and treat SPDX-vs-CycloneDX as an edge serialization detail. The real risk is round-trip lossiness (Chapter 7), not format identity.
- **Produce with `cdxgen` / `syft` / build plugins; consume with Dependency-Track, the `cyclonedx-cli`, and typed libraries;**

validate against the schema — and remember that schema-valid is not the same as complete or accurate.

Further reading

- **ECMA-424**, "CycloneDX Bill of Materials Specification," 1st edition, Ecma International, June 2024.
- **OWASP CycloneDX** project documentation and the **CycloneDX 1.6 JSON Schema** (`cyclonedx.org/docs/1.6/`, `cyclonedx.org/schema/bom-1.6.schema.json`); the XML XSD and the Protocol Buffers `.proto`.
- **CycloneDX Authoritative Guides** (OWASP Foundation): the guides to SBOM, VEX, SaaSBOM, and the xBOM family, for the intended usage of each section.
- **OWASP Dependency-Track** documentation (`docs.dependencytrack.org`) for continuous BOM analysis, VEX ingestion, and the API.
- **`cdxgen`** (`github.com/CycloneDX/cdxgen`) and **`cyclonedx-cli`** (`github.com/CycloneDX/cyclonedx-cli`) documentation; **`syft`** (`github.com/anchore/ syft`) for dual-format generation.
- **Package URL (purl) specification** (`github.com/package-url/purl-spec`) and **NIST CPE 2.3**, for the component identity schemes CycloneDX carries.
- For the CBOM/PQC angle: **NIST FIPS 203/204/205** (the standardized post-quantum algorithms) and the CycloneDX cryptography-BOM guidance, for context on why a cryptographic-asset inventory matters.
- Cross-references in this suite: **Chapter 2 — SPDX in Depth** (the comparison's other half); **Chapter 5 — SBOM Distribution, Storage, and Querying at Scale** (normalization); **Chapter 6 — VEX and Vulnerability Correlation** (the scan-and-suppress pipeline); **Book 2, Chapter 6 — Software Composition Analysis**

in Depth; **Book 5 — Signing and Attestation** (SBOM integrity); **Book 7, Chapter 7** (AI/ML supply chain). ```

Chapter 4 — SBOM Generation: Tools, Techniques, and Accuracy

What this chapter covers. Chapters 2 and 3 gave you the two wire formats an SBOM can be written in. This chapter is about the far harder problem the formats quietly assume is already solved: **where does the data come from, and how accurate is it?** An SBOM is only as good as the process that produced it, and generation is the single most underestimated part of the entire SBOM value chain. A perfectly-formed CycloneDX 1.6 document that is missing a third of the components in your container is worse than useless — it is confidently wrong. The thesis of this chapter, stated once and defended throughout, is that **the point in your pipeline at which you generate an SBOM is the single biggest determinant of its accuracy**, larger than tool choice, larger than format, larger than anything in the schema. Everything else is a refinement on that decision.

Learning goals — after this chapter you should be able to:

- Map the four practical generation points — **source/manifest, build, binary/artifact, runtime** — onto the CISA SBOM types (Chapter 1) and state precisely **what each one sees and what it is structurally blind to**.
- Explain why **reading a lockfile** is the difference between a source SBOM that reflects declared intent and one that reflects resolved reality (Book 2, Chapter 2).
- Describe how the real tools — **Syft, Trivy, cdxgen, the CycloneDX language plugins, Go's build metadata, the Kubernetes `bom` tool, GitHub's dependency submission API** — actually work under the hood, and be honest about the limits of each, especially the limits of binary analysis.
- Enumerate the **hard cases** — static linking, vendoring, shaded JARs, bundled JavaScript, `curl | bash` installs, multi-stage build losses — and explain why each is a completeness gap no format can paper over.

- Distinguish **NTIA minimum-elements conformance** from **actual accuracy**, and confront the measurement problem: how do you even know an SBOM is complete?
- Design SBOM generation as a **standardized, automated, per-artifact CI step** inherited by every service via the paved road, with the SBOM signed and its own provenance recorded.

A boundary note. This chapter assumes Chapter 1's six SBOM types and the "graph, not list" framing, Book 2's identifier model (purl versus CPE, Chapter 5) and its SCA pipeline (Chapter 6), and Book 2, Chapter 2's account of resolution and lockfiles. It hands off *distribution and storage* to Chapter 5, *VEX* to Chapter 6, and a full honest accounting of quality limits to Chapter 7. Here the subject is narrowly the act of **production** and its accuracy.

When you generate determines what you capture

Return to the six SBOM types from Chapter 1 — Design, Source, Build, Analyzed, Deployed, Runtime — but collapse them to the four points where you can *actually run a tool and get a document*: at the **source** (parsing manifests and lockfiles), during the **build** (instrumenting the compiler or build graph), against the **built artifact** (scanning a binary or image after the fact), and at **runtime** (observing what a process loads). Each of these observes the software at a different moment, and — this is the whole point — **each moment exposes different information and hides different information.** No tool, however good, can report on data that is not visible at the point where it runs.

```
flowchart LR
    subgraph SRC["SOURCE / MANIFEST"]
        S1["sees: declared deps,
lockfile-resolved versions,
dev vs prod split"]
        S2["blind to: OS packages,
vendored/static C,
base image, runtime downloads"]
    end
    subgraph BLD["BUILD"]
        B1["sees: the REAL resolved
graph, generated code,
native + build-only deps"]
        B2["blind to: things added
after the build
(base image extras, sidecars)"]
    end
    subgraph BIN["BINARY / ARTIFACT"]
        N1["sees: everything present
in the image — OS pkgs,
embedded manifests, Go buildinfo"]
        N2["blind to: stripped static
libs, un-fingerprintable
vendored source, intent"]
    end
    subgraph RUN["RUNTIME"]
        R1["sees: what is actually
LOADED and executed"]
        R2["blind to: code paths not
exercised in the window,
anything not yet run"]
    end

    SRC -->|"build"| BLD -->|"package"| BIN -->|"deploy + run"| RUN

    classDef sees fill:#14532d,stroke:#4ade80,color:#fff;
    classDef blind fill:#7f1d1d,stroke:#f87171,color:#fff;
    class S1,B1,N1,R1 sees;
    class S2,B2,N2,R2 blind;
```

Read that diagram as a ledger of trade-offs, not a ranking. There is no single "best" generation point; there is a *different* completeness profile at each, and the profiles are partly complementary. The source point knows your *intent* — which dependency you chose, in which manifest, which a scanner can map back to a one-line pull request — but it cannot see the Alpine base image's `musl` or `busybox`. The binary point sees that base image in full but cannot tell you which line of which file to edit, and it must *guess* the identity of anything that does not carry structured

metadata. This asymmetry is why Chapter 1 insisted that mature programs generate *more than one type and reconcile them*, and it is the organizing principle for everything below.

Source and manifest-based generation

The cheapest and most CI-friendly point is the source tree. A tool reads the ecosystem's dependency manifests — `package.json`, `pom.xml`, `build.gradle`, `go.mod`, `requirements.txt`, `Cargo.toml`, `*.csproj` — and emits a component for each declared dependency. This is fast (seconds), needs no build, no network, no container runtime, and slots trivially into a pull-request check. It is the natural producer of the **Source SBOM** type.

The critical subtlety, and the thing that separates a competent source SBOM from a misleading one, is **manifest versus lockfile**. A manifest declares *constraints*: `"express": "^4.18.0"` means "some 4.x at or above 4.18.0." It does not tell you which version actually got installed, and it says *nothing* about the transitive closure — the hundreds of packages Express pulls in. Book 2, Chapter 2 dissected resolution in detail; the consequence for SBOMs is stark. A tool that parses only the manifest produces a document that is **wrong in two directions at once**: it lists version ranges instead of resolved versions (so downstream vulnerability matching, which needs an exact version, has nothing to match on), and it omits the entire transitive graph (so the component with the CVE — almost always transitive, per Chapter 1 — is simply absent).

The lockfile fixes both. `package-lock.json`, `yarn.lock`, `poetry.lock`, `go.sum`, `Cargo.lock`, `Gemfile.lock` record the *fully resolved* transitive closure with **exact, pinned versions and often integrity hashes**. A source SBOM generated from the lockfile is a genuinely useful artifact: exact versions, complete transitive graph, per-component hashes you can verify. The single most important rule of source-based generation is therefore: **generate from the lockfile, never from the bare manifest.** A tool pointed at a repo with no lockfile is producing a wish list, not an inventory.

Even at its best — reading lockfiles — source generation has three structural blind spots that no amount of tooling polish removes:

- **It sees only what the package manager manages.** OS packages baked into a base image (`apt` / `apk` / `yum` layers), a C library vendored

into `third_party/` and compiled directly, a binary fetched by a `curl | bash` line in a Dockerfile — none of these appear in any language manifest, so none appear in the SBOM.

- **It reflects declaration, not the build's actual choices.** If your build uses a private mirror, applies a patch, or resolves differently than a clean `npm install` would (a `resolutions` / `overrides` block, a monorepo workspace hoist), the manifest-plus- lockfile can still diverge from what was truly linked.
- **It cannot tell shipped from unshipped.** `devDependencies`, test frameworks, and build tooling sit in the same lockfile as production dependencies. A naive source SBOM lists your test runner as a component of your shipped service. Good tools honor the scope metadata (`dev`, `optional`, `test`) and let you filter; many downstream consumers do not, and the result is inflated component counts and false-positive vulnerabilities in code that never ships.

Source generation is the right *first* SBOM — attributable, cheap, PR-native — but treating it as the *only* SBOM is the most common accuracy mistake in the field.

Build-time generation

The highest-fidelity point is *inside the build itself*. Instead of guessing from manifests before, or fingerprinting from artifacts after, you instrument the build system to record **what it actually resolved, compiled, and linked** — the real dependency graph as the build saw it, including generated code, native toolchain inputs, and build-only dependencies that no runtime artifact will ever reveal. This produces the **Build SBOM** type, and it is the gold standard for one reason: the build system is the only actor that has *ground truth* about what went into the artifact. It is not inferring; it *is* the process that made the decisions.

Concretely this looks like: a Maven or Gradle plugin that hooks the resolved dependency graph the build already computed; `go version -m` reading the module set the Go toolchain stamped into the binary; a Bazel aspect that walks the action graph and emits a component per resolved external repository. Because the data comes from the build's own internal state, versions are exact, the transitive graph is complete *and* reflects the real resolution (patches, overrides, mirrors and all), and build-time-only inputs are captured.

Build-time generation is also the hardest to implement, which is why it is rarer than it should be. It requires modifying the build — a plugin, an aspect, a wrapper — for every build system in your estate, and polyglot organizations run many. Its payoff is highest exactly where builds are already **hermetic and reproducible** (Book 4, Chapter 2): a hermetic build has already declared its complete, pinned input set, so emitting an SBOM from it is nearly free and nearly perfect. And build-time SBOM generation is the natural companion to **build provenance** (Book 4, Chapter 3; SLSA): the same instrumented build that attests *how* it ran can emit *what* it consumed, and the two together — provenance plus SBOM, both signed — are the strongest verifiable statement you can make about an artifact. We return to this convergence in the operationalizing section, because on a shared build platform it is where the whole strategy pays off.

The one thing build-time generation does *not* see: whatever gets added to the artifact *after* the build it instrumented. If your build produces an application layer that is then stacked on a base image assembled elsewhere, the base image's OS packages are outside the build's view. That gap is exactly what artifact analysis fills.

Binary and artifact analysis

The most widely deployed point today is *after* the build: point a tool at a finished container image, filesystem, or binary and have it **catalog what is actually present**. This produces the **Analyzed SBOM** type, and it is what Syft, Trivy, and similar tools do when you hand them an image reference. Its great virtue is coverage of *the whole artifact*: it sees the base image's OS packages (by reading the `dpkg`, `rpm`, or `apk` package databases that the package managers left on disk), the language dependencies (by reading manifests and lockfiles that happen to be *inside* the image), and structured metadata embedded in binaries. It catches precisely the things source generation is blind to.

Here is where honesty about mechanism matters, because "binary analysis" is routinely oversold. These tools are overwhelmingly **metadata readers, not decompilers.** What they actually do is find and parse files that already describe components:

- **OS packages:** parse the on-disk package databases — `/var/lib/dpkg/status`, `/lib/apk/db/installed`, the `rpm` BerkeleyDB/sqlite

database. This is reliable because the package manager wrote an authoritative record.

- **Language packages:** find and parse manifests/lockfiles inside the image (`node_modules` with `package.json` files, installed Python `*.dist-info/METADATA`, a JAR's embedded `pom.properties` and `MANIFEST.MF`, a Ruby `Gemfile.lock`).
- **Structured build metadata inside binaries:** the few genuinely "binary" catalogers that work well rely on the build having *stamped* the metadata. Go binaries carry a module list you can read with `go version -m` (Syft has a cataloger for exactly this); .NET carries a `*.deps.json`; a Rust binary built with the right tooling can carry an audit section. These are reliable *because* the toolchain embedded structured data — not because the tool reverse-engineered the machine code.

What these tools **cannot** reliably do is identify a component that left no metadata behind. A statically-linked C library compiled into a stripped executable is, to a cataloger, an anonymous blob of machine code. There is no package database entry, no manifest, no embedded version string it can trust. The best a tool can do is opportunistic fingerprinting — matching known byte sequences or version-string patterns — which is heuristic, incomplete, and prone to both misses and misattributions (Book 2, Chapter 6 covered the shaded-JAR and static-linking identification problem in depth). **Do not believe any claim that a scanner "sees inside" arbitrary binaries.** It sees metadata that someone chose to leave; where no one left any, it is blind, and — worse — it is silently blind, reporting a clean catalog of the things it *could* read while omitting the things it could not.

Runtime and deployed generation

The last point is the running system. A **Deployed SBOM** captures what is actually installed in an environment — the artifact plus host packages, sidecars, and operator patches — while a **Runtime SBOM** goes further and observes what a process actually **loads and executes**, typically via an eBPF probe or an in-process agent watching `dlopen`, `mmap` of shared objects, class loading, or module imports.

Runtime generation has a unique and valuable property: it is the only point that can distinguish **present on disk** from **actually loaded**. That distinction is the first (and only the first) step toward reachability — a

library that is installed but never loaded cannot be exploited through your process, and a runtime SBOM can say so where a static one cannot (Book 2, Chapter 7 on reachability and exploitability). This makes runtime data the most *relevant* input for "what is actually exploitable here."

Its limitation is the mirror image of its strength: **it only sees what ran.** A code path exercised for the first time an hour after you captured the SBOM is invisible; a component loaded only under a rare feature flag or error condition may never appear. Runtime SBOMs are therefore not a *complete* inventory of the artifact — they are an inventory of the *observed* subset — and they are best used to *enrich* a build- or binary-time SBOM (annotating which components were seen loaded) rather than to *replace* it. Treat runtime as the reachability signal, not the ground-truth inventory.

Putting the four points in one table

Generation point	CISA type	What it captures well	Structurally blind to	Cost / friction	Best use
Source / manifest (lockfile)	Source	Declared + resolved language deps, exact versions, dev/prod scope, PR attribution	OS packages, vendored/ static C, base image, runtime downloads	Very low; no build needed	PR-time check; "w to fix" attributio
Build (instrumented)	Build	The **real** resolved graph, generated code, native + build-only inputs	Anything added after this build (base image extras)	High to implement; near-free once hermetic	Gold standard pairs wit provenan

Generation point	CISA type	What it captures well	Structurally blind to	Cost / friction	Best use
Binary / artifact (Syft, Trivy)	Analyzed	Whole-image OS + language pkgs, embedded build metadata	Stripped static libs, un-fingerprintable vendored code, *intent*	Low; runs on the finished artifact	Coverage the full shipped image
Runtime (eBPF / agent)	Deployed / Runtime	What is actually loaded/ executed; reachability signal	Anything not exercised in the observation window	Medium; needs production instrumentation	Enrich, n replace; exploitab triage

The tools, and how they actually work

The market has consolidated around a handful of open-source generators plus a set of language- and build-native plugins. What follows is a mechanism-level tour; the goal is to know *what each tool can and cannot see*, which follows directly from *where it runs*.

Syft (Anchore)

Syft is the de-facto open-source tool for the binary/artifact point. You point it at a container image, a directory, or an archive, and it emits an SBOM in SPDX or CycloneDX (JSON or tag-value), plus its own `syft-json`.

```
# Catalog a container image and emit CycloneDX JSON
syft registry.example.com/payments/api:v4.7.1 -o cyclonedx-json > api-
v4.7.1.cdx.json

# Catalog a local directory as SPDX
syft dir:./service -o spdx-json > service.spdx.json
```

Its architecture is a set of **catalogers** — independent modules, each specialized for one package type. There is a `dpkg` cataloger that parses the Debian package database, an `apk` cataloger for Alpine, an `rpm`

cataloger, a `node` cataloger that walks `node_modules` and reads each `package.json`, a `python` cataloger for `*.dist-info`, a `java` cataloger that cracks open JARs/WARs/EARs to read `pom.properties` and `MANIFEST.MF`, a `go-module` cataloger that reads the buildinfo stamped into Go binaries, and so on. Syft runs the catalogers over the unpacked filesystem (for an image, over the squashed layer contents), merges their findings, and assigns each component a purl. Its strengths and limits fall directly out of this design: it is excellent where an authoritative metadata source exists (OS databases, language manifests, Go buildinfo) and blind where none does. Syft does not decompile; a statically-linked C dependency with no metadata will not appear. Knowing the cataloger list is knowing Syft's coverage.

Trivy (Aqua)

Trivy is a broader security scanner — vulnerabilities, misconfigurations, secrets, licenses — that *also* generates SBOMs, using cataloging logic conceptually similar to Syft's (analyzers per package type over images, filesystems, and repositories). Its distinguishing feature is that generation and vulnerability matching live in one tool: it can emit a CycloneDX or SPDX SBOM and, in the same pass or a later one, match that inventory against its vulnerability database.

```
# Generate a CycloneDX SBOM for an image
trivy image --format cyclonedx --output api.cdx.json registry.example.c
om/payments/api:v4.7.1

# Later: scan the stored SBOM against today's advisories (the "match
forever" step, Ch 1)
trivy sbom api.cdx.json
```

That last command is worth dwelling on: it operationalizes Chapter 1's decoupling insight — enumerate the inventory once, then re-match the *stored* SBOM against a moving advisory feed without re-scanning the artifact. Trivy's generation shares source/binary blind spots with Syft (same fundamental reliance on discoverable metadata); the convenience is the unified pipeline, not a fundamentally different cataloging capability.

cdxgen (OWASP CycloneDX)

cdxgen is the CycloneDX project's own multi-language generator, and it leans toward the *source and build* end rather than pure artifact scanning.

It supports a wide spread of ecosystems and can invoke the ecosystem's own tooling to resolve dependencies (e.g., driving Maven or Gradle to compute the real resolved graph rather than parsing `pom.xml` naively), which pushes it closer to build-fidelity for those languages. It also produces richer CycloneDX-native metadata and integrates with the project's reachability/evidence tooling. Where Syft asks "what is in this artifact?", cdxgen more often asks "what does this project resolve to?", which makes it a strong choice at the CI-build point for polyglot repos.

Language- and build-native generators

The most accurate SBOMs for a given ecosystem usually come from a tool that lives *inside* that ecosystem's build, because it reads the same resolved graph the build itself computes — this is build-time generation by another name:

- **Java:** the `cyclonedx-maven-plugin` and `cyclonedx-gradle-plugin` hook the build's own resolved dependency graph. Because they run as part of the build, they see the exact versions and scopes (`compile`, `runtime`, `test`, `provided`) the build resolved — far more reliable than cracking JARs after the fact.
- **JavaScript:** `cyclonedx-npm` (and the CycloneDX Node tooling) reads the installed tree and lockfile to emit CycloneDX; some npm versions also expose an `npm sbom` command.
- **Python:** `cyclonedx-py` reads the installed environment, `requirements.txt`, `poetry.lock`, or `Pipfile.lock`.
- **Go:** the toolchain itself is the generator. `go version -m ./binary` prints the module set and versions the compiler stamped into the binary — genuine build-time truth embedded in the artifact. Syft's Go cataloger and cdxgen both read exactly this data; Go's design makes its binaries unusually honest about their contents.
- **Rust / .NET:** `cargo` audit metadata and the `*.deps.json` that .NET emits play the same embedded-metadata role.

The pattern: when the ecosystem's build can emit the SBOM, prefer it — it is the closest practical approximation to a Build SBOM for that language, and it sidesteps the identity- guessing that artifact scanners must do.

Kubernetes `bom`, and platform tooling

The Kubernetes project maintains `bom`, an SPDX-focused tool used to generate the official SBOMs for Kubernetes releases; it can catalog files, images, and directories and is a reference for how a large OSS project bakes SBOM generation into its release engineering. It is worth knowing as an example of SBOM generation as *release infrastructure* rather than a bolt-on.

GitHub's dependency graph and Dependency Submission API

GitHub occupies a distinct niche: it derives a dependency graph from the manifests and lockfiles in a repository (a source-point view), and — more interestingly — exposes a **Dependency Submission API** that lets a CI job *submit* a computed dependency graph back to GitHub. This matters because it lets a **build-time** resolver (a Gradle or Bazel step that knows the true resolved graph) push that higher-fidelity data into GitHub's graph, which then drives Dependabot alerts against the real transitive closure rather than a naive manifest parse. The `actions` ecosystem also offers ready-made steps (Anchore's `sbom-action` wrapping Syft, Trivy's action, the CycloneDX actions) so a team can add SBOM generation to a workflow in a few lines. The takeaway is architectural: GitHub's graph is a source-point view *unless* you feed it build-point data through submission, at which point it inherits build-point accuracy.

```
flowchart TD
    START["Need an SBOM. Where do I generate?"]
    START --> Q1{"Do I control the build?"}
    Q1 -->|No — third-party
artifact / image| BIN["Artifact scan:
Syft / Trivy on the image"]
    Q1 -->|Yes| Q2{"Is the build hermetic /
plugin available?"}
    Q2 -->|Yes| BUILD["Build-time:
language plugin / Bazel aspect /
go version -m — highest fidelity"]
    Q2 -->|No, but I have a lockfile| SRC["Source-point:
cdxgen / cyclonedx-* from LOCKFILE"]
    Q2 -->|No lockfile| FIX["Fix that first:
commit a lockfile, then source-point"]
    BUILD --> COMBINE
    SRC --> COMBINE
    BIN --> COMBINE
    COMBINE["Also scan the final IMAGE (Syft/Trivy)
to catch base-image + OS packages"]
    COMBINE --> DONE["Merge: build/source SBOM (intent)
+ image SBOM (coverage)"]
```

Accuracy, completeness, and the hard cases

This is the spine of the chapter, and it requires bluntness. SBOM generation is not a solved problem; it is a collection of good-enough heuristics with well-known failure modes. A practitioner who does not know the failure modes will ship confidently wrong inventories.

Why two tools produce different SBOMs for the same artifact

Run Syft and Trivy against the identical container image and you will get two SBOMs that disagree — different component counts, different versions, different identifiers, sometimes different *components*. This is not a bug in either tool; it is inherent, and understanding why inoculates you against the naive expectation that an SBOM is a deterministic property of an artifact.

```
flowchart TB
    IMG["Same image:
payments/api:v4.7.1"]
    IMG --> T1["Tool A (Syft)"]
    IMG --> T2["Tool B (Trivy)"]

    T1 --> A1["Cataloger set A:
includes a binary cataloger
for component X"]
    T2 --> B1["Analyzer set B:
no analyzer for X →
X missing"]

    A1 --> A2["purl for lib Y:
pkg:golang/...@v1.4.0"]
    B1 --> B2["Y identified from a
different source →
pkg:generic/...@1.4"]

    A2 --> A3["Counts dev deps
from lockfile"]
    B2 --> B3["Filters dev deps →
fewer components"]

    A3 --> R["Two valid but
DIFFERENT SBOMs"]
    B3 --> R
```

The disagreements come from four independent sources. First, **different cataloger coverage**: if tool A has an analyzer for a package type and tool B does not, A finds components B misses. Second, **different identification**: two tools may find the same component but mint different purls or CPEs for it (different casing, different namespace assumptions, one produces `pkg:golang/...` where another falls back to `pkg:generic/...`), which then causes *downstream* vulnerability matching to diverge even when the underlying inventory agrees (Book 2, Chapter 5). Third, **source-versus-binary vantage**: a source- point tool and a binary-point tool are literally looking at different data. Fourth, **scoping and merging policy**: whether dev dependencies are included, whether duplicate components across layers are deduplicated, how relationships are expressed. This is the **reproducibility problem** for SBOMs, and it has a sobering corollary for Chapter 7: "complete" and "correct" are not tool-independent properties you can verify by re-running a different scanner and checking for agreement — the scanners are *supposed* to disagree.

The hard-to-capture components

Each of the following is a component class that routinely goes missing, and each is a concrete completeness gap you should be able to name on sight:

- **Statically-linked C/C++.** Compiled into the binary with no package metadata, no version symbol you can trust, often stripped. To a cataloger it is anonymous machine code. This is the single worst gap for native-heavy software, and there is no general solution — only heuristic fingerprinting that misses more than it catches.
- **Vendored code.** A dependency copied wholesale into your source tree (`vendor/`, `third_party/`, a pasted-in single-file library) and compiled as if it were yours. It has no manifest entry, so source tools miss it; if it compiles into a stripped binary, artifact tools miss it too. It is present, exploitable, and invisible.
- **Shaded / relocated JARs.** The Maven Shade plugin (and its Gradle equivalent) inlines dependencies into an uber-JAR and *renames their packages* (`org.apache.foo` → `com.myapp.shaded.org.apache.foo`). The code is present but its coordinates are gone; a scanner that keys on package paths cannot recognize it. This is *exactly* how Log4Shell hid in fat JARs (Chapter 1), and it remains a first-class evasion of naive cataloging.
- **Minified / bundled JavaScript.** A webpack/Rollup/esbuild bundle concatenates dozens of npm packages into a single minified `.js` file with all package boundaries erased. Point a scanner at the *deployed* bundle and it sees one file, not the fifty libraries inside it. You must generate the SBOM *before* bundling, from the lockfile — after bundling the information is destroyed.
- **Dynamically-downloaded dependencies.** Anything fetched at build or first-run — `pip install` from a `postinstall` script, a plugin downloaded on first launch, a model or binary pulled from a bucket at container start. It is in none of the manifests because it is not resolved by the package manager at all.
- **`curl | bash` and ad-hoc Dockerfile installs.** `RUN curl -fsSL https://.../install.sh | bash` or `RUN wget .../tool && chmod +x tool` drops a binary into the image with zero package-manager involvement, so no package database records it. Artifact scanners

see an executable they cannot identify; source tools never knew it existed.

- **Firmware and OS-below-the-package-manager layers.** Kernel modules, firmware blobs, and files placed on the image outside any package manager (`ADD` of a tarball) are invisible to package-database cataloging.
- **Multi-stage build losses.** A multi-stage Dockerfile builds in a fat stage full of compilers and manifests, then `COPY --from=build /app/binary /` into a minimal final stage. All the build metadata — the lockfiles, the `node_modules`, the `go.mod` — lives in the discarded stage. Scan the *final* image and you get a near-empty SBOM for a binary that contains a hundred dependencies. The fix is to generate the SBOM in the build stage, where the metadata still exists, and carry it forward — not to scan the slimmed artifact.

The through-line: **information is destroyed as software moves down the pipeline.** Bundling erases package boundaries; static linking erases metadata; multi-stage builds discard the stage that knew the truth. Generate where the information still exists, then propagate the SBOM forward — do not try to reconstruct it after it is gone.

Identifier quality: garbage in, garbage out

Even for components a tool *does* find, the *quality of the identifiers* determines whether the SBOM is usable downstream. Book 2, Chapters 5 and 6 established that vulnerability matching keys on purl (for OSV) and CPE (for NVD), and that a wrong or missing identifier means a missed vulnerability or a false positive. Generation is where those identifiers are born, and it is where they most often go wrong:

- **Malformed or fallback purls.** A cataloger that cannot confidently determine ecosystem or namespace emits `pkg:generic/name@version`, which matches nothing in OSV. The component is "in the SBOM" but invisible to vulnerability tooling.
- **Version inaccuracy.** A version scraped from a filename or a `MANIFEST.MF` may be wrong (a build timestamp, a `0.0.0` placeholder, a range). Vulnerability matching is exact; a wrong version silently mismatches.
- **Empty supplier fields.** The NTIA minimum elements require a *Supplier Name*, and it is the field most often left blank because

generators frequently cannot determine it from an artifact. An SBOM full of empty suppliers technically fails the minimum elements even while looking complete.

The lesson for downstream: an SBOM's usefulness for vulnerability management is bounded by its *worst* identifiers, not its component count. A thousand components with fallback purls answer no queries.

Minimum-elements conformance is not accuracy

It is tempting to treat "passes the NTIA minimum elements" as "is a good SBOM." It is not. The minimum elements (Chapter 1) specify that each component *carry* certain fields — supplier, name, version, unique identifier, dependency relationship, plus author and timestamp for the document. A conformance checker verifies those fields are **present and well-formed**. It cannot verify they are **true**, and — crucially — it cannot verify the component *list itself is complete*. An SBOM that lists ten components, each with a perfectly- formed purl and a supplier, **passes** the minimum elements even if the artifact actually contains two hundred components. Conformance is a syntactic floor; accuracy is a semantic property the floor does not touch. A program that measures itself only by conformance rate is measuring the wrong thing and will believe it is done long before it is.

The measurement problem

Which raises the hardest question in the chapter: **how do you even know your SBOM is complete?** To verify completeness you would need an independent, authoritative list of what is *actually* in the artifact — but that list is precisely the thing the SBOM was supposed to produce, and no oracle hands it to you. You cannot verify completeness by re-running a different tool, because (per the reproducibility discussion) the tools are supposed to disagree; agreement between two tools with the same blind spot is not evidence of completeness, only of shared blindness. In practice teams triangulate — cross-check a source SBOM against an image SBOM, look for the "impossible" empty SBOM that signals a multi-stage loss, spot-check against known-present components — but there is no clean measurement. NTIA's own baseline builds in the concept of **known unknowns** precisely because completeness cannot be assumed or easily proven; an honest SBOM should be able to *declare where it may be incomplete*. Chapter 7 takes this measurement problem as its central

subject; for now, the operational stance is humility: assume your SBOMs have gaps, know where the common ones are, and design generation to minimize them rather than to pretend they do not exist.

Operationalizing generation

Knowing *where* to generate and *which* tools do what is the theory; the practice is making generation happen automatically, everywhere, on every build. The failure mode here is predictable and near-universal: a team stands up SBOM generation as a manual, one-off, or "we'll run Syft before the audit" activity, and ends up with SBOMs for a handful of flagship services and nothing for the long tail — which is exactly where the forgotten, unpatched, vulnerable stuff lives.

Generate at build time, per artifact, automatically

The paved-road principle (Book 1, Chapters 9 and 10) applies directly: SBOM generation must be a **standardized CI step that every service inherits by default**, not a thing each team remembers to add. Every build of every artifact emits an SBOM, tagged with the exact version it describes, so that **every deployed version has a corresponding SBOM** — because you cannot answer "where is component X across everything we run" (Chapter 1's killer use case) if some fraction of your running versions never produced an SBOM at all.

The canonical per-artifact pipeline is a short, fixed sequence:

```
flowchart LR
    BUILD["Build artifact
(image / binary)"] --> GEN["Generate SBOM
(build-plugin + Syft on image)"]
    GEN --> SIGN["Sign SBOM
(cosign attest)"]
    SIGN --> ATTACH["Attach to artifact
(OCI referrer / attestation)"]
    ATTACH --> PUSH["Push to central
SBOM store (Ch 5)"]
    PUSH --> QUERY["Fleet query, SCA,
VEX (Ch 6), policy"]
```

Concretely, in a GitHub Actions workflow this is a handful of steps bolted onto the build that produces the image:

```
# After the image is built and pushed as ${IMAGE}@${DIGEST}
- name: Generate SBOM from the built image
  uses: anchore/sbom-action@v0
  with:
    image: ${{ env.IMAGE }}@${{ env.DIGEST }}
    format: cyclonedx-json
    output-file: sbom.cdx.json

- name: Sign the SBOM as an attestation on the image
  run: |
    cosign attest --yes \
      --type cyclonedx \
      --predicate sbom.cdx.json \
      ${IMAGE}@${DIGEST}
```

Two design points hide in those few lines. First, generation is keyed to the **digest**, not a mutable tag — the SBOM describes an immutable artifact, so the association cannot rot when `:latest` moves. Second, the SBOM is attached *as an attestation on the artifact itself* (an OCI referrer), so it travels with the image and can be pulled back later by digest; Chapter 5 develops the store-and-query side of this, and distribution is where "attach it to the artifact" becomes a fleet-wide discipline.

Combine generation points for coverage

No single point is complete, so mature pipelines run **more than one and merge**. The common, pragmatic combination is a **build-or-source SBOM** (accurate about intent, attributable to a PR, produced by the language plugin or from the lockfile) *plus* an **image SBOM** (Syft or Trivy over the final container, catching base-image OS packages the source never declared). The build-time SBOM is the gold standard *where achievable*; the image scan is the safety net that catches what the build's view could not — the base image, the `curl | bash` install, the `COPY`'d tarball. Reconciling the two (deduplicating overlaps, keeping the higher-quality identifier where both saw a component) yields materially better coverage than either alone, and it directly implements Chapter 1's "generate more than one type and reconcile" prescription.

Sign the SBOM and record its provenance

An SBOM whose origin you cannot verify is a weak artifact: if you cannot prove *which build* produced it, *when*, and *with what tool*, an attacker who can substitute a doctored SBOM can make a vulnerable artifact look

clean. So the SBOM itself needs the same treatment as any other build output:

- **Sign it.** `cosign attest` (above) binds the SBOM to the artifact's digest with a signature, so a verifier can confirm this SBOM belongs to this image and was not swapped. Book 5 is the full treatment of signing and attestation.
- **Record the SBOM's own provenance.** The document metadata (Chapter 1) — author (the CI system, not a person), timestamp, and the exact tool *and version* that generated it — is not bureaucratic filler; it is what lets you later reason about *which* SBOMs were produced by a generator you have since learned has a blind spot, and regenerate them. An SBOM is a claim; a claim you can trace to a specific automated producer at a specific time is a claim you can trust and, when necessary, distrust deliberately.

The convergence to aim for, on a mature platform, is this: the same instrumented, hermetic build that emits **SLSA provenance** (Book 4, Chapter 3) also emits the **Build SBOM**, and both are **signed** (Book 5) and pushed to the store as attestations on the artifact. At that point "what went in" and "how it was built" are two verifiable, linked statements about one immutable digest — which is the strongest position an SBOM program can occupy.

Distributed-systems lens

At the scale of one repository, SBOM generation is a tooling choice. At the scale of hundreds of polyglot services across dozens of teams deploying many times a day, it is an **inventory- production system**, and the constraints change character entirely.

The first reality is that **consistency beats perfection.** A single team can lovingly hand- tune a build-time SBOM with reconciled source-and-image data and perfect identifiers. That does not scale to three hundred services in eight languages. What scales is a **standardized, automated generation step delivered through the paved road** (Book 1, Chapters 9–10): the shared CI template, the base pipeline, the golden workflow that every service inherits by default and that emits a per-artifact SBOM without the service team writing a line of SBOM config. A uniformly *good-enough* SBOM for all three hundred services is worth far more than an excellent SBOM for the ten that had time to care — because the killer

use case is a *fleet* query, and a fleet query is only as complete as its worst-covered corner. The strategic goal is *coverage of the estate*, not local optimality.

The second reality is that generation feeds a **central store, per artifact, per build** (Chapter 5). The output of the generation step is not a file a team keeps; it is a document pushed to a fleet-wide inventory, keyed by artifact digest and version, so that the stream of SBOMs from hundreds of pipelines becomes one queryable graph. Generation and storage are two halves of one system: generation that does not flow into the store answers no fleet questions, and a store with no automated generation feeding it is empty.

The third reality is the highest-leverage one: a **shared, hermetic build platform** (Book 4, Chapter 10) can make high-fidelity SBOM generation a property every tenant gets **for free.** If the build platform is instrumented once to emit a Build SBOM (and provenance) from its own action graph, then every team that builds on it inherits gold-standard SBOMs without doing anything — the platform, not the tenant, carries the cost of build-time instrumentation, and it carries it once. This is the distributed-systems payoff of build-time generation: it is the hardest technique to implement, but on a shared platform it is implemented *once* and amortized across every service, turning the most accurate SBOM type from a per-team luxury into a platform default. The same argument recurs throughout this curriculum — signing, provenance, policy — and it lands here with particular force, because SBOM accuracy is determined by generation point, and the platform is the one place you can move every tenant to the best generation point at once.

Put the three together and the target architecture is legible: **every artifact, built on a shared instrumented platform, emits a signed Build SBOM plus an image-scan SBOM, keyed to its digest, pushed automatically to a central graph store, for every service, by default.** No team opts in; no team can forget; the long tail is covered because coverage is inherited, not elected. That is what "operationalized SBOM generation" means at fleet scale, and it is the precondition for everything Chapter 5 does with the resulting store.

Key takeaways

- **Where you generate is the biggest determinant of accuracy** — bigger than tool or format. The four practical points (source/manifest, build, binary/artifact, runtime) map onto CISA's types and each sees different truth and is blind to different things. There is no single best point; there is a different completeness profile at each.
- **Source generation must read the lockfile, not the bare manifest.** The manifest declares version *ranges* and omits the transitive closure; the lockfile gives exact versions and the full graph. It still cannot see OS packages, vendored/static code, or anything the package manager does not manage.
- **Build-time generation is the gold standard** — it reads the real resolved graph from the build's own state, captures generated and build-only inputs, and is near-free once builds are hermetic. It pairs naturally with SLSA provenance (Book 4, Chapter 3).
- **Binary/artifact tools (Syft, Trivy) are metadata readers, not decompilers.** They excel where authoritative metadata exists (OS package databases, embedded manifests, Go buildinfo) and are *silently* blind where none does — stripped static libraries, un-fingerprintable vendored code. Do not overstate any tool's ability to "see inside" binaries.
- **Runtime generation uniquely distinguishes loaded from present** (the start of reachability, Book 2, Chapter 7) but only sees what ran; use it to *enrich*, not replace, a build- or binary-time inventory.
- **Two tools disagree on the same artifact by design** — different catalogers, different identifier minting, source-versus-binary vantage, different scoping. Agreement is not proof of completeness; it can be shared blindness.
- **Know the hard cases cold:** static linking, vendoring, shaded/relocated JARs, bundled/ minified JS, dynamically-downloaded deps, `curl | bash` installs, firmware layers, and multi-stage build losses. Information is destroyed as software moves down the pipeline — generate where it still exists and propagate the SBOM forward.
- **Identifier quality gates downstream usefulness.** Fallback `pkg:generic` purls, wrong versions, and empty supplier fields turn a

"complete-looking" SBOM into one that answers no vulnerability queries.

- **NTIA minimum-elements conformance is a syntactic floor, not accuracy.** Passing it proves fields are present and well-formed, not that they are true or that the component list is complete. And you cannot easily *measure* completeness — there is no oracle — which is why honest SBOMs declare their known unknowns (Chapter 7).
- **Operationalize it as a paved-road CI step:** per-artifact, per-build, automatic, keyed to digest; combine a build/source SBOM (intent) with an image SBOM (coverage); sign the SBOM and record its own provenance; push to the central store (Chapter 5).
- **At fleet scale, consistency beats perfection, and a shared hermetic build platform (Book 4, Chapter 10) delivers high-fidelity build-time SBOMs to every tenant for free** — the most accurate technique, implemented once and amortized across the whole estate.

Further reading

- **Anchore Syft** — project documentation and cataloger source (`github.com/anchore/syft`), for the per-ecosystem cataloger architecture and supported package types.
- **Aqua Trivy** — documentation on SBOM generation (`trivy image --format cyclonedx`) and scanning stored SBOMs (`trivy sbom`), Aqua Security.
- **OWASP cdxgen** and the **CycloneDX tool center** — multi-language generation and the CycloneDX Maven/Gradle plugins, `cyclonedx-npm`, `cyclonedx-py`, and `cyclonedx-gomod`.
- **Go module build information** — the `go version -m` command and the `debug/buildinfo` package, for how Go binaries carry an authoritative module list.
- **Kubernetes `bom`** — `github.com/kubernetes-sigs/bom`, the SPDX generator used for Kubernetes release SBOMs, as an example of SBOM generation as release infrastructure.
- **GitHub Dependency Submission API** — GitHub docs, for pushing build-time-resolved dependency graphs into the dependency graph and Dependabot.

- **NTIA**, "The Minimum Elements for a Software Bill of Materials (SBOM)," 12 July 2021 — the fields a conforming SBOM must carry, and the concept of *known unknowns*.
- **CISA**, "Types of Software Bill of Materials (SBOM)," 2023 — the Design/Source/Build/ Analyzed/Deployed/Runtime taxonomy this chapter maps generation points onto.
- **SLSA v1.0** provenance specification and **Sigstore** `cosign` documentation (`cosign attest`), for signing SBOMs and recording their provenance (Book 4, Chapter 3; Book 5).
- **Package URL (purl)** specification and **NIST CPE 2.3**, for why identifier quality at generation time governs downstream vulnerability matching (Book 2, Chapter 5).

Chapter 5 — SBOM Distribution, Storage, and Querying at Scale

---

What this chapter covers. Chapter 4 ended where most SBOM projects end: a well-formed, accurate document, freshly generated in CI, sitting on a build agent about to be garbage- collected. That document is a dead artifact. It becomes *infrastructure* only when it is reliably bound to the thing it describes, pushed somewhere durable, folded into a fleet-wide inventory, and made **queryable** — so that when the next Log4Shell lands at 02:00 you can answer "where is it?" in seconds instead of spending three weeks writing `grep` scripts against a build cache. This is the systems-engineering half of SBOMs, and it is the half most organizations get catastrophically wrong. It is also the half a senior distributed-backend engineer is uniquely equipped to build, because it is not really an SBOM problem at all: it is an ingestion pipeline, a normalized datastore, a correlation job, and a query API, run at fleet scale, joined against a source of truth for "what is actually running where." That last join is the whole game.

Learning goals — after this chapter you should be able to:

- State the **association problem** precisely and explain why binding an SBOM to an artifact by cryptographic **digest** is the only durable answer, not by name or tag.

- Compare the four **distribution models** — embedded, registry-attached (OCI referrers / attestations), a dedicated SBOM service, and customer delivery — and say when each applies, with the referrers model as the modern default.
- Design the **storage layer**: why you normalize both SPDX and CycloneDX into one canonical internal model at ingestion, keyed on **purl**; why deduplication is not optional at fleet scale; and where relational, document, and graph stores each fit.
- Describe **Dependency-Track**, **GUAC**, and **Grafeas** accurately — what each ingests, what graph it builds, and what question it is good at answering — without overstating their maturity.
- Realize the **Log4Shell query** and **blast-radius analysis** against a normalized inventory, and explain the architectural win of **continuous re-matching**: scan the stored SBOMs against new vulnerability data instead of re-scanning artifacts.
- Draw the **reference SBOM platform** end to end and reason about its consistency model — the eventual gap between *built*, *stored*, and *deployed* inventories, and the query latency/scale trade-offs between graph and relational for blast-radius questions.

A boundary note. This chapter assumes Chapter 4's generation points and the "graph, not list" framing from Chapter 1, the purl-versus-CPE identifier model from Book 2, Chapter 5, and the vulnerability databases (OSV, GHSA, NVD) from that same chapter. It hands off the formal treatment of **VEX** to Chapter 6, an honest accounting of SBOM **quality limits** to Chapter 7, the cryptographic machinery of **signing and attestation** to Book 5 (especially Chapter 3 — Sigstore Architecture, and Chapter 6 — in-toto), OCI image and registry internals to Book 6 (Chapters 1 and 2), and **incident response** workflow to Book 8, Chapter 6. Here the subject is the data platform: getting SBOMs to travel, storing them once, and querying them fast.

The association problem: an SBOM is nothing without a binding

Start with the failure mode, because it is endemic. A team stands up SBOM generation, wires it into CI, and starts dropping `sbom.json` files into an S3 bucket, one per build, keyed by a path like `s3://sboms/`

`payments-api/1.4.2/sbom.json`. Six months later Log4Shell lands. Someone opens the bucket and confronts the questions that were never designed for:

- Is `payments-api:1.4.2` the tag currently running in production, or was it rebuilt three times under the same tag? Tags are mutable; the SBOM is not.
- The SBOM was generated from the *source tree* at build time. Does it describe the actual container image that got pushed, including its base-image layers, or only the application dependencies? (Chapter 4: different generation points, different blindness.)
- There are four images that came out of that pipeline — `api`, `worker`, `migrator`, and a debug sidecar. Which one does this SBOM describe?

Every one of these is a variant of a single question: **which exact artifact does this document describe?** A name is not an answer. A tag is not an answer. A tag is a mutable pointer that a `docker push --force` or a re-run of a "release" job silently repoints. The only answer that survives rebuilds, re-tags, mirror copies, and registry migrations is the one thing about an artifact that cannot change without the artifact itself changing: its **content digest**.

An OCI image is addressed by the SHA-256 digest of its manifest — `sha256:9b2a...`. Change one byte of any layer, or the manifest, and the digest changes. A binary or a tarball is addressed by the digest of its bytes. This is the same content-addressing you already rely on in Git (a commit is its SHA), in content-addressable caches, and in Merkle-tree replication. The correct primitive is therefore:

> An SBOM MUST carry the digest of the artifact it describes, and consumers MUST look up SBOMs by digest, never by tag or name.

Both formats have a slot for this. In CycloneDX the top-level `metadata.component` (or a component in the graph) carries a `hashes` array; the whole document can also be wrapped as a subject in an attestation. In SPDX 2.3 a package carries a `checksums` array and a `SPDXID`; SPDX 3.0 makes the subject relationship explicit. But a hash *inside* the document is a claim, not a binding — nothing stops the document from lying or drifting. The binding becomes trustworthy only when the digest is the **address you fetched the SBOM by**, which is exactly what the registry-attached model gives you, and doubly so when

the SBOM is wrapped in a signed attestation whose *subject* is the artifact digest (Book 5, Chapter 6). Content address plus signature over that address is the association problem solved.

Two subtleties a distributed-systems engineer will immediately flag:

- **Multi-arch images.** A `linux/amd64` and a `linux/arm64` build of the "same" version are different images with different digests and, potentially, different component sets (a different libc, different native wheels). They are referenced by an OCI *image index* (manifest list) under one tag. You need an SBOM *per platform digest*, associated with the child manifest, not the index. Treating "the image" as one thing is the multi-arch bug that bites every SBOM platform eventually.
- **The build-versus-image gap.** As Chapter 4 argued, a source/build SBOM and a scan-the-image SBOM describe overlapping but non-identical component sets. If you attach both, attach each to the digest it actually describes and label its type, or you will merge a build-time dependency list onto an image and "lose" the base OS packages.

Distribution: how SBOMs travel with — or near — artifacts

There are four ways an SBOM can reach a consumer. They are not mutually exclusive; a mature program uses three of them for different audiences.

Model	Where the SBOM lives	Binding to artifact	Best for	Weaknesses
Embedded	Inside the artifact (a file in the image, `/usr/share/sbom.json`, or a binary section)	Physically co-located; ships with the bits	Firmware, appliances, air-gapped delivery	Changes the artifact (and thus its digest); can't be updated without a rebuild; bloats the image; you can't enumerate SBOMs without pulling every artifact
Registry-attached (referrers / attestation)	The OCI registry, as a separate manifest that *references* the image by digest	Cryptographic: the referrer's `subject` is the image digest	Containers, the default modern path	Requires OCI 1.1 referrers support (or the tag-schema fallback); registry is now on the critical path for security queries
Dedicated SBOM service / repository	A central store you run (DB + object storage + API)	Logical: keyed by digest in your schema	Fleet-wide inventory, cross-artifact queries, non-OCI artifacts	You must build and operate it; ingestion must be reliable or the inventory silently rots

Model	Where the SBOM lives	Binding to artifact	Best for	Weaknesses
Customer delivery	Sent to a downstream consumer (portal, VEX feed, compliance package)	Contractual + digest in the doc	Regulated procurement, EU CRA, US federal buyers	Point-in-time; consumer trust and freshness are on them; format/ version negotiation

The four are layered, not competing. In practice: **attach** to the registry so the SBOM travels with the artifact and can be verified at admission; **ingest** from the registry into a central service so you can query across the whole fleet; and **deliver** a curated export to customers who need it. Embedding is a niche you reach for only when the artifact must be self-describing in an environment with no registry and no network.

Registry-attached: OCI referrers and attestations

This is the model worth understanding in mechanism, because it is where the industry has converged and because it exploits infrastructure you already run: the container registry.

Before OCI 1.1, there was no standard way to say "this blob is *about* that image." People faked it with tag conventions — cosign's original scheme stored a signature for `image@sha256:abcd...` at a synthetic tag `sha256-abcd....sig` in the same repository. It worked but it was a side-channel: the registry had no idea the two objects were related, you couldn't list "everything attached to this image," and garbage collection could delete the image while orphaning its signatures (or vice versa).

OCI Image Specification 1.1 (released 2024) standardized the relationship with two mechanisms:

1. A `subject` field on a manifest. Any manifest (an image, or an "artifact manifest") can name another manifest, by digest, as its subject. That is the machine-readable "this is *about* that."
2. The **Referrers API** — `GET /v2/<name>/referrers/<digest>` — which returns the list of all manifests whose `subject` is that digest, optionally filtered by `artifactType`. For registries that haven't implemented the endpoint, the spec defines a **fallback tag schema**

(`sha256-<digest>` holds an index of referrers) so clients get the same answer without native support.

An SBOM, then, is not stored "inside" anything. It is pushed to the registry as its own blob with its own manifest, whose `subject` is the image digest and whose `artifactType` identifies it (e.g. `application/vnd.cyclonedx+json` or an in-toto predicate type). The signature over that SBOM is *itself* another referrer pointing at the SBOM (or at the image). The result is a small graph hanging off the immutable image digest:

```
flowchart TD
    IMG["Image manifest
sha256:9b2a… (the artifact)"]
    SBOM["SBOM attestation manifest
artifactType: in-toto SPDX/CycloneDX
subject → sha256:9b2a…"]
    PROV["Provenance attestation
(SLSA, Book 5 Ch 6)
subject → sha256:9b2a…"]
    SIG1["Signature
subject → SBOM manifest"]
    SIG2["Signature
subject → provenance"]

    SBOM -->|"subject (by digest)"| IMG
    PROV -->|"subject (by digest)"| IMG
    SIG1 -->|"subject"| SBOM
    SIG2 -->|"subject"| PROV

    QUERY["Referrers API:
GET /v2/app/referrers/sha256:9b2a…"] -.->|"returns SBOM, PROV,
signatures"| IMG
```

The tooling is mature enough to use in anger, with caveats:

```
# Generate an SBOM for the exact image we built, addressed by digest.
DIGEST=$(cosign triangulate --type digest registry.example.com/
payments/api:1.4.2)
syft "registry.example.com/payments/api@${DIGEST}" -o cyclonedx-json >
sbom.cdx.json

# Attach the SBOM as a SIGNED in-toto attestation whose subject is the
image digest.
# 'attest' wraps the predicate in a DSSE envelope and signs it (keyless
via Fulcio here).
cosign attest --yes \
  --predicate sbom.cdx.json \
  --type cyclonedx \
  "registry.example.com/payments/api@${DIGEST}"

# Later — discover and verify everything attached to that digest.
cosign verify-attestation --type cyclonedx \
  --certificate-identity-regexp '^https://github.com/payments/.+' \
  --certificate-oidc-issuer https://
token.actions.githubusercontent.com \
  "registry.example.com/payments/api@${DIGEST}" | jq '.payload |
@base64d | fromjson'

# List raw referrers (OCI 1.1) with oras, independent of cosign's
conventions.
oras discover --format tree "registry.example.com/payments/api@$
{DIGEST}"
```

A few honest notes on maturity. `cosign attach sbom` (the older, simpler command that stores an SBOM as a plain OCI artifact without wrapping it in a signed attestation) is **deprecated** in favor of `cosign attest`, precisely because an unsigned attachment solves distribution but not trust — an attacker who can push to the registry can replace it. Prefer `attest`. Referrers support is now broad (GHCR, ECR, GAR, Harbor, Zot, recent Docker Registry) but not universal; clients like cosign and oras fall back to the tag schema transparently, so you rarely see the gap, but self-hosted or older registries may only offer the fallback, which is chattier and loses server-side filtering. And the registry becomes security-critical infrastructure: if admission control verifies SBOM/provenance on every deploy, registry availability is now in the deploy path (Book 6, Chapters 2 and 5).

The conceptual payoff is the **"SBOM everywhere"** pattern: every image your fleet produces carries, hanging off its digest, a signed SBOM attestation and a signed provenance attestation, discoverable by anyone

with pull access via one standard API call. That is the ideal distribution substrate — but it is only a substrate. You still cannot answer "which of my 4,000 images contain log4j-core?" by walking referrers image by image. For that you need to pull all of it into one place. Which is the storage problem.

Storage: the SBOM data platform

The scale reality

Do the arithmetic that most SBOM pilots never do. Take a mid-to-large backend estate: 800 services, each built on average a few times a day, each build producing 1–4 images, retained for some window. That is on the order of **millions of SBOM documents per year**. Each document lists hundreds to low-thousands of components. You are looking at **hundreds of millions to billions of component rows** if you store naively. And the access pattern is not "fetch one SBOM" — it is "find every artifact, across all of history-that-still-matters, containing a component matching predicate P," run reactively under incident pressure and proactively on every new CVE.

This is a data-engineering problem with a specific shape: write-heavy ingestion of semi-structured documents, a normalized inventory optimized for reverse lookup (component → artifacts), heavy fan-out joins for blast radius, and a correlation workload that re-runs as external feeds change. None of that is exotic to a backend engineer. What makes it SBOM-specific is two format wars and one brutal redundancy.

Normalize both formats into one canonical model at ingestion

You will receive SPDX (2.3 and 3.0) and CycloneDX (1.4–1.6), because Chapters 2 and 3 exist and different teams and vendors picked differently. **Do not** store raw documents and query across formats at read time — that pushes the format war into every query and every dashboard. Instead, **normalize at ingestion into one internal model**, and keep the raw signed document as an immutable blob for provenance and re-processing.

The canonical model is small. The linchpin is that both formats can express a component's identity as a **Package URL (purl)** — `pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1` — and purl is your join key

across everything: across formats, across SBOMs, and against vulnerability feeds keyed the same way (Book 2, Chapter 5). CPE is a fallback for OS packages and older data; store it too, but treat purl as primary. A workable relational schema:

```
-- One row per unique component identity, stored ONCE for the whole
estate.
CREATE TABLE component (
    id           BIGINT PRIMARY KEY,
    purl         TEXT UNIQUE,          -- canonical identity; the join
key
    cpe          TEXT,                 -- fallback identity for
matching
    name         TEXT NOT NULL,
    version      TEXT,
    ecosystem    TEXT                  -- maven, npm, pypi, deb, apk,
golang, …
);

-- One row per artifact we have an SBOM for, addressed by DIGEST.
CREATE TABLE artifact (
    id           BIGINT PRIMARY KEY,
    digest       TEXT UNIQUE NOT NULL, -- sha256:… — the binding from
§1
    kind         TEXT,                 -- oci-image, jar, binary, …
    repo         TEXT,                 -- registry.example.com/
payments/api
    platform     TEXT,                 -- linux/amd64 (multi-arch!)
    sbom_format  TEXT,                 -- source of truth for the raw
blob
    raw_blob_ref TEXT,                 -- object-store key of the
signed original
    generated_at TIMESTAMPTZ
);

-- The many-to-many edge: which components are in which artifact.
-- This is the table you reverse-scan for blast radius.
CREATE TABLE artifact_component (
    artifact_id  BIGINT REFERENCES artifact(id),
    component_id BIGINT REFERENCES component(id),
    scope        TEXT,                 -- runtime, dev, optional,
provided
    relationship TEXT,                 -- DEPENDS_ON, CONTAINS, base-
layer…
    PRIMARY KEY (artifact_id, component_id)
);
CREATE INDEX ON artifact_component (component_id);  -- the reverse-
lookup index
```

Two design decisions in that schema carry the whole chapter:

- **Deduplication is structural, not an optimization.** `log4j-core@2.14.1` is one row in `component`, referenced by however many thousand artifacts contain it. A popular base-image layer's packages, a ubiquitous TLS library, the standard logging stack — each is stored once and referenced many. Without dedup, that billion-row estimate is real and your inventory is mostly copies of the same fifty thousand popular components. With it, `component` is small, bounded roughly by "distinct (package, version) pairs the org has ever shipped," and the expensive table is the edge table — which is exactly the table you want indexed for reverse lookup. This is the same normalization you would reach for in any inventory system; SBOMs just make the redundancy extreme because every image restates its base layers.
- **The reverse index on `component_id` is the Log4Shell index.** The forward direction (artifact → its components) is how you *store*; the reverse (component → its artifacts) is how you *respond to incidents*. Index the reverse direction or every blast-radius query is a full scan.

Document vs relational vs graph

The component-relationship structure is a graph — a DAG of "depends on / contains" edges, exactly Chapter 1's insistence that an SBOM is a graph, not a list. That tempts people straight to a graph database. Resist reflexively; reason about the workload.

Store	Fits when	Cost / caveat
Relational (Postgres/MySQL) normalized as above	The default. Reverse lookups and version-range predicates are indexed queries; joins to vuln feeds are ordinary joins; you get transactions, mature ops, and SQL everyone reads	Deep transitive traversal ("all things reachable from X through N hops") is recursive-CTE territory and gets awkward past a few levels
Document store (Mongo, OpenSearch, or JSONB columns)	Storing and retrieving whole raw SBOMs; full-text over component names; flexible schema across format versions	Cross-document reverse queries and dedup are painful; you re-implement the normalized index anyway

Store	Fits when	Cost / caveat
Graph DB (Neo4j, JanusGraph, Dgraph)	Blast-radius and provenance queries that are *natively* multi-hop: component → artifacts → services → environments → owners, plus SLSA/VEX edges	Operational maturity and cost; you still ingest from documents; the win appears only when queries are genuinely deep and the graph is dense
Purpose-built (Dependency-Track, GUAC)	You want the inventory + correlation semantics without building them	Opinionated model; you adopt their scale envelope and their gaps

The pragmatic majority answer, and the one to default to, is **relational + a normalized component table + purl as the join key**, with raw documents in object storage and, optionally, a graph layer *on top* when your blast-radius questions genuinely go many hops (component to service to environment to team, joined with provenance and VEX). The graph is a query accelerator for a specific class of question, not the system of record. We return to that trade-off in the distributed-systems lens.

Purpose-built tools, described accurately

Three open-source systems occupy this space. Know what each actually does, and where it stops.

Dependency-Track (OWASP). The most operationally mature of the three for the specific job of a **portfolio-wide component inventory with vulnerability correlation**. You model your estate as *projects* (and project versions); you upload a **CycloneDX** SBOM per project version (SPDX is not its native ingestion path — this matters). Dependency-Track maintains the deduplicated component inventory across the whole portfolio and continuously correlates it against vulnerability sources — the NVD mirror, OSS Index, GitHub Advisories, Snyk, VulnDB — so that when a new advisory lands it flags every affected project **without re-scanning any artifact**. It also ingests **VEX** (CycloneDX VEX) to suppress non-exploitable findings (Chapter 6), exposes metrics and a REST API, and supports policy conditions. Its limits: it is CycloneDX-centric; it is a component/vuln inventory, not a general artifact-provenance graph (it doesn't model SLSA attestation chains); and very large portfolios need

attention to its database and analyzer tuning. For "give me a living inventory and tell me what's vulnerable," it is the fastest path to value.

GUAC — Graph for Understanding Artifact Composition (OpenSSF, originated at Google, with Kusari and others). A different and more ambitious target: ingest **many document types** — SBOMs (SPDX *and* CycloneDX), **SLSA provenance**, **VEX**, scorecard and vulnerability data — and assemble them into **one queryable graph** of the software supply chain. Its purpose is the relationship questions: "what uses component X?", "what is the **blast radius** if this dependency is compromised?", "what is the provenance of this artifact and does it satisfy policy?", "what other artifacts share this suspect dependency?" Architecturally GUAC separates ingestion (collectors normalize documents into a canonical trie/graph model) from a GraphQL query layer over a backing store. It is the closest thing to the fleet-wide impact graph this chapter builds toward. Honest maturity note: GUAC is younger than Dependency-Track, its model and APIs have evolved, and running it well at very large scale is a real engineering commitment — it is powerful and the right mental model, not a turnkey appliance.

Grafeas (Google/open source). Often miscategorized. Grafeas is **not** an SBOM analyzer; it is a **metadata API and store** — a normalized schema and API for attaching *notes* (a class of metadata, e.g. a known vulnerability, a build, an attestation) and *occurrences* (an instance of a note bound to a specific resource by URL/digest) to artifacts. It is the general substrate on which you could store SBOM-derived facts, vulnerability findings, and attestations against artifact digests; Google's Container/Artifact Analysis and Binary Authorization build on this model. Think of it as the neutral metadata plane (the digest-keyed fact store), not the query engine that answers "where is log4j." You bring the correlation logic; Grafeas gives you a consistent place to hang the facts.

The mapping to remember: **Dependency-Track** = inventory + vuln correlation, CycloneDX-first, production-ready today. **GUAC** = the relationship graph across SBOM + provenance + VEX, newer, the right model for blast radius. **Grafeas** = digest-keyed metadata store, a substrate, not an analyzer.

Querying: the entire point

Everything so far — binding by digest, attaching to registries, normalizing, deduplicating — exists to make a small number of questions answerable *fast*. If you cannot answer these, you have an expensive filing cabinet, not a security capability.

The Log4Shell query, realized

In December 2021, the industry's collective answer to "do we run a vulnerable log4j?" was measured in **weeks** of manual archaeology. Against a normalized inventory it is a query that returns in **seconds**, because it is exactly the reverse lookup the schema was built for. Conceptually:

```
-- "Which artifacts contain log4j-core in the affected version range?"
SELECT a.repo, a.platform, a.digest, c.version
FROM   component c
JOIN   artifact_component ac ON ac.component_id = c.id
JOIN   artifact a           ON a.id = ac.artifact_id
WHERE  c.ecosystem = 'maven'
  AND  c.name = 'log4j-core'
  AND  semver_in_range(c.version, '>=2.0-beta9, <2.16.0');   -- the
CVE-2021-44228/45046 range
```

The shape matters more than the SQL. It is `component (predicate) → artifacts`, powered by the reverse index. "Any version" is dropping the range clause; "version in range Y" is the range predicate. The predicate can be a purl, a name, a version range, an ecosystem, a checksum, or a transitive-reachability flag if you stored scope. And crucially, this query needs **no access to the artifacts themselves** — no pulling images, no re-scanning, no build cache. The inventory *is* the answer surface. That decoupling is the architectural win we make explicit below.

Blast radius: from component to the running fleet

The Log4Shell query finds *artifacts*. Incident response needs the next hops: which **services** those artifacts belong to, which **versions of those services are actually deployed**, and in which **environments** — because "an image in the registry contains it" is not the same as "it is running in production right now." This is the fleet-wide impact graph, and it is GUAC's core use case:

```
flowchart TD
    C["Component
log4j-core (vuln range)"]
    A1["artifact
payments/api@sha256:9b2a…"]
    A2["artifact
search/indexer@sha256:71cd…"]
    A3["artifact
legacy/report@sha256:0af3…"]
    S1["service: payments-api"]
    S2["service: search-indexer"]
    S3["service: report-gen"]
    E1["prod / us-east
12 pods LIVE"]
    E2["prod / eu-west
8 pods LIVE"]
    E3["staging only
not in prod"]
    E4["not deployed
(registry only)"]

    C -->|"reverse index"| A1
    C --> A2
    C --> A3
    A1 -->|"digest → service+version"| S1
    A2 --> S2
    A3 --> S3
    S1 -->|"deploy system:
what's running where"| E1
    S1 --> E2
    S2 --> E3
    S3 --> E4

    classDef live fill:#c0392b,color:#fff;
    classDef cold fill:#7f8c8d,color:#fff;
    class E1,E2 live;
    class E3,E4 cold;
```

The graph tells the incident commander what a list cannot: `payments-api` is **live in two prod regions** (page now, that is the real exposure), `search-indexer` is only in staging (fix in the normal cycle), and `report-gen`'s affected image was built but never deployed (inventory hygiene, not an incident). The reverse index answers the first hop; the **deploy/orchestration system** — your CD tool, service catalog, or the cluster's own record of running digests — answers the last two. That last join is what separates a real IR capability from a spreadsheet of "images that

exist." We return to it under freshness. This graph feeds directly into the IR playbook in Book 8, Chapter 6.

Continuous re-matching: scan the SBOM, not the artifact

Here is the decoupling stated as an architecture principle, because it is the single most important idea in the chapter. The traditional model scans **artifacts** for vulnerabilities: point a scanner at an image, it enumerates packages and matches them against a vuln DB, emits findings. That coupling is expensive and stale — every new CVE means re-pulling and re-scanning every artifact, so in practice you scan on a schedule and your findings lag reality by however long the cycle is.

Invert it. **Generate the SBOM once** (Chapter 4), at the point of maximum information, and store the component inventory. Then treat vulnerability detection as a **join between two datasets that change on different clocks**: your (mostly static) SBOM inventory, and the (constantly updated) vulnerability feeds — OSV, GHSA, NVD (Book 2, Chapter 5). When a new advisory arrives, you do not touch a single artifact; you **re-run the match against the stored inventory**. New CVE at 03:00 → a correlation job wakes, resolves the advisory's affected purls/ranges, hits the reverse index, and within minutes every affected artifact — and, via the deploy join, every live service — is flagged.

```
flowchart LR
    subgraph ONCE["Once, at build time"]
        GEN["Generate SBOM
(Ch 4, max information)"] --> STORE["Normalized inventory
components ⋈ artifacts"]
    end
    subgraph FEEDS["Continuously"]
        OSV["OSV / GHSA / NVD
new advisories"]
        VEX["VEX statements
(Ch 6)"]
    end
    subgraph LOOP["Re-match loop (no re-scan of artifacts)"]
        MATCH["Correlation job:
resolve advisory → purls/ranges
→ reverse index"]
        SUPP["Apply VEX:
suppress not-affected"]
        ALERT["Alert / ticket / dashboard
+ deploy join → live services"]
    end

    STORE --> MATCH
    OSV --> MATCH
    MATCH --> SUPP
    VEX --> SUPP
    SUPP --> ALERT
    ALERT -.->|"new build → new SBOM"| GEN
```

This is precisely what Dependency-Track does internally and what `trivy sbom` / `osv-scanner --sbom` do for a single document — evaluate a *stored SBOM* against current feeds rather than re-scanning bits. At fleet scale you run it as a batch/stream job over the whole inventory. The wins compound: findings are as fresh as your feed ingestion (minutes, not the scan cycle); the cost of a new CVE is one query, not N re-scans; and — critically for old, rarely-rebuilt services — you can detect a newly-disclosed vuln in an artifact **nobody has touched in a year**, because you never depended on re-running a scanner against it. The artifact's SBOM is a durable record; the vulnerability knowledge catches up to it.

The unified picture: four datasets, one join

The complete operational view is the join of four continuously-changing datasets, keyed to make them joinable:

- **SBOM inventory** — components ⋈ artifacts, keyed by **purl** and **digest** (this chapter).
- **Vulnerability feeds** — OSV/GHSA/NVD, keyed by **purl/CPE + version range** (Book 2, Chapter 5).
- **VEX** — per-(product, vuln) exploitability statements that suppress or confirm findings, keyed by **product digest + vuln ID** (Chapter 6). Without VEX, a fleet-scale re-match drowns you in not-actually-exploitable findings; VEX is what makes the continuous loop *actionable* rather than noise.
- **Runtime / deployment truth** — "what digest is running where," keyed by **digest → service, version, environment**, sourced from the deploy system, service catalog, or cluster state.

Purl bridges SBOM and vuln feeds; digest bridges SBOM and runtime; the (product, vuln) pair bridges VEX into both. Get those keys consistent at ingestion and every question above is a join. Get them inconsistent — CPE-only vuln data against purl-only SBOMs, tag-keyed deploys against digest-keyed inventory — and you have four datasets that cannot be joined, which is the actual state of most programs.

Architecture: the reference SBOM platform

Assemble the pieces into a platform. The pipeline is: **generate → sign → attach → ingest → normalize & dedup → correlate → serve**, with the deploy system feeding runtime truth and a set of consumers on the query side.

```
flowchart TB
    subgraph CI["CI / build (Ch 4)"]
        BUILD["Build artifact"] --> GEN["Generate SBOM
(Syft/Trivy/cdxgen)"]
        GEN --> SIGN["Sign as attestation
cosign attest (Book 5)"]
    end
    subgraph REG["OCI registry (Book 6)"]
        ATTACH["Attach: SBOM + provenance
as referrers of image digest"]
    end
    subgraph PLAT["SBOM platform (tier-1 internal infra)"]
        INGEST["Ingest: watch registry / receive push
verify signature, pin digest"]
        NORM["Normalize SPDX+CycloneDX → canonical model
dedup components, key by purl"]
        STORE[("Normalized store
relational + object store
(+ optional graph)")]
        CORR["Correlation jobs
⋈ OSV/GHSA/NVD, apply VEX
continuous re-match"]
        API["Query / alert / report API"]
    end
    subgraph TRUTH["Runtime truth"]
        DEPLOY["Deploy / orchestration / catalog
digest → service, version, env"]
    end
    subgraph CONS["Consumers"]
        SEC["Security / vuln mgmt"]
        IR["Incident response (Book 8 Ch 6)"]
        COMP["Compliance / customer SBOM delivery"]
        DASH["Engineering dashboards"]
        PROC["Procurement / vendor risk"]
    end

    SIGN --> ATTACH
    ATTACH --> INGEST
    INGEST --> NORM --> STORE
    STORE --> CORR --> API
    DEPLOY --> CORR
    DEPLOY --> API
    OSV2["OSV / GHSA / NVD"] --> CORR
    VEX2["VEX (Ch 6)"] --> CORR
    API --> SEC
    API --> IR
    API --> COMP
    API --> DASH
    API --> PROC
```

Read it as a data platform, because that is what it is. Ingestion is a pipeline with a verification step (never ingest an SBOM whose signature you didn't check — an unsigned SBOM is an unauthenticated claim about what's in your fleet, and an attacker who can write to your inventory can hide their implant from every query). Normalization is a schema-mapping ETL step. The store is a normalized database with an object-store side-car for raw signed blobs. Correlation is a scheduled/streaming job joining external feeds. The API is the product surface. And the deploy system is a **first-class input**, not an afterthought — it is what turns "artifacts that exist" into "software that is running."

Freshness and lifecycle: inventory the running fleet, not the build history

The most common way a mature-looking SBOM platform still fails IR is a **freshness** failure: it faithfully inventories every artifact ever built and has no idea which are actually deployed. During an incident that is nearly useless — you get 4,000 hits and cannot tell the 30 that are live from the 3,970 that are cold. The fix is the digest→deployment join, sourced from whatever your orchestration layer treats as truth:

- In Kubernetes, the running **image digest** per pod (the resolved digest, not the tag) from the API server or an admission/inventory controller.
- In a service catalog / CD system, the currently-released version → digest mapping per environment.
- Reconciled continuously, because deploys happen continuously and the "live set" is always drifting.

That join reframes retention, too. You do not need to keep every SBOM forever; you need, at minimum, an SBOM for **every digest currently live in any environment**, plus enough history to investigate "what were we running on the date of the breach" (forensics for Book 8, Chapter 6) and to satisfy whatever regulatory retention applies (Chapter 1). A reasonable policy: hot storage for all live and recently-live digests; cold/object storage for the long tail of historical SBOMs and their signatures; and garbage-collection tied to the artifact's own lifecycle in the registry so you never orphan an SBOM whose image is gone or, worse, delete an SBOM whose image is still running.

APIs and consumers

The same normalized store, exposed through one API, serves audiences with genuinely different questions — which is the argument for building it as shared infrastructure rather than letting each team reinvent a corner of it:

- **Security / vulnerability management** — the continuous re-match results, prioritized by reachability (Book 2, Chapter 7) and deployment status; "what is exploitable and live."
- **Incident response** — the blast-radius query on demand, under time pressure, joined to the live fleet (Book 8, Chapter 6).
- **Compliance / customer delivery** — export a signed SBOM (and VEX) for a specific released digest in the customer's required format and version; regulated procurement and the EU CRA (Chapter 1) live here.
- **Engineering dashboards** — per-service component inventories, "how far behind is my dependency," end- of-life component flags; the feedback loop that gets teams to update (Book 2, Chapter 9).
- **Procurement / vendor risk** — ingest *third-party* SBOMs for software you buy and run the same queries against them (Book 8, Chapter 3), so a vendor's log4j is as visible as your own.

Distributed-systems lens

Everything in this chapter is a distributed-systems build, and treating it as anything less is why so many SBOM programs stall at "we generate SBOMs" and never reach "we can answer questions."

It is tier-1 internal infrastructure. The SBOM platform is on the critical path for incident response and, if you gate admission on attestations (Book 6, Chapter 5), on the deploy path too. It earns the same operational rigor as your service registry or metrics backend: SLOs on ingestion lag and query latency, on-call, capacity planning against that millions-of-documents growth curve. An SBOM inventory that is silently three weeks behind on ingestion is worse than none, because it will answer an incident query with false confidence. Treat ingestion completeness and freshness as monitored SLIs, not hopes.

Three inventories, eventually consistent. There are three distinct populations — what was **built** (everything CI ever produced), what is **stored** (everything successfully ingested and normalized), and what is **deployed** (what the fleet is actually running) — and they are **never simultaneously consistent**. A build finishes before its SBOM is ingested; a deploy can promote a digest before your inventory has caught up (or, in a badly-ordered pipeline, before the SBOM exists at all); a rollback can make "live" jump backward. Design for the gaps explicitly: reconcile continuously rather than assuming synchrony, alert on **skew** ("live digests with no SBOM in the store" is a coverage gap you must see), and make queries state their own staleness ("as of ingestion watermark T"). The failure you are preventing is answering "we're clean" from a stored inventory that simply hadn't ingested the vulnerable build yet. Order the pipeline so an SBOM is attached and ingested *before* a digest is eligible for production, and the worst gaps close.

Graph vs relational is a latency/scale trade-off, not a religion. The reverse-lookup query (component → artifacts) is a single indexed step and relational wins on ops maturity and cost. The deep blast-radius query (component → artifacts → services → environments → owners, further joined with provenance and VEX) is genuinely multi-hop, and at some depth and graph density a native graph store answers in one traversal what relational answers with escalating recursive-CTE pain. The engineering call is empirical: measure your actual query depths and fan-outs. Most estates are well served by relational as system-of-record with the deploy join precomputed, adding a graph layer (or GUAC) only when blast-radius questions provably go deep enough to hurt. Don't pay graph-database operational cost for a two-hop query you can index.

Idempotent, digest-keyed ingestion. The same image gets pushed to multiple registries, mirrored, re-tagged, and re-scanned; the same SBOM will arrive more than once. Key ingestion on the artifact **digest** and make it idempotent (upsert by digest, dedup components by purl) so replays and mirrors converge instead of inflating the inventory — the same content-addressing discipline that makes the rest of your distributed infrastructure sane, applied here. Digest is the idempotency key; purl is the dedup key; get both right and the platform self-heals under the messy reality of a real registry topology.

Key takeaways

- **An SBOM is worthless until it is bound to an exact artifact by digest.** Names and tags are mutable pointers; the content digest is the only durable binding. Look up SBOMs by digest, never by tag, and make the digest the *address you fetched by*, ideally inside a signed attestation whose subject is that digest.
- **Registry-attached distribution via OCI 1.1 referrers is the modern default.** The SBOM is a separate manifest whose `subject` is the image digest, discoverable through the Referrers API; `cosign attest` (signed) supersedes the deprecated `cosign attach` (unsigned). This is the "SBOM everywhere" substrate — but a substrate for fleet queries, not a query engine itself.
- **Storage is a real data-engineering problem.** Millions of documents, billions of component references. Normalize both SPDX and CycloneDX into one canonical model at ingestion, key on **purl**, **deduplicate** components so each identity is stored once, and index the **reverse** direction (component → artifacts). Relational + normalized component table is the pragmatic default; add graph only where blast-radius queries provably go deep.
- **Know the tools precisely.** Dependency-Track = CycloneDX-first portfolio inventory with continuous vuln correlation (production-ready). GUAC = the SBOM+provenance+VEX relationship graph for blast radius (newer, the right model, a real ops commitment). Grafeas = a digest-keyed metadata store/substrate, not an analyzer. Don't overstate any of their maturity.
- **Querying is the entire point.** The Log4Shell query is a reverse lookup that returns in seconds; the blast-radius query extends it through services to *live* environments via the deploy system. If you can't do these, you have a filing cabinet.
- **Scan the SBOM, not the artifact.** Generate once; re-match the stored inventory against ever-changing OSV/GHSA/NVD feeds. New CVEs cost one query, not N re-scans, and you can detect vulnerabilities in artifacts nobody has rebuilt in a year. This decoupling is the architectural win.
- **The deploy/orchestration system is a first-class input.** Joining SBOM inventory (by digest) to "what is running where" is what turns

"images that exist" into "software that is live," and it is the difference between an IR capability and a spreadsheet.

- **Treat it as tier-1 distributed infrastructure.** Eventual consistency between built/stored/deployed inventories, digest-idempotent ingestion, purl-keyed dedup, monitored freshness SLOs, and an explicit latency/scale choice between relational and graph. This is a distributed-systems build; engineer it like one.

Further reading

- **OCI Image Specification 1.1** and the **OCI Distribution Specification** — the `subject` field, `artifactType`, the **Referrers API** (`/v2/<name>/referrers/<digest>`) and the fallback tag schema (`opencontainers.org` specs on GitHub).
- **Sigstore cosign** documentation — `cosign attest`, `cosign verify-attestation`, and the deprecation of `cosign attach sbom`; the DSSE envelope and in-toto predicate types (Book 5, Chapter 3).
- **in-toto Attestation Framework** and the **SPDX / CycloneDX** predicate types — how an SBOM is carried as a signed attestation whose subject is an artifact digest (Book 5, Chapter 6).
- **ORAS** (`oras.land`) — `oras attach` / `oras discover`, for working with OCI referrers independent of cosign's conventions.
- **OWASP Dependency-Track** documentation — the project/portfolio model, CycloneDX ingestion, continuous analysis against NVD/OSS Index/GitHub Advisories, and VEX support.
- **GUAC** (`guac.sh`, OpenSSF) — the ingestion-collector/GraphQL architecture and the "what uses X" / blast-radius query model over SBOM, SLSA provenance, and VEX.
- **Grafeas** (`grafeas.io`) — the notes/occurrences metadata API and its use as a digest-keyed fact store underneath Google Container/Artifact Analysis and Binary Authorization.
- **OSV** (`osv.dev`) and the **OSV schema** — purl-keyed, range-based vulnerability data designed for matching against SBOM inventories, and `osv-scanner --sbom` (Book 2, Chapter 5).
- **Aqua Trivy** — `trivy sbom`, scanning a *stored* SBOM against current vulnerability data as the single-document form of continuous re-matching.

- **Package URL (purl) specification** (`github.com/package-url/purl-spec`) — the join key that makes cross-format normalization and vuln correlation possible.
- **CISA**, "Software Bill of Materials (SBOM) Sharing" guidance and the SBOM-a-rama materials — on distribution models and customer delivery.

Chapter 6 — VEX and Vulnerability Correlation

What this chapter covers. By now you can produce an accurate SBOM (Chapters 2–4) and store and query it at fleet scale (Chapter 5). Point a scanner at that inventory and you get the thing everyone actually wanted: a list of known vulnerabilities affecting your software. You also get the thing nobody wanted — a *flood* of them, the overwhelming majority of which are not exploitable in your product. This chapter is about the mechanism that closes that gap: **VEX**, the Vulnerability Exploitability eXchange. VEX is a machine-readable assertion, made by someone who actually knows, about whether a specific product is affected by a specific vulnerability, and *why*. It is the piece that turns "SBOM plus scanner" from a noise generator into a workable vulnerability-management program. We cover the status semantics precisely (they are easy to get subtly wrong), the three formats you will meet in production (CSAF, CycloneDX VEX, OpenVEX), how a VEX statement flows from producer to consumer as a signed attestation, and how to operationalize the whole thing so that one triage decision suppresses noise across an entire estate instead of being re-made five hundred times.

Learning goals — after this chapter you should be able to:

- Explain **the problem VEX solves**: why an SBOM plus a vulnerability feed produces a finding list that is mostly non-actionable, and why recording "not affected, because…" in a machine-readable, attributable form is the only thing that scales.
- State the **four VEX statuses** and the **five `NOT_AFFECTED` justifications** exactly as CISA defines them, and choose the correct one for a given situation — including the reachability justification that ties directly to Book 2, Chapter 7.

- Read and write VEX in the **three real formats** — CSAF 2.0's VEX profile, CycloneDX's `vulnerabilities` / `analysis` block, and OpenVEX — and know which to reach for and why.
- Wire up the **correlation pipeline**: SBOM inventory × vulnerability feed × VEX → a de-noised, prioritized finding list, with VEX statements signed and distributed as attestations (Book 5).
- Operationalize VEX at fleet scale: **auto-generate** reachability VEX from a shared analysis platform, record human triage **once**, consume upstream vendors' VEX, and avoid the failure modes — stale VEX, scope errors, and over-suppression.

This chapter is the operational payoff for two threads. From Book 2 it inherits the vulnerability-database and identifier plumbing (Chapter 5 — Vulnerability Databases and Identifiers), Software Composition Analysis (Chapter 6), and the reachability and exploitability material (Chapter 7 — Reachability, Exploitability, and Prioritization), which is the intellectual core of what a good `NOT_AFFECTED` claim asserts. From Book 3 it inherits the CycloneDX vulnerability model sketched in Chapter 3 and the SBOM platform of Chapter 5, which is where your VEX ends up living.

The problem: an SBOM plus a scanner is a noise machine

Run the standard pipeline. Generate a CycloneDX or SPDX SBOM for a container image (Chapter 4). Feed it to a matcher — `osv-scanner`, `grype`, Trivy, Dependency-Track — which joins each component's package URL against a vulnerability feed (OSV, GHSA, NVD; Book 2, Chapter 5). Out comes a list:

```
$ grype sbom:app.cdx.json
NAME            INSTALLED   VULNERABILITY   SEVERITY
libxml2         2.9.13      CVE-2022-40303  High
openssl         3.0.7       CVE-2023-0464   High
zlib            1.2.13      CVE-2023-45853  Critical
glibc           2.36        CVE-2023-4911   High
... 380 more findings
```

For a typical microservice built on a general-purpose base image, that list runs to hundreds of entries. Here is the uncomfortable truth, which Book 2, Chapter 7 established in detail: the large majority of those findings are *not exploitable in your product*. The CVE is real, the component version is

really present, the match is correct — and yet the vulnerability cannot be triggered as your software is built and deployed. The reasons recur:

- **The vulnerable code is not reachable.** `libxml2` is present because something in the base image links it, but your service never parses untrusted XML and never calls the affected function. The finding is a component match, not a reachable path. This is the reachability problem from Book 2, Chapter 7, wearing a different hat.
- **The vulnerable configuration is not the one you run.** The CVE requires a feature flag, a legacy cipher, a specific compile option, or a network mode you do not enable.
- **The vulnerable code is not even present.** The version *string* matches, but the distro backported a fix without bumping the version (Book 2, Chapter 5's version-range false-positive problem), or the affected file was compiled out.
- **A mitigation already neutralizes it.** A WAF rule, a seccomp profile, a compiler hardening flag, or an architectural control means the adversary cannot reach the precondition.

Now consider what happens *without* a way to write these conclusions down. A security engineer spends forty minutes establishing that `CVE-2022-40303` in `libxml2` is not reachable in service A. Next week the same CVE surfaces in service B, which shares the base image, and someone re-triages it. Next quarter a customer runs their own scanner against your published image and files a support ticket about the same CVE, and someone re-triages it *again* — this time under external pressure. The triage conclusion is real knowledge, produced at real cost, and it evaporates every time because it lives in a Slack thread and a closed Jira ticket instead of in a machine-readable artifact that travels with the software.

That is the problem VEX solves. **A VEX statement is the durable, attributable, machine-processable record of a triage conclusion.** It is the *negative* complement to a security advisory. An advisory says "component X version Y contains CVE-Z" — a statement about the component in the abstract. A VEX statement says "*my product* P, which includes X, is **not affected** by CVE-Z, because the vulnerable code is not in an execute path" — a statement about the vulnerability *in the context of a specific product*. Advisories are produced by the world (NVD, GHSA, the component's own maintainers). VEX is produced by the party who

integrated the component and therefore knows how it is actually used: the software producer. The scanner asks "does my inventory contain anything with a known CVE?" VEX answers the only question that matters operationally: "of those, which ones are actually a problem?"

Crucially, VEX does not *replace* the advisory or the scanner. It *rides alongside* the finding stream and annotates it. The dream state, which the rest of this chapter builds toward, is a scanner output that shows only findings *not already adjudicated* by an authoritative VEX — the real work, with the noise filtered by decisions someone already made and recorded.

The VEX status model

VEX has a small, precise vocabulary, and precision matters because the whole value proposition is that a machine can act on the status without a human re-reading a paragraph. The authoritative semantics come from the CISA VEX working group documents (the "VEX Use Cases," "Minimum Requirements for VEX," and "VEX Status Justifications"). Every real format maps onto this model, though — annoyingly — with different label spellings, which we will untangle per-format below.

The four statuses

A VEX statement asserts one of exactly **four statuses** about a (product, vulnerability) pair:

- **`NOT_AFFECTED`** — No remediation is required with respect to this vulnerability. This is the workhorse status, and the entire reason VEX exists: it is how a producer says "yes, a scanner will flag this, and I am telling you it is noise, here is why." Because it is a suppression signal, CISA requires it to carry a machine-readable *justification* (the five reasons below) and/or an *impact statement*. A bare `NOT_AFFECTED` with no reason is not conformant and should not be trusted.
- **`AFFECTED`** — The product *is* affected by this vulnerability. A conformant `AFFECTED` statement should carry an **action statement**: remediation or mitigation guidance for the consumer (upgrade to version N, apply this configuration, disable this feature). `AFFECTED` is the honest admission; it is what a producer emits before a fix ships.
- **`FIXED`** — The product contains a fix for the vulnerability. This is how a producer says "this version and later are patched," which lets a

consumer's tooling automatically clear the finding once it observes the fixed version in the SBOM.

- `UNDER_INVESTIGATION` — The producer does not yet know whether the product is affected; triage is in progress. This is a genuinely useful state at scale: it lets a producer publish "we have seen this CVE and are working on it" immediately, which is far better than silence, and it lets consumer tooling distinguish "not yet triaged" from "triaged and cleared."

Do not invent statuses. There is no `WONT_FIX` status, no `MITIGATED` status, no `LOW_RISK` status. Those concepts are expressed *within* the four statuses — a "won't fix" is an `AFFECTED` with an action statement that says so; a "mitigated" is usually a `NOT_AFFECTED` with the `inline_mitigations_already_exist` justification.

Every statement also carries, at minimum, three things beyond the status (the CISA "minimum requirements"): a **product identifier** (which product and which versions — a purl, a CPE, or a product-tree reference), a **vulnerability identifier** (a CVE ID or other vulnerability name), and **provenance/timestamp** metadata (who is asserting this, and as of when — the timestamp is what makes staleness detectable later).

The five `NOT_AFFECTED` justifications

When the status is `NOT_AFFECTED`, CISA defines exactly **five** standardized machine- readable justifications. These are the load-bearing vocabulary of VEX; learn them cold and do not paraphrase the labels, because tooling matches on them literally.

1. `component_not_present` — The vulnerable component is not in the product at all. This is the strongest, cleanest justification: the scanner matched on something that is not actually shipped. Example: your final distroless image was assembled in a multi-stage build and the vulnerable `git` binary existed only in the builder stage; a naïve SBOM that captured the builder layer flags its CVEs, but the runtime image does not contain `git`. The vulnerable component is simply gone.

2. `vulnerable_code_not_present` — The component *is* present, but the specific vulnerable code is not. This is the backport case and the compile-out case. Example: Debian backported the fix for a CVE into `libfoo` while keeping the upstream version string `1.2.3`; the

version-range match fires, but the affected code path was patched out. Or: the vulnerability lives in an optional module you compiled without. The binary is present; the vulnerable *code* is not.

3. `vulnerable_code_not_in_execute_path` — The vulnerable code is present in the binary, but there is no execution path in your product that reaches it. **This is the reachability justification**, and it is the direct machine-readable output of the reachability analysis from Book 2, Chapter 7. Example: your Go service statically links a library that contains a vulnerable parsing function, but call-graph analysis (`govulncheck`) proves that no code path from any of your entrypoints ever calls that function. The code sits in the binary, dead, unreachable. This justification is the single biggest source of legitimate `NOT_AFFECTED` volume, and — as we will see — the most automatable.

4. `vulnerable_code_cannot_be_controlled_by_adversary` — The vulnerable code is present and reachable, but an adversary cannot supply the input needed to trigger it. Reachability is necessary but not sufficient for exploitability (again, Book 2, Chapter 7): the tainted data has to come from an attacker-controllable source. Example: a deserialization bug in a config-parsing library is reachable, but the only input it ever parses is a build-time-baked, read-only config file that no external actor can influence. The path is live; the attacker has no way to steer it.

5. `inline_mitigations_already_exist` — The vulnerable code is present, reachable, and controllable, but a built-in mitigation prevents exploitation. The compensating control is *inline* — part of the product or its deployment — not an assumption about the customer's environment. Example: a SQL-injection sink is reachable, but every call is gated by a parameterized-query layer and an input-validation allowlist that the adversary cannot bypass; or a memory-corruption bug is neutralized by a compiler hardening flag (stack canaries, CFI) that turns exploitation into a crash rather than code execution.

Notice that justifications 1 through 5 form a natural chain of increasingly weak (but still valid) claims, mirroring the exploitability funnel from Book 2, Chapter 7: *is the component even here → is the vulnerable code here → is it reachable → can the adversary drive it → is it mitigated anyway*. Each step down the chain is a legitimate reason a finding is not actionable, and each is progressively harder to establish and to trust.

`component_not_present` is nearly mechanical; `inline_mitigations_already_exist` is a security-architecture judgment call. That gradient matters when you decide how much you trust someone else's VEX.

The decision tree

The status-and-justification logic is a decision tree, and drawing it makes the semantics concrete. This is exactly the reasoning a triage engineer performs — and, increasingly, the reasoning a tool automates.

```
flowchart TD
    START["Scanner flags: product P contains CVE-Z"] --> Q0{"Is a fixed
version shipped
in P?"}
    Q0 -->|Yes| FIXED["Status: FIXED"]
    Q0 -->|No| Q1{"Is the vulnerable
component present
in P at all?"}
    Q1 -->|No| J1["NOT_AFFECTED
component_not_present"]
    Q1 -->|Yes| Q2{"Is the vulnerable
code present
in the shipped build?"}
    Q2 -->|No| J2["NOT_AFFECTED
vulnerable_code_not_present"]
    Q2 -->|Yes| Q3{"Is the vulnerable
code reachable from
any entrypoint?"}
    Q3 -->|No| J3["NOT_AFFECTED
vulnerable_code_not_in_execute_path"]
    Q3 -->|Yes| Q4{"Can an adversary
control the input
that triggers it?"}
    Q4 -->|No| J4["NOT_AFFECTED
vulnerable_code_cannot_
be_controlled_by_adversary"]
    Q4 -->|Yes| Q5{"Does an inline
mitigation prevent
exploitation?"}
    Q5 -->|Yes| J5["NOT_AFFECTED
inline_mitigations_already_exist"]
    Q5 -->|No| Q6{"Is a fix
available yet?"}
    Q6 -->|Not yet| INV["Status: UNDER_INVESTIGATION
or AFFECTED with
mitigation guidance"]
    Q6 -->|Yes, not applied| AFF["Status: AFFECTED
action: upgrade to fixed version"]
```

The tree is worth internalizing because it is the same shape regardless of format: only the field names change. A well-run VEX program is, operationally, a system for walking this tree once per (product, CVE) pair and recording the leaf you land on.

The three formats

Three formats matter in practice. They express the same status model with different weights, different ergonomics, and different constituencies. You will *consume* all three in a real fleet and probably *produce* at least two, so understand the object model of each rather than picking a favorite.

CSAF 2.0 and its VEX profile

CSAF — the Common Security Advisory Framework — is an **OASIS** standard (CSAF 2.0 was approved in 2022) and the heavyweight of the space. It is the successor to the older CVRF format and is designed as a full **security advisory** format: machine-readable disclosures that big vendors publish for everything they ship. VEX in CSAF is not a separate format but a **profile** of CSAF (`csaf_vex`), one of several document profiles the standard defines. This is the format the large vendors already speak — **Red Hat, Cisco, Oracle, Siemens, SICK, and others** publish CSAF advisories and VEX, and Red Hat in particular has moved its entire security-data feed to CSAF/VEX.

A CSAF 2.0 document is JSON with three top-level structural pillars:

- **`document`** — metadata: the `category` (e.g., `csaf_vex`), `title`, the `tracking` block (document ID, version, revision history, status), the `publisher` (name, category, namespace), and the `csaf_version`.
- **`product_tree`** — the catalog of products this document talks about, expressed as nested `branches` (vendor → product family → product name → version) that mint `full_product_name` entries each with a stable `product_id`. Critically, the product tree supports `relationships` — "product X *installed on* product Y", "component C *bundled with* product P" — which is how CSAF expresses that a vulnerability in a bundled component maps to a specific integrated product. This relationship modeling is CSAF's superpower and its complexity tax; it can represent a large product portfolio in one document, at the cost of verbosity.
- **`vulnerabilities`** — an array, one entry per CVE, each carrying the `cve` ID, `scores` (CVSS), `flags` and `threats` (impact context), `remediations`, and — the VEX heart — **`product_status`**. `product_status` sorts the product IDs from the product tree into buckets: `fixed`, `first_fixed`, `known_affected`, `first_affected`,

`last_affected`, `recommended`, `known_not_affected`, and `under_investigation`. Those buckets are how CSAF encodes the four VEX statuses: `known_not_affected` → `NOT_AFFECTED`, `known_affected` / `first_affected` / `last_affected` → `AFFECTED`, `fixed` / `first_fixed` → `FIXED`, `under_investigation` → `UNDER_INVESTIGATION`.

The `NOT_AFFECTED` justification lives in the vulnerability's **`flags`** array, whose `label` uses exactly the five CISA justification labels (`component_not_present`, `vulnerable_code_not_present`, `vulnerable_code_not_in_execute_path`, `vulnerable_code_cannot_be_controlled_by_adversary`, `inline_mitigations_already_exist`), each pointing at the affected `product_ids`. A trimmed skeleton:

```
{
  "document": {
    "category": "csaf_vex",
    "csaf_version": "2.0",
    "publisher": {
      "category": "vendor",
      "name": "Example Corp",
      "namespace": "https://example.com"
    },
    "title": "Example Corp VEX: CVE-2022-40303 in AcmeProxy",
    "tracking": {
      "id": "EXAMPLE-VEX-2024-0001",
      "version": "1",
      "status": "final",
      "current_release_date": "2026-07-31T00:00:00Z"
    }
  },
  "product_tree": {
    "branches": [
      { "category": "vendor", "name": "Example Corp", "branches": [
        { "category": "product_name", "name": "AcmeProxy 4.2",
          "product": { "product_id": "ACMEPROXY-4.2", "name": "AcmeProx
y 4.2" } }
      ]}
    ]
  },
  "vulnerabilities": [
    {
      "cve": "CVE-2022-40303",
      "product_status": { "known_not_affected": ["ACMEPROXY-4.2"] },
      "flags": [
        { "label": "vulnerable_code_not_in_execute_path",
          "product_ids": ["ACMEPROXY-4.2"] }
      ]
    }
  ]
}
```

CSAF is strict (a published JSON Schema plus mandatory profile tests and a conformance program), auditable, and expressive enough for a hardware vendor's entire catalog. The trade-off is weight: the product-tree modeling is a real learning curve, and hand-authoring CSAF is unpleasant. It is a format for organizations that publish advisories as a first-class product output and want one framework for both the advisory and the VEX.

CycloneDX VEX

CycloneDX (Book 3, Chapter 3) carries VEX natively in its **`vulnerabilities`** array, with an **`analysis`** object per vulnerability. This is the developer-friendly option, and it has a decisive ergonomic advantage: the VEX can live *inline in the same BOM* that inventories the components, so the assertion and the thing it asserts about travel together. It can also be emitted as a standalone VEX-only CycloneDX document (a BOM with a populated `vulnerabilities` array and empty/minimal `components`), which is the pattern when you want to distribute VEX separately from the SBOM.

CycloneDX uses its *own* vocabulary rather than the raw CISA labels — a wrinkle you must know when normalizing across formats. The `analysis.state` enum is `resolved`, `resolved_with_pedigree`, `exploitable`, `in_triage`, `false_positive`, and `not_affected`. The `analysis.justification` enum is `code_not_present`, `code_not_reachable`, `requires_configuration`, `requires_dependency`, `requires_environment`, `protected_by_compiler`, `protected_at_runtime`, `protected_at_perimeter`, and `protected_by_mitigating_control`. And `analysis.response` carries remediation intent: `can_not_fix`, `will_not_fix`, `update`, `rollback`, `workaround_available`. These map onto the CISA model but not one-to-one: CycloneDX's `code_not_reachable` is CISA's `vulnerable_code_not_in_execute_path`; `protected_*` and `requires_*` collectively cover CISA's `vulnerable_code_cannot_be_controlled_by_adversary` and `inline_mitigations_already_exist`. Any cross-format VEX consumer needs a translation table between these vocabularies; do not assume the strings are portable.

A CycloneDX `analysis` block asserting the reachability case:

```
{
  "vulnerabilities": [
    {
      "bom-ref": "vuln-cve-2022-40303-libxml2",
      "id": "CVE-2022-40303",
      "source": { "name": "NVD", "url": "https://nvd.nist.gov/vuln/
detail/CVE-2022-40303" },
      "affects": [
        { "ref": "pkg:deb/debian/libxml2@2.9.13-2?arch=amd64" }
      ],
      "analysis": {
        "state": "not_affected",
        "justification": "code_not_reachable",
        "response": ["will_not_fix"],
        "detail": "Static call-graph analysis (govulncheck) shows no
path from any service entrypoint reaches xmlNodeDumpOutput; the
affected function is dead code in the shipped binary.",
        "firstIssued": "2026-07-31T00:00:00Z",
        "lastUpdated": "2026-07-31T00:00:00Z"
      }
    }
  ]
}
```

The `affects[].ref` points at a `bom-ref` (or purl) elsewhere in the same document, which is the inline binding that makes CycloneDX pleasant: the VEX statement is anchored to the exact component entry in the exact SBOM. CycloneDX VEX is the natural choice when your SBOM is already CycloneDX (most modern scanners emit it) and your producers are developers who want VEX to be a normal part of the build output rather than a separate advisory-publishing workflow. It is consumed natively by Dependency-Track, which is the reference implementation for the ingest side.

OpenVEX

OpenVEX is the minimalist. Born in the **OpenSSF** in 2023, it exists because CSAF is heavy and CycloneDX VEX is coupled to a full BOM schema, and neither is ideal when what you want is a *tiny, standalone, signable* VEX document that you can generate in CI, attach to an artifact as an attestation, and reason about mechanically. OpenVEX deliberately does almost nothing except encode the CISA status model in the smallest possible JSON-LD envelope.

An OpenVEX document is a small object with a `@context` (versioned spec URI), an `@id`, an `author` / `role`, a document-level `timestamp` and integer `version`, and a `statements` array. Each statement is a (vulnerability, products, status) triple with an optional `justification` (for `not_affected`), `impact_statement`, `action_statement` (for `affected`), and its own `timestamp`. Statuses are lowercase — `not_affected`, `affected`, `fixed`, `under_investigation` — and the justification labels are exactly the five CISA labels. That fidelity to the CISA vocabulary (unlike CycloneDX's re-spelling) is deliberate.

```
{
  "@context": "https://openvex.dev/ns/v0.2.0",
  "@id": "https://example.com/vex/2026-libxml2-acmeproxy",
  "author": "Example Corp Product Security",
  "role": "Document Creator",
  "timestamp": "2026-07-31T00:00:00Z",
  "version": 1,
  "statements": [
    {
      "vulnerability": { "name": "CVE-2022-40303" },
      "products": [
        { "@id": "pkg:oci/acmeproxy@sha256:abcd1234...", "subcomponents
": [
          { "@id": "pkg:deb/debian/libxml2@2.9.13-2" }
        ]}
      ],
      "status": "not_affected",
      "justification": "vulnerable_code_not_in_execute_path",
      "impact_statement": "xmlNodeDumpOutput is not reachable from any
entrypoint in AcmeProxy 4.2."
    }
  ]
}
```

The `products` and `subcomponents` use purls, and the crucial modeling nuance is that OpenVEX distinguishes the *product* (the thing you ship — an OCI image identified by digest) from the *subcomponent* (the vulnerable dependency inside it). That is exactly right: the vulnerability is in the subcomponent, but the *affected/not-affected claim* is about the product. Pinning the product to an OCI digest is what makes the statement precise and what lets a consumer bind it to a specific image.

OpenVEX ships with a reference tool, **`vexctl`**, that creates, merges, and — critically — *attests* VEX documents, attaching them to container images via Sigstore so they ride in the OCI registry alongside the artifact

(more below). It also implements a `vexctl filter` mode that applies a set of VEX documents to a scanner's results to suppress adjudicated findings — the consumer side of the loop. OpenVEX's minimalism is the whole point: a document you can generate, sign, diff, and version-control without an advisory-publishing apparatus.

SPDX 3.0's security profile (brief)

For completeness: **SPDX 3.0** (Book 3, Chapter 2) introduced a **Security profile** that can also express VEX, modeled — in SPDX's characteristic way — as **relationships** rather than a dedicated document type. The profile defines VEX assessment relationship classes, including `VexNotAffectedVulnAssessmentRelationship` (with a `justificationType` drawn from the CISA labels and optional impact statement), `VexAffectedVulnAssessmentRelationship` (with action statements), `VexFixedVulnAssessmentRelationship`, and `VexUnderInvestigationVulnAssessmentRelationship`, all relating a `Vulnerability` element to the affected SPDX element(s). This is the natural home for VEX if your inventory is already SPDX 3.0 and you want everything in one graph model. Tooling for the SPDX security profile is younger than for the other three, so in mid-2026 you are far more likely to meet CSAF, CycloneDX VEX, or OpenVEX in the wild; know that SPDX can express VEX, and treat it as the fourth normalization target rather than a primary producer format.

Choosing among them

Dimension	CSAF 2.0 (VEX profile)	CycloneDX VEX	OpenVEX
Governing body	OASIS	OWASP / ECMA-424	OpenSSF
Encoding	JSON, schema-validated	JSON (or XML)	JSON-LD (tiny)
Weight	Heavy	Medium	Minimal
Standalone or inline	Standalone advisory	Inline in BOM *or* standalone	Standalone only
Justification vocabulary	CISA labels (in `flags`)	CycloneDX's own enum	CISA labels (verbatim)

Dimension	CSAF 2.0 (VEX profile)	CycloneDX VEX	OpenVEX
Product model	Rich `product_tree` + relationships	`bom-ref` / purl inside BOM	purl product + subcomponents
Signing / attestation	Detached signature, PGP/GPG norms	Via BOM signing (Book 5)	First-class (`vexctl` + Sigstore)
Typical producers	Red Hat, Cisco, Oracle, Siemens	App/dev teams, scanner output	CI pipelines, OSS projects
Reference consumer	CSAF aggregators, vendor portals	OWASP Dependency-Track	`vexctl filter`, Grype/Trivy VEX
Reach for it when…	You publish advisories for a product portfolio	Your SBOM is already CycloneDX	You want a tiny signed statement per artifact

The honest guidance: this is a normalize-don't-choose situation, exactly as with the SBOM formats in Chapter 3. If you are a *large vendor* publishing advisories, you will almost certainly land on CSAF, because it is the advisory framework and VEX is one profile of it. If you are a *development organization* producing software and want VEX as build output, CycloneDX VEX (inline with your CycloneDX SBOM) or OpenVEX (standalone, signed, attached to the image) are both good, and many shops use OpenVEX for the auto-generated reachability statements and CycloneDX for the richer human-authored analysis. On the *consume* side you must handle all of them: an ingest pipeline that normalizes CSAF, CycloneDX VEX, and OpenVEX into a single internal status model — the four statuses and five justifications — is the durable design, because your upstream vendors will send whatever they send.

How VEX flows: producer to consumer

VEX is only valuable if it *moves* — from the party who knows (the producer) to the party drowning in findings (the consumer), with enough integrity that the consumer can act on it automatically. The flow has three legs: produce, distribute (as a signed attestation), and consume (as a suppression filter).

```
flowchart LR
    subgraph Producer["Producer (software vendor / internal team)"]
        BUILD["Build produces
artifact + SBOM"] --> TRIAGE["Triage: reachability tools
+ human security review"]
        TRIAGE --> GENVEX["Generate VEX
status + justification"]
        GENVEX --> SIGN["Sign VEX
cosign / vexctl attest"]
    end
    SIGN --> DIST["Distribute:
OCI referrers, VEX feed,
attestation store"]
    subgraph Consumer["Consumer (downstream / customer / other team)"]
        SCAN["SBOM x vuln feed
= raw findings"] --> INGEST["Ingest + verify
VEX signatures"]
        DIST --> INGEST
        INGEST --> FILTER["Apply VEX:
suppress NOT_AFFECTED/FIXED"]
        FILTER --> RESULT["De-noised,
prioritized findings"]
    end
```

VEX as a signed attestation

A `NOT_AFFECTED` claim is a *security assertion*, and an unauthenticated security assertion is worthless — anyone can write a JSON file that says "not affected." So VEX belongs in the same trust machinery as everything else in the supply chain: it should be **signed** and **attributed**, and ideally **bound to the exact artifact** it describes.

The mature pattern, which ties directly to Book 5 (Signing and Attestation), is to package the VEX document as an **in-toto attestation** with a VEX predicate, sign it with Sigstore (`cosign attest` or `vexctl attest`), and push it into the **OCI registry as a referrer** of the image it describes (Chapter 5's OCI referrers / `oci-referrers` model). Now the VEX travels with the artifact: whoever pulls `pkg:oci/acmeproxy@sha256:abcd…` can also pull its attached VEX attestations, verify the signature against the producer's identity (a Fulcio-issued certificate tied to an OIDC identity — Book 5), and confirm the statement was made by the party they think it was. The `subject` of the attestation is the image digest, so the binding between VEX and artifact is cryptographic, not a fragile filename convention.

```
# Producer: attach a signed OpenVEX attestation to an image
vexctl attest --sign acmeproxy.openvex.json \
  ghcr.io/example/acmeproxy@sha256:abcd1234...

# Consumer: verify and apply VEX when scanning
cosign verify-attestation --type openvex \
  --certificate-identity-regexp 'https://github.com/example/.*' \
  --certificate-oidc-issuer https://
token.actions.githubusercontent.com \
  ghcr.io/example/acmeproxy@sha256:abcd1234...

grype ghcr.io/example/acmeproxy@sha256:abcd1234... \
  --vex acmeproxy.openvex.json
```

Scanners have grown native support for this consume step: Grype and Trivy both accept VEX documents (`--vex`) and OpenVEX attestations and will drop or downgrade findings that a verified VEX marks `not_affected` / `fixed`. That is the loop closing — a producer's signed statement mechanically removing noise from a consumer's scan.

The trust question

Here is the uncomfortable part, and it is not a technical problem you can sign your way out of: **do you believe the producer's `NOT_AFFECTED`?** The producer has a structural incentive to under-report — every `AFFECTED` statement is a support burden, a possible SLA breach, a bad look. A dishonest or merely sloppy producer can suppress a real finding by asserting it away. Signing does not fix this; signing tells you *who* made the claim and that it was not tampered with, which is necessary but not sufficient. A signed lie is still a lie; it is just an *attributable* one.

What VEX actually changes is the *epistemics* of the triage, not the *need* for judgment. Before VEX, a `NOT_AFFECTED` conclusion was invisible: buried in a ticket, unattributable, unauditable. After VEX, the claim is **explicit** (a specific status and justification), **attributable** (signed by a named identity), **timestamped** (so staleness is detectable), and **machine-processable** (so you can audit *all* of a vendor's `not_affected` claims at once and spot a vendor who marks everything unreachable). That is a large improvement, but it relocates the judgment rather than removing it. You still decide *which producers' VEX you trust and for which justifications*. A reasonable posture: auto-suppress on `component_not_present` and `vulnerable_code_not_present` (mechanical, hard to fake), require a spot-check policy for

`vulnerable_code_not_in_execute_path` (trust the tool-generated ones, sample the human ones), and treat `inline_mitigations_already_exist` from external vendors with real skepticism (it is a judgment call you cannot verify from outside). Trust is per-producer and per-justification, and VEX is what makes that granularity expressible.

Operationalizing VEX at scale

A single VEX statement is easy. A VEX *program* across hundreds of services and thousands of transitive dependencies is where the engineering is. Three ideas make it tractable: generate what you can, record human decisions once, and correlate ruthlessly.

Generating VEX: automation meets human triage

VEX generation is a spectrum from fully mechanical to irreducibly human, and the whole game is to push as much as possible toward the mechanical end.

The automatable frontier is reachability. This is the most exciting development in the space and the direct operational cash-out of Book 2, Chapter 7. Tools that perform static call-graph analysis can *prove* that a vulnerable symbol is unreachable and emit the corresponding `vulnerable_code_not_in_execute_path` VEX with no human in the loop. The clearest example is **`govulncheck`** for Go: it consumes the Go vulnerability database, does symbol-level reachability against your actual call graph, and reports only vulnerabilities whose *vulnerable functions* your code can actually call. A CVE that `govulncheck` clears is, by construction, a valid `vulnerable_code_not_in_execute_path` `NOT_AFFECTED` — and you can wire the build to serialize that conclusion straight into OpenVEX. Equivalent capabilities exist in varying maturity for other ecosystems (reachability in JVM, Node, and Python SCA tooling). Even `component_not_present` and `vulnerable_code_not_present` can be partly automated: multi-stage-build layer analysis can generate `component_not_present` when a flagged component exists only in a builder stage, and distro backport metadata can generate `vulnerable_code_not_present` for known-backported fixes.

The irreducible human core is architecture and configuration. `vulnerable_code_cannot_be_controlled_by_adversary` and `inline_mitigations_already_exist` are security-architecture judgments a

tool cannot make: they require knowing what data is attacker-controlled and what compensating controls are actually in force. These are the statements your security team writes by hand — and the key discipline is to write each one **exactly once**. The failure mode VEX exists to kill is re-triaging the same transitive CVE five hundred times. When an engineer establishes that `CVE-2022-40303` in `libxml2` is unreachable, that conclusion must become a durable, centrally stored VEX statement keyed to the (component, CVE) pair, so that *every* service that includes that component and *every* consumer of those services inherits the decision automatically. The economic argument is overwhelming: triage is expensive and the same transitive dependencies recur across the entire estate, so amortizing one decision across the fleet is the difference between a tractable program and an impossible one.

The correlation pipeline

Put the pieces together and you get the pipeline this whole book has been building toward: **SBOM inventory (Chapter 5) × vulnerability feed (Book 2, Chapter 5) × VEX = a prioritized, de-noised finding list.** The correlation is a series of joins that progressively strips noise:

```
flowchart TD
    SBOM["SBOM inventory
(central store, Ch 5)
~thousands of components"]
    FEED["Vulnerability feeds
(OSV / GHSA / NVD, Bk2 Ch5)"]
    SBOM --> MATCH["Match: component purl x CVE
= RAW FINDINGS"]
    FEED --> MATCH
    MATCH --> N1["380 findings
(mostly noise)"]
    N1 --> VEXAUTO["Apply auto-generated VEX
(reachability: govulncheck etc.)"]
    VEXAUTO --> N2["~90 findings
unreachable ones suppressed"]
    N2 --> VEXHUMAN["Apply human-triage VEX
(central VEX store)"]
    VEXHUMAN --> N3["~25 findings
prior decisions reused"]
    N3 --> VEXVENDOR["Apply upstream vendor VEX
(verified signatures)"]
    VEXVENDOR --> N4["~12 findings
vendor-cleared suppressed"]
    N4 --> PRIOR["Prioritize by EPSS / KEV /
severity (Bk2 Ch7)"]
    PRIOR --> ACT["~4 actionable findings
ACTUAL WORK"]
```

The numbers are illustrative, but the *shape* is real and repeatedly observed: raw findings in the hundreds, actionable findings in the low single digits or low tens, with VEX doing the heavy lifting of the reduction and exploit-prediction signals (EPSS, CISA KEV; Book 2, Chapter 7) ordering what remains. The de-noised list is the deliverable. Everything before it is plumbing; the point of the plumbing is that a human looks at four things instead of three hundred and eighty, and the four are the right four.

Pitfalls

VEX is powerful enough to hurt you. Four failure modes recur, and a serious program designs against each.

- **Stale VEX.** A `NOT_AFFECTED` is a claim about a *specific build*. The moment the code changes — a refactor that adds a call into the previously-dead function, a config change that enables the vulnerable feature — the claim may silently become false while the VEX

document keeps asserting it. This is the most dangerous failure because it *hides a real vulnerability behind an authoritative-looking suppression*. The mitigations are structural: pin VEX to artifact digests (a VEX for `@sha256:abcd…` simply does not apply to the next build's `@sha256:ef01…`), regenerate auto-generated reachability VEX on every build so it tracks the current call graph, and put an expiry/review-date on human-authored statements so a `not_affected` that is two years old gets re-examined rather than trusted forever. The timestamp in every VEX statement exists precisely so staleness is detectable.

- **Scope errors.** A VEX for the wrong version or the wrong product is worse than no VEX: it suppresses a finding that is genuinely present. The discipline is precise product identification — digest-pinned OCI subjects, exact purl version ranges — and treating a VEX whose scope you cannot resolve as *not applicable* rather than guessing. Loose version matching in VEX application is a foot-gun.
- **Over-suppression.** The organizational failure mode: VEX becomes a tool for making the dashboard green rather than for recording truth. A team under pressure marks findings `not_affected` with thin justification to clear the board. Because VEX is attributable and auditable, you can *detect* this — audit the `not_affected` statements, sample the human ones, look for producers or teams whose suppression rate is anomalous — but you have to actually run that audit. VEX makes over-suppression visible; it does not prevent it.
- **VEX sprawl.** Thousands of documents across formats, versions, and producers, with no authoritative merge, produces the same chaos VEX was meant to cure — now with signatures. The answer is centralization: a single VEX store (part of the SBOM platform, Chapter 5) that ingests all formats, normalizes to the four-status model, deduplicates, resolves conflicts by producer trust and recency, and serves the merged view to every scanner. `vexctl merge` and the platform's ingest layer are the tools; the principle is that there is *one* answer to "what do we believe about (component, CVE)," not one per document.

Distributed-systems lens

VEX is, at bottom, a mechanism for making *fleet-scale* vulnerability management tractable, and the distributed-systems framing is where it stops being a file format and becomes an architecture.

The central move is to treat triage conclusions as **shared, authoritative state** rather than per-team local knowledge. In a large backend organization you have many services, many teams, many repos, and a *heavily shared* dependency base — the same base images, the same core libraries, the same transitive graph reappearing across hundreds of services (Book 2, Chapter 8 on internal registries; Book 3, Chapter 5 on the central store). That sharing is exactly why centralized VEX pays off superlinearly: a single `NOT_AFFECTED` decision about a common transitive dependency suppresses noise across *every* service that includes it and *every* consumer of those services. The economics invert from "cost scales with services × CVEs" to "cost scales with *distinct* (component, CVE) pairs," and the latter grows far more slowly because of the sharing. The **internal VEX store becomes the organization's institutional memory**: the durable, queryable record of "we already looked at this CVE, and here is what we concluded and who concluded it." That memory is the single highest-leverage asset a mature program has, because it is what stops the same forty-minute triage from happening five hundred times.

Two platform capabilities make this real. First, **auto-generate reachability VEX from the shared build/analysis platform.** If your organization runs a hermetic, centralized build platform (Book 4, Chapter 10), it already has every service's real call graph in a uniform place — which is exactly the input reachability analysis needs. Run `govulncheck`-class analysis as a platform step and emit `vulnerable_code_not_in_execute_path` OpenVEX for every build, for free, across the whole estate. This is the same "implement once, amortize across every tenant" pattern that Chapter 4 identified for SBOM generation, applied to VEX: the most valuable class of VEX statement, produced mechanically by shared infrastructure, regenerated on every build so it never goes stale. Second, **centralize human triage as signed VEX in the SBOM platform** so that one security engineer's decision propagates automatically rather than being re-litigated per team. The platform ingests the decision once, signs it, and serves it to

every scanner in the fleet as a suppression that no downstream team has to reproduce.

Finally, the same architecture runs in reverse on the *consume* side: **ingesting upstream vendors' VEX cuts third-party noise the same way your internal VEX cuts internal noise.** Red Hat, your base-image vendor, and your language runtime all publish VEX (increasingly CSAF); a fleet-scale program pulls those feeds into the same central VEX store, verifies their signatures, and applies them so that the CVEs a vendor has already adjudicated for you never reach a human. The internal store and the external feeds merge into one normalized view, and every scanner in the organization queries that one view. That is the endgame: vulnerability management at fleet scale where the flood is reduced to a stream, the stream is prioritized by exploitability, and every suppression in it is an explicit, attributable, signed, non-stale decision that someone — a tool or a person — made exactly once.

Key takeaways

- **An SBOM plus a scanner is a noise machine.** The output is dominated by findings that are real component matches but non-exploitable in your product (unreachable, wrong config, code not present, already mitigated). Without a way to record "not affected, because…", every service and every consumer re-triages the same CVE, and real issues drown.
- **VEX is the durable, attributable, machine-readable record of a triage conclusion** — the negative complement to an advisory. Advisories say a component is vulnerable; VEX says whether a *specific product* is affected, and why. It rides alongside findings and annotates them; it does not replace the scanner.
- **Four statuses, exactly:** `NOT_AFFECTED`, `AFFECTED`, `FIXED`, `UNDER_INVESTIGATION`. There is no "won't fix" or "low risk" status; those live inside the four. `NOT_AFFECTED` must carry a justification and/or impact statement to be meaningful.
- **Five `NOT_AFFECTED` justifications, exactly, per CISA:** `component_not_present`, `vulnerable_code_not_present`, `vulnerable_code_not_in_execute_path` (the reachability one, Book 2, Chapter 7), `vulnerable_code_cannot_be_controlled_by_adversary`, and `inline_mitigations_already_exist`. They form a funnel of

increasingly weak but still-valid claims; do not invent or paraphrase the labels.

- **Three formats you will meet:** CSAF 2.0's VEX profile (OASIS heavyweight, `product_tree` + `product_status` + `flags`, used by Red Hat/Cisco/Oracle), CycloneDX VEX (developer- friendly, inline `vulnerabilities` / `analysis` with its *own* vocabulary), and OpenVEX (OpenSSF minimalist, standalone, signable, `vexctl`, verbatim CISA labels). SPDX 3.0's security profile is a fourth, younger option. On consume you normalize all of them into the four-status model.
- **VEX belongs in the trust machinery:** sign it, attribute it, bind it to the artifact digest as an in-toto attestation attached via OCI referrers (Chapter 5, Book 5). Signing makes the claim attributable and tamper-evident — but a signed `not_affected` is only as good as the producer's honesty; VEX makes the judgment explicit and auditable, it does not remove it.
- **Auto-generate what you can, record human decisions once, correlate ruthlessly.** Reachability tools (`govulncheck`) auto-emit `vulnerable_code_not_in_execute_path` VEX on every build; architecture/config judgments are written by hand exactly once and reused fleet-wide. The pipeline SBOM × vuln feed × VEX turns hundreds of raw findings into a handful of actionable ones.
- **Design against the four pitfalls:** stale VEX (pin to digests, regenerate reachability per build, expire human statements), scope errors (precise product IDs, don't guess), over-suppression (audit the `not_affected` claims VEX makes visible), and sprawl (centralize and normalize into one authoritative view).
- **At fleet scale VEX becomes architecture:** a central VEX store is the org's "we already looked at this CVE" memory; the shared build platform auto-produces reachability VEX for free across every tenant; one signed human decision suppresses noise everywhere; and ingesting upstream vendors' VEX cuts third-party noise the same way. Cost scales with *distinct* (component, CVE) pairs, not services × CVEs.

Further reading

- **CISA SBOM/VEX working group** — "Vulnerability-Exploitability eXchange (VEX) — Use Cases" (2022), "Minimum Requirements for Vulnerability Exploitability eXchange (VEX)" (2023), and "VEX Status Justifications" (June 2022), the authoritative source for the four statuses and five justifications used throughout this chapter.
- **CSAF 2.0** — OASIS Committee Specification, "Common Security Advisory Framework Version 2.0" (2022), especially the VEX profile (`csaf_vex`) and the `product_tree` / `product_status` / `flags` model. See also the `csaf` reference tooling.
- **Red Hat CSAF/VEX** — Red Hat's security-data documentation on migrating its advisory and VEX feed to CSAF, as a large-vendor reference implementation.
- **CycloneDX 1.6 specification** — the `vulnerabilities` array and the `analysis` object (`state`, `justification`, `response`, `detail`), and the ECMA-424 standardization; OWASP CycloneDX, and the OWASP Dependency-Track documentation for the consume side.
- **OpenVEX** — the OpenSSF OpenVEX specification (`openvex.dev`), the `openvex/spec` repository, and the **`vexctl`** tool (`openvex/vexctl`) for creating, merging, attesting, and filtering with VEX.
- **SPDX 3.0 Security profile** — the SPDX 3.0 model documentation for the VEX assessment relationship classes (`VexNotAffectedVulnAssessmentRelationship` and siblings).
- **`govulncheck`** — the Go vulnerability scanner (`golang.org/x/vuln/cmd/govulncheck`) and the Go team's writing on symbol-level reachability, as the reference example of auto-generating `vulnerable_code_not_in_execute_path` assertions.
- **Grype and Trivy VEX support** — Anchore Grype and Aqua Trivy documentation on `--vex` and OpenVEX attestation consumption, for the scanner-side suppression loop.
- **Cross-references in this suite:** Book 2, Chapter 5 (Vulnerability Databases and Identifiers), Chapter 6 (Software Composition Analysis in Depth), and Chapter 7 (Reachability, Exploitability, and Prioritization); Book 3, Chapter 3 (CycloneDX in Depth) and Chapter 5 (SBOM Distribution, Storage, and Querying at Scale); Book 5

(Signing and Attestation) for Sigstore, in-toto attestations, and OCI referrers.

Chapter 7 — SBOM Quality, Completeness, and Limitations

What this chapter covers. The previous chapters were constructive: what an SBOM is (Chapter 1), how to write it in SPDX (Chapter 2) or CycloneDX (Chapter 3), how to generate it (Chapter 4), how to distribute and query it at scale (Chapter 5), and how to correlate it with vulnerabilities using VEX (Chapter 6). This chapter is the audit. It asks the questions a skeptical senior engineer *should* ask before betting an incident-response plan on a pile of JSON documents: **how good are these SBOMs actually, how would I measure that, and what will they never tell me no matter how good they get?** The uncomfortable thesis is that most SBOMs in circulation today are worse than their owners believe, that "we have SBOMs for 100% of our services" is a coverage metric masquerading as a visibility metric, and that even a flawless SBOM is an *inventory*, not a security assessment. None of this makes SBOMs worthless — they are necessary infrastructure. It makes them a foundation, and you have to know the shape of the foundation before you build on it. False confidence is the specific failure mode this chapter exists to prevent.

Learning goals — after this chapter you should be able to:

- Decompose "SBOM quality" into its distinct dimensions — **completeness, accuracy, identifier validity, required-field presence, graph fidelity, freshness, and provenance** — and explain why **conformance, completeness, and accuracy are three different things** an SBOM can pass or fail independently.
- Use the real measurement tools — **sbomqs, ntia-conformance-checker, the CycloneDX/SPDX validators, eBay's sbom-scorecard** — and state precisely what each one checks and, more importantly, what none of them can check.
- Confront the **completeness measurement problem**: you can validate what is present, but you cannot in general know what is missing without ground truth you rarely have; and know the

comparison-based techniques that partially substitute for that ground truth.

- Enumerate the **systematic accuracy failures** — static linking, vendoring, shaded JARs, bundled JavaScript, stripped binaries, firmware, broken identifiers — and explain why *false negatives* (a present component absent from the SBOM) are the dangerous class.
- State the **fundamental limits**: an SBOM does not tell you whether you are exploitable, whether a component is malicious, whether you have first-party bugs, whether the build was honest, or anything about zero-days — and map each limit to the mechanism that *does* address it.
- Design a **fitness-for-purpose** quality bar and a CI **quality gate**, and track quality as a fleet-wide distribution rather than a binary coverage checkbox.

A boundary note. This chapter builds directly on Chapter 4's account of generation and its hard cases, and on Chapter 4's "measurement problem" — here both are deepened. It assumes Book 2's identifier model (purl versus CPE, Book 2, Chapter 5), its SCA pipeline (Book 2, Chapter 6), and its treatment of reachability (Book 2, Chapter 7) and malicious packages (Book 2, Chapter 4). It hands off VEX to Chapter 6, provenance to Books 4 and 5, and fleet-level metrics to Book 8, Chapter 8 — Metrics, Audits, and Executive Reporting.

What "quality" is not: one word for seven properties

The first mistake is treating "quality" as a scalar. Ask "is this a good SBOM?" and you have asked an unanswerable question, because the document can be excellent along one axis and useless along another, and the axes do not correlate. A CycloneDX 1.6 file can be schema-perfect and NTIA-conformant while omitting a third of the components in the image it describes. A file with every component present can carry version strings that no vulnerability database will ever match. An SBOM can be flawless the day it is generated and a lie six hours later when the base image is rebuilt. "Quality" is shorthand for at least seven distinct properties, and you must be able to name which one you mean.

```
flowchart TD
    Q["SBOM QUALITY
(not one number)"]
    Q --> C["COMPLETENESS
are all real components present?
the hardest to measure"]
    Q --> A["ACCURACY
are the listed components and
versions actually correct?"]
    Q --> I["IDENTIFIERS
valid purls / CPEs that
downstream matching can use"]
    Q --> F["REQUIRED FIELDS
NTIA minimum elements,
supplier, author, timestamp"]
    Q --> G["GRAPH FIDELITY
real dependency relationships,
not a flat bag of components"]
    Q --> R["FRESHNESS
does it describe THIS artifact,
this digest, right now?"]
    Q --> P["PROVENANCE
do you trust who or what
generated it, and is it signed?"]

    classDef hard fill:#7f1d1d,stroke:#f87171,color:#fff;
    classDef med fill:#78350f,stroke:#fbbf24,color:#fff;
    classDef ok fill:#14532d,stroke:#4ade80,color:#fff;
    class C hard;
    class A,I,G med;
    class F,R,P ok;
```

Completeness asks whether every component actually present in the artifact appears in the document. It is first because it is both the most important and, as the rest of this chapter labors, the one property you fundamentally cannot verify without an oracle you do not have.

Accuracy asks whether the components that *are* listed are correctly identified: right name, right version, right supplier. A complete list of wrong facts is not progress. Accuracy and completeness are orthogonal — you can have a short accurate list or a long inaccurate one.

Identifier validity is a special, load-bearing slice of accuracy. Book 2, Chapter 5 established that a component is only useful to downstream tooling if it carries an identifier those tools can match on — a well-formed **purl** for the OSV/GHSA world, a correct **CPE** for the NVD world. A component present in the SBOM with a `pkg:generic` fallback purl and no

CPE is, for vulnerability-matching purposes, nearly as invisible as a component that is absent. This is why it earns its own axis: an SBOM can be complete and accurate in prose and still answer zero vulnerability queries.

Required-field presence is conformance: does the document carry the fields a standard demands? The NTIA minimum elements (supplier, component name, version, other unique identifiers, dependency relationship, author of the SBOM, timestamp) are the canonical floor. This is the *most* mechanically checkable property and, not coincidentally, the one most often mistaken for quality writ large.

Graph fidelity is Chapter 1's "graph, not list" insistence applied to quality: are the `dependsOn` / `DEPENDS_ON` relationships present and correct, or is the file a flat inventory with every component hanging off the root? A flat list still tells you *what* is present, but it destroys the *why* — you cannot answer "which of my services pulls in this transitively, and through what" without the edges.

Freshness asks whether the document describes the artifact you are actually running. SBOMs decay. An SBOM generated against `myservice:1.4.2` is a fabrication the moment a `myservice:1.4.3` rebuild pulls a patched `libssl`. Freshness is why Chapter 5 keyed SBOM storage to the immutable image *digest*, never a mutable tag.

Provenance asks whether you trust the SBOM's origin. An SBOM is itself an artifact in the supply chain, and an attacker who can rewrite your SBOM can hide the very component they injected. Provenance — who or what generated this, and is that claim cryptographically verifiable — is addressed by signing and attestation (Books 4 and 5), not by anything inside the component list.

The distinction that everything hangs on

Hold three of these apart deliberately, because conflating them is the single most common error in SBOM programs:

- **Conformance** — does the document have the required fields, well-formed? Answered by validators and NTIA checkers. *Cheap, mechanical, and about form.*
- **Completeness** — are all the real components present? Requires ground truth. *Expensive, often impossible, and about the world.*

- **Accuracy** — are the present components correctly identified? Requires per-component verification. *Expensive, partial, and about correspondence.*

An SBOM can pass conformance perfectly while failing completeness catastrophically, because conformance is a property of the *document* and completeness is a property of the *relationship between the document and the artifact*. A validator never sees the artifact. It cannot know that `busybox` is in the image; it can only confirm that whatever components you *did* list each carry a supplier and a version. Nothing about "this file conforms to the NTIA minimum elements" implies "this file lists everything in my container." Treat any tool that emits a single quality score as reporting a weighted blend of these independent properties, and always ask which property drove the number.

Measuring what is there: tools and rubrics

Given the dimensions, what can you actually measure, and with what? The tooling divides neatly by which property it inspects, and the division exposes a hard boundary: **everything in this section validates the document; none of it validates the document against the artifact.** Every tool here is blind to completeness in the strong sense. Keep that in view as we go.

Schema validity is table stakes, not quality

The floor beneath the floor is *is this even a valid document of its claimed format*. CycloneDX ships `cyclonedx-cli validate`, which checks a file against the JSON Schema for the declared spec version; the SPDX ecosystem offers the online SPDX Online Tool and libraries such as the Python `spdx-tools` (`pyspdxtools --validate`) that check against the SPDX schema and the tag-value/JSON grammar.

```
# CycloneDX: schema validation against a specific spec version
cyclonedx-cli validate --input-file sbom.cdx.json --input-version v1_6

# SPDX: parse + validate structural correctness
pyspdxtools -i sbom.spdx.json
```

Schema validity means the JSON parses, the required top-level fields exist, enums hold valid values, and references resolve. It says nothing about truth. A schema validator will happily pass an SBOM that lists one

component — `pkg:npm/left-pad@1.0.0` — for a 900-MB container running a JVM and a Postgres client. It is a necessary gate (a malformed SBOM breaks every downstream consumer, as Chapter 5's ingestion pipeline demonstrated) and it is the weakest possible signal of quality. Pass it and move on; do not report it as "our SBOMs are good."

NTIA minimum-elements conformance: the floor

The NTIA "Minimum Elements for a Software Bill of Materials" (12 July 2021, published in response to Executive Order 14028) defines the field-level floor: for each component, a **supplier**, **component name**, **version**, **other unique identifiers**, and **dependency relationship**; and for the document, the **author of the SBOM data** and a **timestamp**. It also specifies *automation support* (one of SPDX, CycloneDX, or SWID) and *practices and processes* — the human commitments around depth, frequency, and handling of known unknowns.

The tool most associated with checking this is the SPDX community's **ntia-conformance-checker** (maintained under the SPDX org), which takes an SPDX document and reports whether each minimum element is present:

```
$ ntia-checker --file sbom.spdx.json --verbose
NTIA conformant: False
Missing supplier names:
 - glibc
 - openssl
 - zlib
Missing version info: (none)
Total components: 214
Components missing supplier: 3
```

Read that output carefully. It is telling you three of 214 components lack a supplier field. It is **not** telling you whether 214 is the right number. If the real image contains 260 components and 46 were never cataloged, `ntia-conformance-checker` reports nothing about them, because it cannot see them — they are not in the file, and the file is all it has. This is the conformance/completeness gap made concrete: a document can be **100% NTIA-conformant and 82% complete**, and the conformance checker will call it conformant without a flicker. Conformance is a property of the rows that exist, not of the rows that should.

The NTIA document itself is honest about this. It introduces the concept of **"known unknowns"** — an SBOM author explicitly declaring that a portion of the tree is not enumerated (a partial component, an opaque third-party binary). A mature SBOM that *declares* its incompleteness is more trustworthy than one that silently implies completeness it does not have. Conformance can encode "I know I don't know"; it cannot detect "I don't know that I don't know."

Quality scoring: sbomqs, sbom-scorecard, and the rubrics

Above bare conformance sit the *quality scorers*, which grade an SBOM across multiple weighted categories and emit a composite score. The most widely used open-source scorer is **sbomqs** (Interlynk). It parses SPDX or CycloneDX and scores dozens of features grouped into categories — roughly: **structural** (is it spec-compliant and parseable), **NTIA minimum elements** (are the mandated fields present), **semantic** (do fields carry meaningful, correctly-typed values), **quality** (do components have valid identifiers, licenses with recognized SPDX IDs, checksums, non-empty suppliers), and **sharing** (is the document licensed for distribution). Recent versions also score against external profiles such as the German **BSI TR-03183-2** guideline and OpenChain Telco.

```
$ sbomqs score sbom.cdx.json
SBOM Quality Score: 6.8  components:214  sbom.cdx.json
+-----------------------+--------------------------------+-----------+
| CATEGORY              | FEATURE                        | SCORE     |
+-----------------------+--------------------------------+-----------+
| NTIA-minimum-elements | comp_with_supplier             | 9.9/10.0  |
|                       | comp_with_version              | 10.0/10.0 |
|                       | sbom_authors                   | 10.0/10.0 |
| Quality               | comp_with_valid_licenses       | 7.1/10.0  |
|                       | comp_with_uniq_ids (purl/cpe)  | 5.4/10.0  |
|                       | comp_with_checksums            | 4.2/10.0  |
| Semantic              | comp_with_primary_purpose      | 3.0/10.0  |
| Structural            | spec_valid                     | 10.0/10.0 |
+-----------------------+--------------------------------+-----------+
```

eBay's sbom-scorecard takes a similar approach with a leaner rubric (presence of purls, of licenses, of versions; a "spec compliance" check) and is useful precisely because comparing two scorers on the same file exposes how much a "score" depends on the rubric's weighting. OWASP's **Software Component Verification Standard (SCVS)** ships a *BOM Maturity Model* — not a tool but a rubric — that grades SBOM practices

across levels and is the right reference when you want a vendor-neutral definition of "what a Level 2 SBOM program looks like."

Here is the essential caveat, and it is the same one as for conformance: **every field a scorer checks is a field that must already be in the document.** `comp_with_uniq_ids: 5.4` means 54% of the *listed* components have a valid purl or CPE — a genuinely useful accuracy signal. It says nothing about the components that were never listed. A scorer's number goes *up* as your listed components get better identifiers and *cannot go down* when a whole class of component is missing, because the missing class contributes no rows to score. It is entirely possible to raise your sbomqs score by enriching a badly incomplete SBOM, producing a higher-scoring document that is no more complete. Scorers measure the quality of what you captured, not the fraction of reality you captured.

The table: dimension, measurement, tool

Quality dimension	What "good" means	How to measure it	Tools
Schema validity	Parses, valid enums, refs resolve	Validate against format schema	`cyclonedx-cli validate`, `pyspdxtools`, SPDX Online Tool
Required fields (conformance)	NTIA minimum elements present	Check each mandated field per component	ntia-conformance-checker, sbomqs (NTIA category)
Identifier validity	Valid purls / correct CPEs	Parse identifiers, check purl grammar, match against known types	sbomqs (`comp_with_uniq_ids`), eBay sbom-scorecard, custom purl linters
Field richness / semantics	Licenses (SPDX IDs), checksums, suppliers non-empty	Score presence + well-formedness of value fields	sbomqs (Quality/ Semantic), sbom-scorecard

Quality dimension	What "good" means	How to measure it	Tools
Graph fidelity	Real dependency edges, not a flat list	Count relationships vs components; check for a connected tree	Custom (relationship-count ratio); partial in sbomqs
Completeness	**All real components present**	**Comparison against ground truth (see next section)**	**No single tool — diff-based, no oracle**
Freshness	Describes this exact artifact	Compare SBOM subject digest to running artifact digest	Registry/attestation lookup (Chapter 5), admission policy
Provenance	Trusted, signed origin	Verify signature / attestation	`cosign verify-attestation`, in-toto (Books 4–5)

Notice which row has no tool. Completeness is the property that matters most for vulnerability response and the only one with no validator, because it is the only one that requires knowledge outside the document.

The completeness problem: how do you know what is missing?

You can validate what is present. You cannot, in general, know what is absent. This is not a tooling gap that a better parser will close; it is structural. To *know* an SBOM is complete you would need an independent, authoritative enumeration of every component in the artifact — and if you had that, you would use *it* as your SBOM. The oracle you would need to check the SBOM is exactly the artifact the SBOM was supposed to produce. Chapter 4 named this the measurement problem; here we deepen it and salvage what can be salvaged.

What can be salvaged is *relative* completeness through **comparison**. You cannot compare an SBOM to the truth, but you can compare it to *another observation of the same artifact made from a different vantage point*, and every disagreement is a lead. Two techniques dominate.

Tool-diffing. Run two independent generators (say Syft and Trivy, per Chapter 4) against the same image and diff the component sets. Where they agree, your confidence rises — with a caveat below. Where they disagree, you have found either a false positive in one or a false negative in the other, and both are worth investigating. This is cheap and repeatable in CI. Its limit is *shared blindness*: two binary scanners that both rely on OS package databases and embedded manifests will *both* miss a statically-linked, stripped C library, and their perfect agreement is not evidence of completeness — it is evidence they share a blind spot. Agreement bounds your false-positive rate, not your false-negative rate.

Source-versus-binary diffing. The more powerful comparison exploits the vantage asymmetry Chapter 4 built its whole argument on. Generate a *source/build* SBOM (from lockfiles and the build graph — high fidelity on your application's declared dependency closure, with exact versions and intent) and an *image/binary* SBOM (from scanning the final artifact — high fidelity on OS packages and whatever is physically present). Then diff them. The two are *supposed* to differ in structured ways, and the structure of the difference is diagnostic.

```
flowchart TD
    subgraph SRC["SOURCE / BUILD SBOM"]
        SA["app deps from lockfiles
exact versions, intent,
full transitive closure"]
    end
    subgraph BIN["IMAGE / BINARY SBOM"]
        BA["everything physically present:
OS packages, base layer,
embedded manifests"]
    end

    SRC --> DIFF{"DIFF the
component sets"}
    BIN --> DIFF

    DIFF --> ONLYS["ONLY in source:
build-time / dev deps not shipped,
or a base image that stripped them
→ expected, or a packaging bug"]
    DIFF --> BOTH["IN BOTH:
corroborated — highest confidence"]
    DIFF --> ONLYB["ONLY in binary:
OS + base-layer packages
→ expected (source can't see them)"]
    DIFF --> GAP["NEITHER captured it:
static libs, vendored C, shaded JARs
→ the DANGEROUS blind spot,
invisible to BOTH vantages"]

    classDef good fill:#14532d,stroke:#4ade80,color:#fff;
    classDef info fill:#1e3a5f,stroke:#60a5fa,color:#fff;
    classDef bad fill:#7f1d1d,stroke:#f87171,color:#fff;
    class BOTH good;
    class ONLYS,ONLYB info;
    class GAP bad;
```

Read the four quadrants. Components **in both** are corroborated from independent vantages — your highest-confidence rows. Components **only in the source SBOM** are usually build-time or dev dependencies that never made it into the runtime image (expected, and a way to *shrink* your reported attack surface honestly) — but occasionally they signal a packaging bug where something you depend on was silently dropped. Components **only in the binary SBOM** are almost always OS and base-layer packages the source vantage structurally cannot see (expected). The lethal quadrant is the one the diagram cannot draw a box around cleanly: components **in neither** — statically linked libraries, vendored C

compiled straight into a binary, a shaded JAR whose original coordinates were rewritten. Neither vantage sees them, so the diff cannot surface them either. Comparison narrows uncertainty; it does not eliminate it. Your known blind spots remain blind to every observation you make with the same class of tool, which is why the last resort is not measurement but *documentation*: write down, per build system, which classes of component your pipeline is structurally unable to see, and carry that list into incident response (Book 8).

Systematic accuracy failures

Random errors average out at scale; systematic errors replicate. The accuracy problems that matter for a fleet are not the occasional mangled version string — they are the whole *classes* of component that a standard build pipeline gets wrong the same way every time, because a systematic gap in your paved road is a blind spot stamped identically onto every service that rides it. This section catalogs the classes. It builds on Chapter 4's hard cases and reframes them through the lens of *what the resulting SBOM claims versus what is true*.

Identification failures: present but invisible

These are components that physically exist in the artifact but carry no metadata a scanner can read, so they are either absent from the SBOM or listed under a wrong or meaningless identity.

- **Statically linked libraries.** A C/C++/Rust/Go binary that statically links `zlib` or `openssl` contains that code with no package database entry, no manifest, and — once stripped — often no symbol table. The binary *is* vulnerable to a `zlib` CVE; the SBOM shows one component (the binary) and `zlib` is simply gone. Go is the partial exception: `go version -m` reads the module list the toolchain embeds, so Go binaries self-describe. C static linking has no such convention, and this is the archetypal invisible-but-present case.
- **Vendored / copied code.** A project that copies a third-party source tree into its own repository (a `vendor/` directory of C, an inlined header, a pasted utility file) presents that code to the compiler as first-party source. No package manager knows it exists, so no source or binary scanner attributes it to its upstream. It carries the upstream's vulnerabilities under your name.

- **Shaded / relocated JARs.** Java "shading" (via the Maven Shade plugin) rewrites a dependency's package namespace — `org.apache.commons` becomes `com.myapp.shaded.org.apache.commons` — and merges it into an uber-JAR. The classes are present and vulnerable; the coordinates that would identify them are deliberately erased. This is how Log4Shell hid: a shaded Log4j inside a fat JAR matched no naive scan for `log4j-core`.
- **Minified / bundled JavaScript.** A webpack or esbuild bundle concatenates and minifies dozens of npm packages into one `main.min.js`. The original package boundaries and versions are gone; a scanner sees one file. The npm lockfile at build time is the only place the identities survive — which is exactly why Chapter 4 insisted on source-side generation for this ecosystem.
- **Stripped binaries and firmware.** Stripping removes symbols; firmware images pack code, data, and filesystems into opaque blobs with no package metadata at all. Binary composition analysis here degrades to fuzzy matching and heuristics — informative, but a guess, not an inventory. Do not let an SBOM present a heuristic firmware guess with the same confidence as a lockfile-derived component.

The common thread: **each of these makes a component present in the artifact but absent or misidentified in the SBOM**, and that is precisely the error class that produces a false-negative vulnerability result — the dangerous kind, discussed next.

Identifier problems: present but unmatchable

A component can be listed and still be useless downstream if its identifier is wrong, missing, or ambiguous. Book 2, Chapter 5 is the reference for the identifier model; here is what breaks in practice:

- **Missing or fallback purls.** A generator that cannot confidently determine a component's ecosystem emits `pkg:generic/something@1.0` or no purl at all. OSV and GHSA match on purl; no purl means no match.
- **Wrong purls.** A subtler failure: a purl with the wrong ecosystem, a normalized name that does not match the advisory's, or an epoch/qualifier mismatch. The component looks identified but silently matches nothing.

- **CPE ambiguity.** CPEs are human-assigned NVD strings with notorious vendor/product inconsistency (`openssl:openssl` versus `openssl_project:openssl`). A near-miss CPE fails to match NVD advisories that use the other form. Book 2, Chapter 5 covered why purl↔CPE translation is lossy.
- **Version mismatches.** `1.2.3` versus `1.2.3-r0` versus `1.2.3.el8` — distro-patched versions, epochs, and build suffixes routinely defeat naive version-range matching, causing both false negatives (patched-but-flagged) and, worse, false negatives where a vulnerable version reads as something the matcher does not recognize.
- **Empty supplier fields.** NTIA mandates a supplier, but many generators emit an empty or placeholder value. It passes some conformance checks weakly and provides no help disambiguating a component with a common name.

Why false negatives are the dangerous class

Distinguish the two error directions cleanly, because they have wildly asymmetric costs.

- A **false positive** — a component in the SBOM that is not really in the artifact, or a vulnerability flagged that does not apply — costs analyst time. It is noise. VEX (Chapter 6) exists largely to suppress it. It is annoying and it erodes trust in the tooling, but it does not hurt you in an incident.
- A **false negative** — a component physically present in the artifact but *absent from the SBOM* — is silent. When the next Log4Shell-class advisory lands and you run the query "which of my 4,000 services ship this component," the affected service with the invisible static library or shaded JAR returns *clean*. You will not patch it, because you do not know it is there. The SBOM gave you confidence and the confidence was false.

This asymmetry should govern how you invest in SBOM quality. A program obsessed with trimming false positives (nicer dashboards) while tolerating systematic false negatives (invisible component classes) is optimizing the cheap error and ignoring the expensive one. A false negative is a vulnerability query that lies to you at exactly the moment it matters most.

Format and tool divergence

Chapter 4 established that two competent tools produce materially different SBOMs for the same artifact — different catalogers, different identifier minting, different source-versus-binary vantage, different scoping of what counts as a component. That divergence is a *quality* problem too, not just a generation curiosity: it means "we have an SBOM" is underspecified until you say *which tool produced it at which pipeline stage*, and it undermines comparability across an organization where different teams picked different generators. If service A's SBOM comes from Syft-at-build and service B's from Trivy-at-scan, a fleet-wide query for a component executes against two populations with different systematic blind spots, and a clean result from B means something weaker than a clean result from A. Standardizing the generation path across the paved road is as much a *quality* decision as an operational one.

The transitive-closure boundary: SBOM-of-an-SBOM gaps

Even a tool that captures its ecosystem perfectly faces scoping questions with no format-mandated answer, and different answers produce SBOMs that are not wrong so much as *differently scoped* — which is its own comparability hazard:

- **Dev versus prod dependencies.** Does the SBOM include test frameworks and build tools, or only what ships? Both are defensible; a consumer comparing two SBOMs that made opposite choices will misread the difference as a completeness gap.
- **Multi-stage build losses.** A builder stage installs a compiler and dev headers; the final stage copies only the binary. An SBOM generated against the final image legitimately omits the builder's toolchain — but if you needed to know the compiler version for a build-integrity question, it is gone (Chapter 4).
- **Base-layer and OS components.** How deep does the SBOM go into the base image — every `apk` / `dpkg` package, or only what the application directly links? Chapter 4's source vantage cannot see these at all; a source-only SBOM is *scoped* to exclude them, which is fine only if the consumer knows that.
- **Dynamically loaded plugins.** Code loaded at runtime via `dlopen`, JVM classpath scanning, or a plugin directory may not be present at build time and thus not in a build SBOM. Runtime generation (Chapter 4) is the only vantage that sees it.

None of these has a universally correct answer. The point is that the *transitive closure boundary* — how deep, how wide, dev-inclusive or not — is a decision, and an SBOM that does not document its own scoping decision is one a consumer will silently misinterpret. This is another argument for NTIA's "known unknowns": an SBOM that states its boundaries is honest; one that implies a completeness it never attempted is not.

Fundamental limitations: what an SBOM cannot do

Everything above concerns making SBOMs *better*. This section concerns what a *perfect* SBOM — complete, accurate, correctly identified, fresh, signed — still cannot tell you. These are not quality defects; they are category boundaries. An SBOM is an inventory of components. It is not a security assessment, and treating it as one is the overclaim that discredits the whole practice when it inevitably fails to deliver.

```
flowchart LR
    subgraph CAN["WHAT AN SBOM TELLS YOU"]
        C1["what components are present"]
        C2["at what versions"]
        C3["in what dependency relationships"]
        C4["→ which KNOWN components map to
which KNOWN vulnerabilities"]
    end
    subgraph CANNOT["WHAT IT DOES NOT TELL YOU"]
        X1["whether you are EXPLOITABLE
(needs reachability + VEX)"]
        X2["whether a component is MALICIOUS
(a bad package is still a listed component)"]
        X3["whether you have FIRST-PARTY bugs
(your own code isn't a component)"]
        X4["whether the BUILD was honest
(needs provenance)"]
        X5["anything about ZERO-DAYS /
unknown vulnerabilities"]
    end

    CAN -.->|"inventory ≠ assessment"| CANNOT

    classDef can fill:#14532d,stroke:#4ade80,color:#fff;
    classDef cannot fill:#7f1d1d,stroke:#f87171,color:#fff;
    class C1,C2,C3,C4 can;
    class X1,X2,X3,X4,X5 cannot;
```

Walk each boundary, because each maps to a *different mechanism* that does the job the SBOM cannot.

An SBOM does not tell you whether you are exploitable. "Component X is present and X has CVE-2024-nnnnn" is a statement about presence, not exposure. The vulnerable function may never be called, the code path may be unreachable, the feature may be compiled out, or a compensating control may neutralize it. Determining exploitability needs **reachability analysis** (Book 2, Chapter 7) to establish whether the vulnerable code is actually invoked, and **VEX** (Chapter 6) to record and communicate the vendor's affected/not-affected determination. The SBOM is the input to that analysis, never its conclusion. An SBOM-only program drowns in "present but not exploitable" findings — which is precisely why VEX exists.

An SBOM does not tell you whether a component is malicious. This trips people up because it feels like it should. But a malicious package — a typosquat, a compromised maintainer's update, an `event-stream`-style payload (Book 2, Chapter 4) — *is a component*, and a faithful SBOM lists it, correctly, by name and version. The SBOM does its job perfectly and tells you nothing about the threat, because "is this component malicious?" is a question about the component's *behavior*, not its identity. Detecting malice needs the machinery of Book 2, Chapter 4 — behavioral analysis, install-script inspection, reputation signals — an entirely different discipline. An SBOM full of malware is a complete, accurate, high-quality SBOM.

An SBOM does not cover your first-party code. Your own application logic — the SQL injection you wrote, the auth bypass in your handler — is not a "component" and appears nowhere in the SBOM. First-party vulnerabilities are the domain of SAST, DAST, code review, and testing, wholly outside the SBOM's remit. An organization that believes "we have SBOMs, so we have visibility into our vulnerabilities" has confused third-party inventory with total security posture.

An SBOM does not tell you the build was honest. An SBOM describes *what is in* an artifact; it says nothing about *how the artifact came to be* or whether the SBOM itself was generated by an untampered pipeline. An attacker who compromises the build can produce a malicious binary *and* a clean-looking SBOM that omits their injection. Establishing build integrity is the job of **provenance** — SLSA attestations, in-toto, signing (Books 4 and 5) — which is why this chapter's quality dimensions

included *provenance* and why an unsigned SBOM of unknown origin is a weak artifact regardless of how complete its component list appears.

An SBOM tells you nothing about zero-days. The entire mechanism is *match known components to known vulnerabilities*. A vulnerability not yet in any database (OSV, NVD, GHSA — Book 2, Chapter 5) matches nothing, no matter how perfect your SBOM. This is not an SBOM weakness specifically; it is the same fundamental limit as all of Software Composition Analysis (Book 2, Chapter 6). The value of the SBOM is *latent and retroactive*: when the zero-day becomes a known day — when the advisory publishes — a good SBOM store lets you answer "am I affected" across the fleet in minutes instead of a week of frantic grepping. That retroactive query is the real product. The SBOM does not prevent the attack; it collapses your response time after disclosure.

The overclaim, critiqued fairly

"SBOMs will secure the software supply chain" is the sentence to retire. It is false in the way that matters and it sets the practice up to be judged a failure. An SBOM prevents no attack. It does not stop a malicious dependency from being installed, a build from being poisoned, or a zero-day from landing. What it does — and this is genuinely valuable, not a consolation prize — is provide **transparency and response capability**. It converts "we think we might use Log4j somewhere" into "these 37 services ship log4j-core 2.14.1, here are their digests, here are their owners," in the first hour of an incident rather than the second week. It is the difference between an incident response that is a database query and one that is an archaeological dig across thousands of repositories.

Hold both truths at once without cynicism. SBOMs are **necessary infrastructure and not a solution**. They are necessary because you cannot respond to a supply-chain incident you have no inventory for; the organizations that suffered most in Log4Shell were the ones who could not answer "where do we use this." They are not a solution because inventory is not assessment, presence is not exploitability, and known is not all. An engineer who internalizes this builds the SBOM as the *foundation layer* it is — feeding reachability, VEX, provenance, and malware detection — rather than as a finish line.

The table: limitation to remedy

The SBOM cannot tell you…	Because…	What you need instead	Reference
Whether you are exploitable	Presence ≠ reachability of vulnerable code	Reachability analysis + VEX	Book 2 Ch 7; Book 3 Ch 6
Whether a component is malicious	A malicious package is a correctly-listed component	Behavioral / package analysis	Book 2 Ch 4
Whether you have first-party bugs	Your own code is not a "component"	SAST / DAST / review / testing	—
Whether the build was honest	Inventory ≠ integrity of the process	Provenance: SLSA, in-toto, signing	Books 4 & 5
Anything about zero-days	Matching is known-component → known-vuln	Nothing prevents them; SBOM enables fast *retroactive* response	Book 2 Ch 6
Whether the SBOM itself is trustworthy	The SBOM is an attackable artifact	Signed attestation of the SBOM	Books 4 & 5; Ch 5

Getting to good-enough

The correct target is not a perfect SBOM. There is no perfect SBOM — completeness is unmeasurable and some component classes are structurally invisible. The correct target is **fit for purpose**: the quality bar is a function of what you will *do* with the document, and different uses stress different dimensions. Pursuing perfection wastes effort on dimensions your use case does not need while a program that never ships waits for an unachievable ideal.

Fitness for purpose

Match the quality investment to the job:

- **Rapid vulnerability response** (the marquee use case) needs **component + version coverage and valid identifiers** above all. If you will query "who ships X at version Y," you need those two fields correct and a valid purl on every component. Licenses and rich descriptions are irrelevant to this job; a missing purl is fatal to it. Optimize accuracy and identifier validity; accept that some deep base-layer completeness is a stretch goal.
- **License compliance** needs correct, SPDX-ID-valid **license fields** on every component, and cares far less about exact patch versions or reachability. A different dimension entirely leads.
- **Customer / regulatory delivery** (an SBOM you hand to a buyer under a contract or a CRA obligation, Book 8, Chapter 1) needs **NTIA minimum elements plus format conformance plus a signature** — the deliverable is judged on conformance and provenance, and a conformance-clean, signed document is the contractual product even if its completeness is imperfect (declare known unknowns).

The same underlying generation can serve all three, but the *quality gate* you enforce should weight the dimensions the consuming use case actually depends on. "Good enough" is defined by the query you will run, not by a universal score.

Improving quality, concretely

The levers, in rough order of impact, all trace back to earlier chapters:

1. **Generate at build time** (Chapter 4). The single biggest quality lever, because the build has the real resolved graph with exact versions and can see generated and build-only inputs. It moves whole component classes from "invisible" to "captured."
2. **Combine generation points.** Emit a build/source SBOM (intent, exact app deps) *and* an image/binary SBOM (coverage of OS and base layers), and reconcile them (the diff of the completeness section). One vantage's blind spot is often the other's strength.
3. **Enrich identifiers.** Post-process to add or correct purls and CPEs, fill supplier fields, and normalize versions — directly lifting the identifier-validity dimension that gates vulnerability matching.

4. **Validate and score in CI, and gate on it.** Fail the build when the SBOM drops below a threshold, so quality is enforced by the pipeline rather than hoped for.
5. **Sign the SBOM** (Books 4–5). Attach a `cosign` attestation keyed to the artifact digest so the provenance dimension is satisfied and the SBOM cannot be silently swapped.

The quality gate in CI

Make the quality bar an executable gate on the paved road, not a review guideline. The gate generates, validates, scores, and either passes the SBOM forward to storage (Chapter 5) or fails the build.

```
flowchart TD
    A["Build artifact
(keyed to digest)"] --> B["Generate SBOM
(build-time, Ch 4)"]
    B --> C{"Schema valid?
cyclonedx-cli / pyspdxtools"}
    C -->|no| FAIL["FAIL BUILD
broken SBOM = broken
downstream ingestion"]
    C -->|yes| D{"NTIA elements present?
ntia-conformance-checker"}
    D -->|no| FAIL
    D -->|yes| E{"Quality score ≥ threshold?
sbomqs"}
    E -->|below| FAIL
    E -->|meets| F["Enrich identifiers
(purls / CPEs / supplier)"]
    F --> G["Sign / attest
cosign attest (Books 4–5)"]
    G --> H["Push to SBOM store
keyed to digest (Ch 5)"]

    classDef fail fill:#7f1d1d,stroke:#f87171,color:#fff;
    classDef ok fill:#14532d,stroke:#4ade80,color:#fff;
    class FAIL fail;
    class H ok;
```

```
# The gate, as a CI step
syft "$IMAGE" -o cyclonedx-json > sbom.cdx.json

cyclonedx-cli validate --input-file sbom.cdx.json --input-version v1_6
\
  || { echo "SBOM schema invalid"; exit 1; }

SCORE=$(sbomqs score sbom.cdx.json --json | jq '.files[0].avg_score')
awk -v s="$SCORE" 'BEGIN { exit (s >= 7.0) ? 0 : 1 }' \
  || { echo "SBOM quality $SCORE below threshold 7.0"; exit 1; }

cosign attest --predicate sbom.cdx.json --type cyclonedx
"$IMAGE_DIGEST"
```

Set the threshold deliberately and raise it over time. Starting at a gate that fails on schema-invalid and NTIA-non-conformant SBOMs, then ratcheting the sbomqs threshold up as the fleet's tooling improves, is a workable adoption path — the same ratchet strategy Book 8, Chapter 4 (Policy as Code) applies to any org-wide control. The gate's honesty depends on remembering what it *cannot* check: it enforces every dimension except completeness, so a build can pass the gate cleanly and still ship an SBOM missing a class of static libraries. The gate raises the floor on the measurable dimensions; it does not close the blind spots.

Honest metrics

The final lever is reporting. Track SBOM quality as a **distribution across the fleet**, not a binary have/have-not, and feed it to the metrics program of Book 8, Chapter 8. "We have SBOMs for 100% of services" is a coverage number, and reporting it as if it meant "we have 100% visibility" is the executive-facing version of the conformance/completeness confusion this whole chapter attacks. The honest dashboard shows the *distribution* of quality scores — the p50 and the long tail of low-scoring services — plus explicit callouts of the known structural blind spots ("static C libraries are not captured by our standard pipeline"). A program that reports "100% coverage, median sbomqs 8.1, but our Go-CGO and firmware builds have documented static-linking gaps" is telling the truth. A program that reports "100% SBOM coverage" and stops is manufacturing exactly the false confidence that gets an organization blindsided in the next incident.

Distributed-systems lens

At fleet scale the danger is not the random error — it is the **systematic** one. A single SBOM that mislabels one component is a local annoyance. A *standard build pipeline* that mislabels or omits an entire class of component — every statically linked library, every shaded JAR, every bundled front-end — stamps that identical blind spot onto every one of the thousands of services that inherit the pipeline. The error does not average out; it *replicates*. When the advisory for that component class lands, the fleet-wide query returns clean across the board, and the cleanliness is uniform *because the blindness is uniform*. The paved road that gives you consistency also gives you consistent blind spots, and consistent blind spots are the ones that hurt at scale.

This reframes every recommendation above as a fleet decision:

- **Quality-gate in the paved-road pipeline, once.** The CI gate is not a per-team practice to be adopted; it is a property of the shared build platform (Book 4, Chapter 10) that every tenant inherits by default. Implement the generate-validate-score-sign-store flow once, in the platform, and every service gets a gated, signed, digest-keyed SBOM for free — the same amortization argument Chapter 4 made for generation itself, now extended to quality. Consistency of *tooling* also fixes the format-divergence problem: a fleet where every SBOM came from the same generator at the same stage is a fleet where a clean query result means the same thing everywhere.
- **Measure the quality *distribution*, not a coverage checkbox.** Emit each service's sbomqs score, format, generation stage, and known-blind-spot flags as a metric into the store (Chapter 5) and report the distribution. The interesting number is never the mean; it is the shape of the tail — the services on the old pipeline, the ones with `pkg:generic` fallbacks, the CGO binaries. That tail is your real exposure map.
- **Enumerate the blind spots explicitly, and hand them to incident response.** Because you cannot measure completeness, do the next best thing: maintain a written, per-pipeline catalog of *what your SBOMs structurally cannot see*. When Book 8's incident-response process runs a fleet query, it must account for "the SBOM will not show static C libraries, so for a `zlib`-class advisory, the SBOM query is necessary but not sufficient — also check these CGO-

heavy services by hand." A fleet that knows its own blind spots can compensate for them in an incident. A fleet that believes its SBOMs are complete will trust a query that lies to it. The most valuable thing you can know about your SBOMs, at scale, is exactly the list of things they will not tell you.

The distributed-systems payoff of high-quality SBOMs is real and worth the investment: it is the collapse of incident response from an archaeological dig into a database query, executed uniformly across the whole estate. But that payoff is only as trustworthy as your honesty about the query's blind spots. Build the SBOM as excellent, gated, signed foundation infrastructure — and treat it as exactly that: a foundation the rest of the program is built on, never the finished building.

Key takeaways

- **"Quality" is seven independent properties**, not one number: completeness, accuracy, identifier validity, required-field presence, graph fidelity, freshness, and provenance. An SBOM can excel on some and fail others; the axes do not correlate.
- **Conformance, completeness, and accuracy are different things.** Conformance is a property of the document (are the fields present?); completeness is a property of the document-to-artifact relationship (are all real components listed?); accuracy is correspondence (are the listed components correct?). An SBOM can be 100% NTIA-conformant and badly incomplete simultaneously, and every validator will still call it conformant.
- **The tools validate what is present, never what is missing.** `cyclonedx-cli validate`, ntia-conformance-checker, sbomqs, and eBay's sbom-scorecard all score the rows that exist. Their scores rise as listed components improve and cannot fall when a whole class is absent. None can measure completeness, because completeness needs an oracle that would *be* the SBOM.
- **Completeness is approached only by comparison** — tool-diffing and, more powerfully, source-versus-binary diffing — and even that is defeated by *shared blindness*: components neither vantage can see (static libs, vendored C, shaded JARs) survive every diff. What cannot be measured must be *documented* as an explicit blind spot.

- **False negatives are the dangerous error class.** A component present in the artifact but absent from the SBOM (invisible static library, shaded JAR, minified bundle) returns *clean* on the incident query that matters most. Optimize against false negatives, not just the cheap false positives that VEX already handles.
- **A perfect SBOM is still only an inventory.** It cannot tell you whether you are exploitable (needs reachability + VEX), whether a component is malicious (a bad package is a correctly-listed component), whether you have first-party bugs, whether the build was honest (needs provenance), or anything about zero-days. Each limit maps to a different mechanism in another book.
- **"SBOMs will secure the supply chain" is an overclaim to retire.** SBOMs prevent no attack; they deliver transparency and fast *retroactive* response — turning a supply-chain incident from an archaeological dig into a database query. Necessary infrastructure, not a solution.
- **Pursue fit-for-purpose, not perfect.** Rapid vuln response needs component+version coverage and valid purls; license compliance needs license fields; customer delivery needs NTIA + conformance + a signature. Weight the quality gate toward the dimensions your actual use case queries.
- **Enforce quality as a CI gate on the paved road**: generate at build time, combine vantages, enrich identifiers, validate + score + fail below threshold, sign and store keyed to digest. Implement once in the shared platform; every service inherits it.
- **At fleet scale the systematic gap is the killer.** A blind spot baked into the standard pipeline replicates identically across every service. Quality-gate once in the platform, measure the quality *distribution* (watch the tail), and hand your explicit, per-pipeline blind-spot catalog to incident response so IR accounts for what the SBOM will not show.
- **Report honestly.** "100% SBOM coverage" is a coverage metric, not a visibility metric. Report the score distribution and the known structural gaps; never let a coverage number masquerade as completeness.

Further reading

- **NTIA**, "The Minimum Elements for a Software Bill of Materials (SBOM)," 12 July 2021 — the field-level floor and the crucial concept of *known unknowns* (declaring the parts of the tree an SBOM does not enumerate).
- **Interlynk sbomqs** (`github.com/interlynk-io/sbomqs`) — open-source SBOM quality scorer; read the category/feature definitions to see exactly which per-component properties are scored and, by omission, which cannot be.
- **SPDX ntia-conformance-checker** (`github.com/spdx/ntia-conformance-checker`) — checks an SPDX document against the NTIA minimum elements; the canonical example of conformance checking that is structurally blind to completeness.
- **eBay sbom-scorecard** (`github.com/eBay/sbom-scorecard`) — a leaner quality rubric; useful precisely for comparing against sbomqs to see how much a "score" depends on the rubric.
- **OWASP Software Component Verification Standard (SCVS)** and its **BOM Maturity Model** — a vendor-neutral rubric for what SBOM practices at each maturity level look like.
- **CycloneDX** `cyclonedx-cli` and **SPDX** `spdx-tools` / SPDX Online Tool — schema validators; remember that schema validity is table stakes, not quality.
- **BSI TR-03183-2** (German Federal Office for Information Security) — a technical guideline specifying SBOM content and quality requirements, now a scored profile in sbomqs; a good example of a regulator formalizing quality expectations.
- **Package URL (purl)** specification and **NIST CPE 2.3** — the identifier standards whose correctness gates all downstream vulnerability matching (Book 2, Chapter 5).

- **CISA SBOM community resources** — the working-group outputs on SBOM types, sharing, and quality; useful for the evolving practice-and-process side of the NTIA elements.

Chapter 8 — SBOMs for Services: Containers, Serverless, and SaaS

What this chapter covers. Every chapter before this one carried a hidden assumption, inherited from the origin of the whole idea: an SBOM describes a **shippable artifact**. NTIA's minimum elements, the SPDX and CycloneDX document models (Chapters 2 and 3), the generation pipeline (Chapter 4), even the distribution and query machinery (Chapter 5) — all of it is framed around a thing you build, name, sign, and hand to someone. That framing fits a firmware image, a downloaded installer, a library published to a registry. It fits almost nothing that a distributed-backend organization actually operates. You do not ship a binary to a customer. You **run a fleet of services** — containers scheduled across a cluster, functions invoked in someone else's execution environment, request paths that fan out to a payment API, an identity provider, and three managed datastores. The "software" you are responsible for is not a file. It is a running system whose boundary is genuinely fuzzy, and whose composition changes every time a base image is rebuilt or a scheduler moves a pod. This chapter adapts the SBOM to that reality. It is, for this suite's audience, the most directly load-bearing chapter in the book: it answers the question "what is an SBOM *for us*, given that we run services and don't ship products?"

Learning goals — after this chapter you should be able to:

- Explain **why a shipped-artifact SBOM under-describes a running service**, and decompose a service's true bill of materials into the layers that actually execute (app, language deps, OS packages, base image, injected sidecars/agents, platform runtime) plus the external services it calls.
- Treat the **container image as the unit of SBOM** for a backend org — generate per-image SBOMs with syft/Trivy, understand base-image inheritance and multi-stage builds, attach SBOMs via OCI referrers keyed to the image digest, and reason about the distroless trade-off.

- Map the **shared-responsibility boundary for serverless**: SBOM your function code and its deps, understand Lambda layers as a dependency and supply-chain mechanism, and acknowledge the irreducible gap where the provider's managed runtime is opaque.
- Model **service and SaaS dependencies** with the CycloneDX SaaSBOM `services` array — external endpoints, data flows, trust boundaries, data classification — and understand why the runtime supply chain (polyfill.io, Ledger Connect Kit) is a real class of compromise that component SBOMs miss entirely.
- Join **build-time SBOMs to the orchestrator's deployment truth** to produce a runtime fleet inventory — "what is actually running where," the query that answers Log4Shell for a service org.
- Assemble the three pieces — per-service container SBOMs, the service-dependency graph, and the runtime deployment inventory — into a **complete service-fleet BoM**, and know which parts are authoritative, which are best-effort, and which are permanently the provider's.

A boundary note. This chapter assumes the generation mechanics of Chapter 4, the referrers/attach machinery and the deploy-join idea from Chapter 5, the CycloneDX model from Chapter 3, and VEX from Chapter 6. It leans on Book 6 (Cloud-Native) for base-image and distroless detail (Book 6, Chapters 3 and 4) and on Book 1, Chapter 9 for the runtime supply-chain framing. It hands fleet-level metrics to Book 8, Chapter 8, and provenance/signing to Books 4 and 5.

The mismatch: you SBOM artifacts, but you run services

Start with the thing the classic model gets right, because the failure is instructive. A shipped binary has a **crisp boundary**. When a vendor ships `nginx-1.25.3.tar.gz` or an appliance firmware image, "what is in this" has an answer that does not change after the artifact leaves the build: the bytes are fixed, the components are whatever got compiled or bundled in, and the SBOM is a faithful manifest of those bytes. The document and the artifact are the same age forever. That is the property every SBOM standard was designed around — a static artifact with a stable digest and a stable content.

A running service has none of that crispness. Ask "what is in the `checkout` service" and you have to decide what *in* means, because at least seven distinct things run when a `checkout` request is served, and they belong to different owners with different lifecycles:

```
flowchart TD
    subgraph RUN["What actually runs when 'checkout' serves a request"]
        APP["Your application code
(first-party)"]
        LANG["Language dependencies
(npm / pip / Go modules / Maven)"]
        OS["OS packages in the container
(apk / dpkg / rpm)"]
        BASE["Base image
(distro or distroless — most components live here)"]
        SIDE["Injected sidecars / agents
(service mesh proxy, log shipper, APM agent)"]
        PLAT["Runtime platform
(kernel, container runtime, node OS, CNI)"]
    end
    APP --> LANG --> OS --> BASE
    SIDE -. "injected at deploy, not in your Dockerfile" .-> RUN
    PLAT -. "the cluster you don't build" .-> RUN
    EXT["External services it CALLS
(Auth0, Stripe, RDS, Kafka, LaunchDarkly, Datadog)"]
    RUN ==> EXT

    classDef yours fill:#14532d,stroke:#4ade80,color:#fff;
    classDef shared fill:#78350f,stroke:#fbbf24,color:#fff;
    classDef theirs fill:#7f1d1d,stroke:#f87171,color:#fff;
    class APP,LANG yours;
    class OS,BASE,SIDE shared;
    class PLAT,EXT theirs;
```

Three of these layers are things you authored or chose (app, language deps, and the OS/base image you selected). Two arrive at deploy time without appearing in your Dockerfile at all — a service mesh sidecar injected by a mutating admission webhook, an APM or log-shipping agent added by the platform team. One is the substrate you rent (the node kernel, the container runtime, the CNI). And one is not "in" the service in any byte sense at all but is unquestionably part of its supply chain: the **external services it calls**. A compromise or outage in Stripe, Auth0, or your managed Kafka degrades or breaks `checkout` exactly as surely as a bug in its own code. The boundary of "what is in this service" is not fuzzy because we are being sloppy; it is fuzzy because a service is a

composition that only fully exists at runtime, assembled from parts with different owners.

That fuzziness is not an excuse to give up; it is a requirement to be precise about *purpose*, because two different jobs need two different slices of the composition:

- **For vulnerability response**, you need every layer that *executes bytes* — app, language deps, OS packages, base image, sidecars, and ideally the platform. When Log4Shell drops, "does anything we run contain a vulnerable `log4j-core`" is a question about the union of all executing layers, including a sidecar's bundled JVM that no application team put there. This is a **component** question, and it is answered by container SBOMs joined to a deploy inventory.
- **For architecture and risk**, you need the **service dependencies** — what this service calls, across what trust boundary, carrying what data. When your identity provider has an incident, "what breaks and what data is exposed" is a question about the *edges* of the service graph, not the contents of any container. This is a **topology** question, and no component SBOM answers it. It needs a SaaSBOM.

The rest of the chapter builds both, plus the runtime join that makes either one true rather than aspirational. Keep the two purposes distinct; conflating them is how organizations end up with a pile of container SBOMs and still cannot answer "if Auth0 is breached, what is exposed?"

Deployment model	What you must inventory	Who owns the rest	Authoritative identity anchor
Container / pod	App + language deps + OS packages + base image; sidecars separately; platform via cluster inventory	Platform team owns node OS/ runtime; base-image team owns the base	Image **digest** (`sha256:…`)
Serverless (zip/package)	Your handler code + bundled deps + Lambda layers you attach	**Provider** owns the managed runtime, OS, and execution environment (opaque)	Function version + package hash

Deployment model	What you must inventory	Who owns the rest	Authoritative identity anchor
Serverless (container image)	Same as container (you supply the whole image)	Provider owns only the host/ microVM below your image	Image digest
SaaS-composed service	The above *plus* every external service/API it calls	Each SaaS vendor owns its own internals	Service identity + endpoint + the vendor's own attestations

Container SBOMs: the workhorse

For the overwhelming majority of a backend fleet, the unit of SBOM is the **container image**, and its identity anchor is the **image digest** — the content-addressed `sha256:…` that names an exact, immutable set of bytes. Tags lie (`:latest` , `:v2.3` , and even `:v2.3.1` can all be re-pushed to point at new content); digests do not. Every SBOM you generate, sign, and store must be keyed to a digest, because the entire query architecture of Chapter 5 — "which running images contain X" — degrades to guesswork the moment identity is a mutable tag.

Layered composition and the base-image inheritance problem

An OCI image is a stack of content-addressed layers plus a config. Your Dockerfile's `FROM` line pulls in a base image — itself one or more layers — and your `RUN` , `COPY` , and `ADD` instructions add layers on top. The consequence that dominates container SBOMs: **most components come from the base, not from you.** A Debian-based image carries a couple of hundred `dpkg` packages before your application contributes a single dependency; a Node or Python base adds a language runtime and its standard toolchain. Empirically, for a typical microservice the base layers account for the large majority of SBOM entries, and your application layer contributes a comparatively small tail. (Book 6, Chapter 3 quantifies this and the distroless response.)

```
flowchart BT
    subgraph IMG["Final image (one digest) — its SBOM is the UNION of
all layers"]
        L0["Base image layers
OS + language runtime
~200+ OS packages (apk/dpkg/rpm)"]
        L1["Dependency layer
npm / pip / Maven install
your language deps"]
        L2["App layer
COPY your compiled code / bundle
first-party"]
    end
    L0 --> L1 --> L2
    BASE_SBOM["Base-image SBOM
(generated ONCE by the base-image team,
inherited by every child)"] -. describes .-> L0
    APP_SBOM["Per-build delta
(what YOUR build added)"] -. describes .-> L1
    APP_SBOM -. describes .-> L2

    classDef base fill:#1e3a5f,stroke:#60a5fa,color:#fff;
    classDef app fill:#14532d,stroke:#4ade80,color:#fff;
    class L0 base;
    class L1,L2 app;
```

This has a direct operational implication for how you generate and store SBOMs. There are two strategies, and mature organizations run both:

- **Per-final-image SBOM.** Scan the assembled image; the SBOM is the union of everything in every layer. This is what a scanner deployed in CI or against a registry produces by default, and it is what a vuln-response query needs, because at incident time you care about the *whole running image*, not who contributed which package.
- **Per-layer / base-inheritance model.** Generate the base image's SBOM **once**, when the base is built, and treat every child image as *base SBOM + build delta*. This is the same "generate shared components once and inherit" pattern Chapter 5 argues for at fleet scale, applied to the layer structure. It means a base-image CVE is analyzed against one authoritative base SBOM and the blast radius is computed by "which images are `FROM` this base," rather than re-deriving the base's contents from every one of the thousand images built on top of it. The base-image team becomes a supplier that ships an SBOM with its base, exactly as an external vendor would.

The base-inheritance model is not merely an optimization; it is a correctness and ownership win. When the base carries a vulnerable OpenSSL, the fix belongs to the base-image team, and the inheritance graph tells you precisely which child images inherit the fix once the base is rebumped. Without it, every application team re-discovers the same base CVE independently and you have no single place to assert "the base is patched as of this digest."

Multi-stage builds: what actually ends up in the final image

Multi-stage builds complicate SBOM generation in a way that is easy to get wrong. A common pattern:

```
# ---- build stage: heavy, full of toolchain and dev deps ----
FROM golang:1.22 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /out/checkout ./cmd/checkout

# ---- final stage: tiny, only the binary ----
FROM gcr.io/distroless/static-debian12
COPY --from=build /out/checkout /checkout
ENTRYPOINT ["/checkout"]
```

The build stage contains the entire Go toolchain, the module cache, and every transitive build-time dependency. **None of it ships.** Only `/checkout` is copied into the final stage. If you generate the SBOM by scanning the build stage — or by scanning the source tree — you will produce an SBOM full of components that are not in the artifact anyone runs, which is a **false-positive** problem: you will chase CVEs in a compiler that never reaches production. Conversely, and more dangerously, scanning only the final distroless stage with a package-database scanner may find *nothing*, because a statically linked Go binary carries its dependencies compiled in, with no `dpkg` database to read (Chapter 4's static-linking problem, and Chapter 7's false-negative class).

The correct posture is: **generate the SBOM against the final image**, and use a generator that understands both package databases *and* language-native evidence — for Go, that means reading the build metadata embedded in the binary (`go version -m`), which syft and Trivy both do. The SBOM-per-final-image must reflect the bytes that run, and

multi-stage builds mean the bytes that run are a deliberate subset of the bytes that were built. This is one of the clearest cases where "SBOM the artifact, not the build" is not a stylistic preference but a matter of getting the component set right.

Generating: syft/Trivy on the image, keyed to the digest

Mechanically, container SBOM generation is a solved problem, and Chapter 4 covered the internals. The service-specific points are about *what to run it against* and *what identity to stamp on the result*:

```
# Resolve the tag to an immutable digest FIRST — identity is the
digest, not the tag.
DIGEST=$(crane digest registry.example.com/checkout:v2.3.1)
IMAGE="registry.example.com/checkout@${DIGEST}"

# syft reads apk/dpkg/rpm databases AND language manifests/binaries in
the image.
syft "${IMAGE}" -o cyclonedx-json > checkout.cdx.json

# Trivy does the same and can emit SPDX or CycloneDX; it also drives
vuln scanning.
trivy image --format cyclonedx --output checkout.cdx.json "${IMAGE}"
```

Both tools catalog the two families that matter for a service: **OS packages** (by parsing the `/lib/apk/db`, `/var/lib/dpkg/status`, or rpm database inside the image) and **language dependencies** (by parsing `package-lock.json`, `requirements.txt`/installed `*.dist-info`, `go.mod`/binary build info, `pom.xml`/JAR metadata, and so on). The union is the executable content of that image. What neither tool sees is anything with no package manager and no readable metadata — a binary `COPY`ed in from a URL, a vendored C library statically linked with symbols stripped — and that gap is the subject of Chapter 7; here it is enough to know it exists and that it is why "generated an SBOM" is not the same as "the SBOM is complete."

Attaching the SBOM: OCI referrers and the digest anchor

Having generated the SBOM, you attach it to the image so that "give me the SBOM for the thing running in prod" is a lookup keyed by the exact digest that is running. The mechanism is the OCI **referrers** model (Chapter 5, and Book 6): the SBOM is pushed as its own OCI artifact whose manifest carries a `subject` field pointing at the image's digest,

discoverable through the Referrers API (`/v2/<name>/referrers/<digest>`). In practice you attach it as a signed in-toto attestation:

```
# Attach as a signed attestation whose SUBJECT is the image digest
(keyless, Fulcio/Rekor).
cosign attest --yes \
  --predicate checkout.cdx.json \
  --type cyclonedx \
  "${IMAGE}"

# Anyone (a Kyverno policy, an IR responder, GUAC's collector) can
later discover and verify it:
cosign verify-attestation --type cyclonedx \
  --certificate-identity-regexp '.*' \
  --certificate-oidc-issuer https://
token.actions.githubusercontent.com \
  "${IMAGE}"
```

Because the subject is the digest, the SBOM travels with the image across registries and cannot be silently detached or swapped without breaking verification. This is what makes the Chapter 5 query "pull the SBOM for `sha256:…`" a reliable primitive rather than a hope that some sidecar database still has a row for that tag. The `cosign attach sbom` command that predated this is deprecated precisely because it produced an unsigned association; use `cosign attest` and the referrers model.

Distroless and minimal images: the SBOM trade-off

Distroless and other minimal base images (Book 6, Chapter 3) strip the image to the application plus its direct runtime dependencies — no shell, no package manager, often no libc beyond what is needed. The security case is strong and well known: fewer components mean a smaller attack surface and fewer things to patch, and the SBOM is correspondingly smaller and easier to reason about. Two hundred `dpkg` packages become a handful.

But there is a real trade-off for SBOM *generation*, and it is worth stating plainly because it is counterintuitive. Package-database scanners work by reading package databases. A distroless image has **no `dpkg` / `apk` / `rpm` database** to read, because the tooling that would have written one was never installed. So a naive scan of a distroless image can report *fewer* components than are actually present — not because the image is cleaner, but because the evidence a scanner relies on was removed along with the package manager. The application's own language dependencies

are usually still discoverable (a Go binary's build info, a Python `dist-info` directory, a JAR's manifest survive), but any C libraries that *were* installed via the distro and then had their metadata stripped become invisible. The net effect is genuinely a security *improvement* (there is less there, and less to exploit) coupled with a genuine *observability* cost (the SBOM tool has fewer authoritative sources, so its completeness for the residual OS layer is lower). The right response is to generate the SBOM of the distroless base **at the point it is built**, where the package metadata still exists, and carry that base SBOM forward via inheritance — rather than trying to reconstruct it from the stripped final image. This is the base-inheritance model earning its keep again.

Serverless and FaaS: the shared-responsibility boundary

Serverless breaks the container model's most useful property: **you no longer possess the whole artifact.** With a container you supply every byte from the base image up, so an image SBOM can in principle be complete. With AWS Lambda's zip/package deployment, Google Cloud Functions, or Azure Functions, you supply your handler code and its bundled dependencies, and the provider supplies the **managed runtime beneath it** — a base OS, the language runtime, and an execution environment (microVM, sandbox) that you do not build, cannot inspect, and whose exact contents change without your involvement. The bill of materials of a running Lambda is genuinely split across an ownership boundary, and no tool you run can cross it.

```
flowchart TD
    subgraph YOURS["YOUR responsibility — you CAN SBOM this"]
        CODE["Function handler code
(first-party)"]
        DEPS["Bundled dependencies
(node_modules / site-packages / vendored)"]
        LAYERS["Lambda layers you attach
(shared deps, extensions)"]
    end
    subgraph PROV["PROVIDER responsibility — OPAQUE, rely on
attestations"]
        RT["Managed language runtime
(e.g. Amazon Linux + nodejs/python runtime)"]
        OS["Base OS / system libraries"]
        EXEC["Execution environment
(Firecracker microVM, sandbox, orchestration)"]
    end
    CODE --> DEPS --> LAYERS
    LAYERS ==>|"deployed onto"| RT
    RT --> OS --> EXEC

    classDef yours fill:#14532d,stroke:#4ade80,color:#fff;
    classDef theirs fill:#7f1d1d,stroke:#f87171,color:#fff;
    class CODE,DEPS,LAYERS yours;
    class RT,OS,EXEC theirs;
```

What you can SBOM, and how

Your side of the line is tractable and you should treat it exactly like any other language-dependency SBOM (Chapter 4). The deployment package is a zip (or an OCI image, discussed below); its contents are your code plus a `node_modules`, `site-packages`, vendored modules, or a compiled artifact. Point a generator at the package or the build tree with the lockfile present:

```
# Build the deployment bundle, then SBOM exactly what will be zipped
and uploaded.
npm ci --omit=dev
zip -r function.zip index.js node_modules/

syft dir:. -o cyclonedx-json > function.cdx.json    # reads package-
lock.json + node_modules
# or, from the lockfile alone for the pre-bundle dependency view:
trivy fs --format cyclonedx --output function.cdx.json .
```

The identity anchor here is weaker than a container digest — a function *version* (Lambda publishes immutable numbered versions) plus the

SHA-256 of the deployment package, which AWS exposes as `CodeSha256`. Key your SBOM to that, and you can answer "which deployed function versions contain this dependency" for your own code.

Lambda layers: a dependency mechanism and a supply-chain surface

Lambda **layers** are a first-class dependency mechanism that deserves specific attention because they are both convenient and a supply-chain risk. A layer is a zip of libraries or a runtime extension that is merged into the function's filesystem (under `/opt`) at invocation. Teams use them to share common dependencies across many functions, to add binary tooling, or to inject a vendor's observability/security **extension** that runs alongside your handler in the same execution environment. That last case is the sharp edge: an extension layer is third-party code, often supplied as a public layer ARN, that executes in your function's context with access to its environment (including secrets injected as environment variables) and its network egress. It is, architecturally, the serverless analogue of an injected sidecar — a component that is part of your running service but was authored and controlled by someone else.

For SBOM purposes, layers must be inventoried **as their own components with their own versions**, not folded silently into the function. A layer is referenced by a versioned ARN (`arn:aws:lambda:…:layer:my-layer:7`); the version is immutable, so it is a usable identity anchor. Your function's effective SBOM is the union of its package SBOM and the SBOMs of every attached layer version — and if a layer is a vendor's extension, you are trusting the vendor's SBOM (or lack of one) exactly as you trust a base image's. Treat public/third-party layers with the same scrutiny as any external dependency: pin the version, know what is in it, and know that it runs with your function's privileges.

Container-image-based Lambda: back to the container model

AWS also lets you deploy a Lambda as an **OCI container image** (up to 10 GB) rather than a zip. When you do, the SBOM story collapses back to the container case: you supply the whole image, so you can generate a complete image SBOM with syft/Trivy, key it to the image digest, and attach it via referrers, precisely as in the previous section. The provider's responsibility shrinks to the microVM host beneath your image. If SBOM completeness matters and your function is non-trivial, container-image

packaging is the option that restores your ability to inventory the full executable content — a real reason to prefer it over zip packaging in a supply-chain-conscious org.

The irreducible visibility gap

Here is the honest conclusion that the shared-responsibility model forces, and it is different in kind from the "our tooling is imperfect" gaps of earlier chapters: **for zip/package serverless you cannot produce a complete SBOM of the running service, because part of the running service is the provider's and is not exposed to you.** The managed Amazon Linux base, the language runtime build, the patch level of the system libraries under your handler — these are real executing components with real CVEs, and you have no scanner vantage point from which to enumerate them. This is not a bug to be fixed with a better tool; it is a property of the deployment model.

The correct response is twofold. First, **document the boundary explicitly** in the service's SBOM record: state which components are yours and inventoried, and mark the managed runtime as a provider-owned component you have declared but not enumerated (CycloneDX lets you represent an external component and annotate its provenance). Second, **rely on provider attestations** for the other side: AWS patches and rebuilds managed runtimes and publishes runtime deprecation and update information; providers increasingly publish their own security attestations and, over time, SBOMs for managed runtimes. Your posture is the same shared-responsibility posture you already accept for the node kernel under a container: you own your layers, you hold the provider accountable for theirs via their attestations and SLAs, and you do not pretend a gap is filled when it is not. Chapter 7's rule that false confidence is the failure mode to prevent applies with full force here.

Model	SBOM coverage you can produce	Irreducible gap	How the gap is covered
Container image	App + deps + OS + base (whole image)	Node kernel, container runtime, CNI	Platform team's node/base attestations; cluster inventory

Model	SBOM coverage you can produce	Irreducible gap	How the gap is covered
Distroless container	App + deps; base via inheritance SBOM	Stripped OS libs with no metadata	Base SBOM generated at base build time
Lambda (zip)	Handler + bundled deps + layers	**Entire managed runtime + OS + exec env**	Provider attestations; document boundary
Lambda (container)	Whole image (as container)	MicroVM host only	Provider attestation for host
SaaS-composed	Your own components only	The vendor's entire internals	Vendor SBOM/ attestation; SaaSBOM declares the edge

SaaS and service dependencies: the SaaSBOM frontier

Everything so far inventories *bytes that run inside your boundary*. The distributed-systems reality that classic SBOMs miss entirely is that a microservice's real dependency set includes the **other services and SaaS APIs it calls** — and those are supply-chain dependencies in the strict sense that a compromise, malicious change, or outage in them affects you, often without any change to your own code. Your `checkout` service depends on Stripe for payments, Auth0 for token verification, an RDS Postgres for state, a managed Kafka for events, LaunchDarkly for feature flags, and Datadog for telemetry. None of these appears in any container SBOM. All of them are ways your service can be harmed by something you did not build.

The runtime supply chain is a real attack class

This is not theoretical. Two real incidents make the category concrete, and both are exactly the kind of thing a component SBOM cannot see because the malicious code was never in any artifact you built or scanned — it was served, at runtime, by a third party you depended on.

- **Ledger Connect Kit (December 2023).** Ledger's `@ledgerhq/connect-kit`, a JavaScript library that many decentralized-application

front-ends loaded to connect hardware wallets, was compromised after a former employee's npm credentials were phished. A malicious version was published and, via the CDN that served the library, distributed to every site that pulled it — injecting a wallet-draining script into applications whose own code had not changed. The dependency was a *service/CDN-delivered component*, and the compromise propagated through the delivery channel, not through anyone's build. Reported losses were in the hundreds of thousands of dollars before the malicious version was pulled.

- **polyfill.io (June 2024).** The `polyfill.io` service delivered JavaScript polyfills to a very large number of sites via a `<script src="https://cdn.polyfill.io/…">` include. After the domain changed hands, the service began serving **malicious code** to a subset of requests — conditionally, targeting mobile users and evading detection — redirecting them to scam/malware sites. Every site that trusted the runtime `<script>` include was affected the moment the upstream service turned hostile, with no change to the sites' own repositories. Estimates put the number of affected sites in the range of a hundred thousand or more; major CDNs responded by blocking or rewriting the domain.

The common structure is what matters here: a dependency that lives *outside your artifact*, delivered or invoked *at runtime*, whose integrity you do not control and whose turning-malicious is invisible to every scan you run against your own bytes. Book 1, Chapter 9 develops this "runtime supply chain" framing in full; the SBOM consequence is direct. If your bill of materials only lists what is inside your containers, it structurally cannot represent the class of risk that polyfill.io and Ledger Connect Kit exemplify. You need a model whose first-class objects are *services and the edges to them*.

The CycloneDX SaaSBOM: modeling services, not just components

CycloneDX (Chapter 3) is the standard that has this model. Alongside `components`, a CycloneDX document carries a `services` array — the **SaaSBOM** — for declaring external services your system depends on, each with its endpoints, the direction and classification of data that flows across the boundary, whether the connection crosses a trust boundary, and whether the service is authenticated. This is the vocabulary for

saying "service A depends on service B and on Stripe and on Auth0" in a machine-readable, queryable way.

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.6",
  "metadata": {
    "component": { "type": "application", "name": "checkout",
"version": "2.3.1" }
  },
  "services": [
    {
      "bom-ref": "svc-stripe",
      "provider": { "name": "Stripe, Inc." },
      "name": "Stripe Payments API",
      "endpoints": ["https://api.stripe.com/v1/payment_intents"],
      "authenticated": true,
      "x-trust-boundary": true,
      "trustZone": "external-third-party",
      "data": [
        { "flow": "outbound", "classification": "PCI-cardholder" },
        { "flow": "inbound",  "classification": "payment-token" }
      ]
    },
    {
      "bom-ref": "svc-auth0",
      "provider": { "name": "Okta / Auth0" },
      "name": "Auth0 token verification",
      "endpoints": ["https://tenant.us.auth0.com/.well-known/
jwks.json"],
      "authenticated": true,
      "x-trust-boundary": true,
      "data": [{ "flow": "inbound", "classification": "signing-keys" }]
    },
    {
      "bom-ref": "svc-orders",
      "name": "orders (internal service)",
      "endpoints": ["https://orders.internal.svc.cluster.local/grpc"],
      "authenticated": true,
      "x-trust-boundary": false,
      "trustZone": "internal-mesh",
      "data": [{ "flow": "bi-directional", "classification": "PII-
order" }]
    }
  ]
}
```

The important fields, and why each earns its place for a distributed backend:

- **`endpoints`** — the concrete addresses, so a query can distinguish "calls Stripe" from "calls a self-hosted mock" and so egress policy can be reconciled against declared dependencies.
- **`data` with `flow` and `classification`** — the data-flow and data-classification model. This is what turns "we use Stripe" into "we send cardholder data outbound to Stripe," which is precisely the sentence a PCI or GDPR assessment needs and a component SBOM can never produce.
- **`x-trust-boundary` / `trustZone`** — whether the edge crosses out of your trust domain. An internal mesh call to `orders` and an external call to Stripe are both dependencies, but they carry categorically different risk, and the SaaSBOM makes that distinction explicit and queryable.
- **`authenticated`** — whether the edge is authenticated at all, a first-order risk signal.

Represented this way, "service A depends on service B and on Stripe and on Auth0" is not a diagram in someone's head; it is data. And when Auth0 has an incident, the query "which services have an edge to provider Okta/Auth0, and what data classification crosses that edge" returns an answer — the topology question the opening section promised the component SBOM could not answer.

```
flowchart LR
    CO["checkout (svc)"]
    ORD["orders (internal svc)"]
    subgraph EXT["External / third-party (trust boundary crossed)"]
        STRIPE["Stripe API
PCI cardholder outbound"]
        AUTH0["Auth0 / Okta
signing keys inbound"]
        LD["LaunchDarkly
flags inbound"]
        DD["Datadog
telemetry outbound"]
    end
    subgraph MANAGED["Managed infra (your data, vendor-operated)"]
        RDS["RDS Postgres
PII at rest"]
        KAFKA["Managed Kafka
order events"]
    end
    CO -->|"bi-dir, PII-order"| ORD
    CO -->|"outbound, cardholder"| STRIPE
    CO -->|"inbound, signing keys"| AUTH0
    CO -->|"inbound, flags"| LD
    CO -->|"outbound, telemetry"| DD
    CO -->|"read/write, PII"| RDS
    ORD -->|"produce, order events"| KAFKA

    classDef svc fill:#14532d,stroke:#4ade80,color:#fff;
    classDef ext fill:#7f1d1d,stroke:#f87171,color:#fff;
    classDef mgd fill:#78350f,stroke:#fbbf24,color:#fff;
    class CO,ORD svc;
    class STRIPE,AUTH0,LD,DD ext;
    class RDS,KAFKA mgd;
```

The service-dependency graph as an org asset

The per-service SaaSBOM is useful; the **union of all of them across the fleet** is a strategic asset. Combine every service's `services` array and you have the organization's complete service-dependency graph — every internal edge and every external/SaaS edge, annotated with data flows and trust boundaries. Joined with the per-service *component* SBOMs, you get the full picture the opening section demanded: **what code runs** (components) *and* **what it talks to** (services). That combined graph answers questions neither half can answer alone: "if this SaaS vendor is breached, which services and which data classifications are exposed, and do any of those services *also* run a vulnerable component that would compound the incident?" Practically, this graph is what GUAC-

style stores (Chapter 5) and internal service catalogs want to ingest; the SaaSBOM is the standardized, per-service contribution that makes the org-wide graph assemblable from parts rather than hand-maintained in a wiki that is wrong within a quarter.

A caveat worth stating: the SaaSBOM `services` model is a **declarative** artifact — it captures intended/known dependencies, and it is only as complete as the discipline (or automation) that populates it. A service that adds a call to a new SaaS without updating its SaaSBOM has a gap exactly like an SBOM that misses a vendored library. The strongest implementations derive edges from observed reality — egress logs, service-mesh telemetry, API-gateway records — rather than trusting a hand-written list, which connects directly to the runtime-inventory theme of the next section.

Deployed vs built: runtime SBOMs for services

Chapter 7 named the freshness axis: an SBOM describes the artifact it was generated from, and the gap between "what we built" and "what is actually running in prod" is where inventory quietly goes wrong. For a service fleet this gap is not an edge case; it is the normal state. You built a hundred image digests last week; some are running, some were rolled back, some run in three regions and some in one, and a canary is running a digest that is not yet the tag's target. The build-time SBOM tells you what *could* be running. Only the orchestrator knows what *is*.

The orchestrator is the source of truth for "what is running"

In a Kubernetes fleet, the authoritative answer to "what is running right now" lives in the cluster: the images (by digest, if you resolve them) that are actually pulled and executing in Pods across all namespaces and clusters. This is the deploy-join of Chapter 5, and it is the pivot that turns a pile of build-time SBOMs into a live inventory:

```
# Enumerate every running container image, resolved to its digest,
across the fleet.
kubectl get pods --all-namespaces \
  -o jsonpath='{range .items[*]}{range .status.containerStatuses[*]}
{.imageID}{"\n"}{end}{end}' \
  | sort -u
# imageID carries the ...@sha256:<digest> that actually pulled — the
identity your SBOMs are keyed to.
```

The join is then mechanical and is the heart of a real fleet inventory: for each **running digest**, look up the **stored SBOM** attached to that digest (Chapter 5's store, populated via referrers), and you have "what is actually running, and everything inside it." A new CVE becomes a single query against the *stored inventory of running digests* — you do not rescan anything, and crucially you find the vulnerability even in an image nobody has rebuilt in a year, because the answer was pre-computed at build time and the deploy system tells you it is live. This is what answers Log4Shell **for a service org**: not "which images in the registry contain `log4j-core`" (which over-reports, including long-dead images) but "which *running* Pods, in which clusters and namespaces, contain it" — the question that scopes the actual incident.

```
flowchart TD
    subgraph BUILD["Build time — Chapter 4/5"]
        CI["CI builds image → digest"]
        GEN["syft/Trivy → SBOM"]
        ATT["cosign attest → attach via OCI referrers"]
        STORE["SBOM store / Dependency-Track / GUAC
keyed by digest"]
        CI --> GEN --> ATT --> STORE
    end
    subgraph RUNTIME["Runtime truth — the orchestrator"]
        K8S["Kubernetes API
running Pods → imageID@sha256"]
        EBPF["eBPF / runtime agent
what is actually LOADED in-process"]
    end
    subgraph QUERY["Fleet inventory query"]
        JOIN["JOIN running digests ⋈ stored SBOMs"]
        ANS["'Log4Shell': which RUNNING pods,
which clusters, contain the CVE"]
        JOIN --> ANS
    end
    STORE --> JOIN
    K8S --> JOIN
    EBPF -. refines .-> JOIN

    classDef b fill:#1e3a5f,stroke:#60a5fa,color:#fff;
    classDef r fill:#14532d,stroke:#4ade80,color:#fff;
    classDef q fill:#78350f,stroke:#fbbf24,color:#fff;
    class CI,GEN,ATT,STORE b;
    class K8S,EBPF r;
    class JOIN,ANS q;
```

Where even the deploy join is not enough: eBPF and what is loaded

The Kubernetes join tells you which *images* run, which is the right granularity for the vast majority of vuln response. It does not tell you which components inside an image are actually *loaded and executed*, nor does it catch things injected into a process after start — a library `dlopen`'d at runtime, an interpreted module imported dynamically, an agent that attaches to a running process. Runtime agents, increasingly built on **eBPF**, observe the process from the kernel side: which shared objects a process actually maps, which files it opens, what it executes. Two distinct uses follow. First, **reachability refinement** — an eBPF observation that a vulnerable `.so` present in the image is never loaded is real-world evidence for a VEX `not_affected` / `vulnerable_code_not_in_execute_path` justification (Chapter 6), turning a static "present" into a runtime "not exercised." Second, **catching what the image SBOM missed** — components pulled in at runtime that were never in the built artifact at all, the runtime-supply-chain case again but now inside your own process. Runtime SBOM/agent tooling is younger and noisier than image scanning; treat it as a refinement and a safety net over the build-time-plus-deploy-join backbone, not a replacement for it. The backbone is authoritative and cheap; the runtime layer is higher-fidelity but partial and operationally heavier.

Putting it together: the complete service-fleet BoM

Assemble the three pieces and you have the actual SBOM target for a distributed-backend organization — which is emphatically **not** "an SBOM for our product," because there is no product. There is a fleet of service-version-images, a graph of what they call, and a live map of what runs where:

```
flowchart TD
    subgraph P1["1 — Per-service, per-version COMPONENT SBOMs (build
time)"]
        C1["checkout@sha256:… → CycloneDX SBOM"]
        C2["orders@sha256:… → CycloneDX SBOM"]
        C3["base images → inherited SBOMs"]
        C4["sidecars / agents → their own SBOMs"]
    end
    subgraph P2["2 — Service-dependency graph (SaaSBOM)"]
        S1["each service's CycloneDX 'services' array"]
        S2["internal edges + external SaaS edges
+ data flows + trust boundaries"]
        S1 --> S2
    end
    subgraph P3["3 — Runtime deployment inventory (orchestrator)"]
        R1["K8s: running digests per cluster/namespace"]
        R2["eBPF: loaded components (refinement)"]
    end
    HUB["CENTRAL SBOM PLATFORM (Ch 5)
digest-keyed store + service graph + deploy join
= the complete service-fleet BoM"]
    P1 --> HUB
    P2 --> HUB
    P3 --> HUB
    HUB --> Q1["'Log4Shell': which running pods are affected?"]
    HUB --> Q2["'Auth0 breached': which services + data exposed?"]
    HUB --> Q3["'base X has a CVE': which running images inherit it?"]

    classDef a fill:#14532d,stroke:#4ade80,color:#fff;
    classDef b fill:#7f1d1d,stroke:#f87171,color:#fff;
    classDef c fill:#78350f,stroke:#fbbf24,color:#fff;
    classDef h fill:#1e3a5f,stroke:#60a5fa,color:#fff;
    class C1,C2,C3,C4 a;
    class S1,S2 b;
    class R1,R2 c;
    class HUB h;
```

The three feeds are complementary and each answers a class of question the others cannot:

- **Component SBOMs (build time)** answer "what code runs inside our services" — the vuln-response substrate, keyed by digest, generated once per image with base images and sidecars inherited rather than re-derived.
- **The service-dependency graph (SaaSBOM)** answers "what our services talk to and what data crosses each edge" — the runtime

supply chain and the blast radius of a *vendor* incident, which no component SBOM contains.

- **The runtime deployment inventory (orchestrator + eBPF)** answers "what is actually running where, right now" — the join that converts both of the above from "what we built and declared" into "what is live," and scopes any incident to real, running Pods.

Centralized in the Chapter 5 platform, keyed on digests and service identities, this is the organization's real SBOM. It is not a document you hand to a customer; it is a queryable, continuously updated model of a running distributed system.

Distributed-systems lens

This entire chapter is the lens — it exists because the classic artifact model does not fit an organization that runs services — but crystallize it into the load-bearing claims:

- **The unit of SBOM for a backend org is the service-version-image, not a shipped product.** There is no single artifact whose SBOM describes "your software." There is a fleet of digest-identified images, and the digest is the identity anchor for everything downstream.
- **Base images and sidecars are shared components; SBOM them once and inherit.** Most of a service's components come from the base. Generate the base and sidecar SBOMs at *their* build time, treat the base-image team as an internal supplier, and compute blast radius through the inheritance graph rather than re-deriving shared layers from every child.
- **The service-dependency graph (SaaSBOM) captures the runtime supply chain that classic SBOMs miss.** polyfill.io and Ledger Connect Kit are not exotic; they are the normal shape of a runtime-delivered third-party compromise. If your bill of materials only lists bytes inside your containers, it structurally cannot represent that risk. The CycloneDX `services` model can.
- **Join build-time SBOMs to the orchestrator's deployment truth for real inventory.** "What we built" over-reports; "what is running" is the question incidents actually ask. Kubernetes (refined by eBPF) is the source of truth for the join, and the join is what makes Log4Shell a query instead of a fire drill.

- **Managed and serverless runtimes create irreducible SBOM gaps you must acknowledge, not paper over.** For zip-packaged serverless you cannot enumerate the provider's runtime; that is a property of the shared-responsibility model, not a tooling deficiency. Document the boundary, cover the far side with provider attestations, and never let a coverage number imply a completeness it does not have.

Key takeaways

- A running service's bill of materials is a **runtime composition** — app + language deps + OS packages + base image + injected sidecars/agents + platform + the external services it calls — with a genuinely fuzzy boundary and multiple owners. Two purposes slice it differently: vuln response needs every executing layer; architecture/risk needs the service-dependency edges.
- **Containers are the workhorse.** SBOM the *final* image (multi-stage builds mean built ≠ shipped), key everything to the **digest**, generate with syft/Trivy to catch OS *and* language components, and attach via **OCI referrers** as a signed `cosign attest`. Inherit base-image SBOMs rather than re-deriving them per child.
- **Distroless** shrinks the attack surface and the SBOM but removes the package databases scanners read — a security win with an observability cost, answered by generating the base SBOM at base build time.
- **Serverless splits the artifact across a shared-responsibility line.** SBOM your handler, deps, and Lambda **layers** (each layer is a versioned, third-party component running with your privileges); container-image Lambda restores full-image SBOMs. The managed runtime is an **irreducible gap** — document it and rely on provider attestations.
- **SaaSBOM (CycloneDX `services`)** models the runtime supply chain component SBOMs miss: endpoints, data flows, classifications, and trust boundaries for every service and API you call. The org-wide union is the **service-dependency graph** — the asset that answers "if this vendor is breached, what is exposed."
- **The complete service-fleet BoM** is component SBOMs + the SaaSBOM graph + the orchestrator's runtime deployment inventory,

centralized and digest-keyed. That composite — not any single-artifact SBOM — is a distributed-backend organization's real SBOM target.

Further reading

- **OCI Image Specification 1.1** and the **Referrers API** (`/v2/<name>/referrers/<digest>`, `subject` / `artifactType`) — the mechanism for attaching a digest-anchored SBOM to an image (`opencontainers.org`; Chapter 5, Book 6, Chapter 4).
- **Anchore syft** and **Aqua Trivy** documentation — cataloging OS package databases (apk/dpkg/rpm) and language ecosystems from container images; `trivy image --format cyclonedx`, `syft <image> -o cyclonedx-json`.
- **CycloneDX 1.6 specification**, the **`services` (SaaSBOM)** model and the CycloneDX **Authoritative Guide to SaaSBOM** (OWASP) — external services, endpoints, data flows, and trust boundaries (Chapter 3).
- **Sigstore cosign** — `cosign attest --type cyclonedx`, `cosign verify-attestation`, and the deprecation of `cosign attach sbom`; the DSSE/in-toto predicate model (Book 5, Chapter 3).
- **AWS Lambda documentation** — the runtime **shared-responsibility model**, **Lambda layers** and **extensions**, container-image packaging, and `CodeSha256` /published function versions.
- **AWS Well-Architected / Shared Responsibility Model** and equivalent **Google Cloud Functions** and **Azure Functions** platform-responsibility documentation — the provider-versus-customer boundary for managed runtimes.
- The **polyfill.io** supply-chain incident (June 2024) — write-ups by Sansec and the Cloudflare/Fastly responses — and the **Ledger Connect Kit** compromise (December 2023) — Ledger's post-incident report — as canonical runtime/third-party-service compromises (Book 1, Chapter 9).
- **Kubernetes API reference** — `containerStatuses[].imageID` as the running-digest source of truth for the build-time-to-runtime join (Chapter 5).

- **OWASP Dependency-Track** and **GUAC** (`guac.sh`) — ingesting per-image SBOMs and per-service SaaSBOMs into a queryable, digest-keyed fleet model (Chapter 5).
- **eBPF** (`ebpf.io`) and runtime-inventory tooling — kernel-side observation of loaded components, for reachability refinement (Chapter 6) and catching runtime-injected components image SBOMs miss.

Chapter 9 — Operationalizing SBOMs in the Enterprise

What this chapter covers. The preceding chapters were about documents and pipelines: what an SBOM is and why regulators want it (Chapter 1), how to express it in SPDX (Chapter 2) or CycloneDX (Chapter 3), how to generate it at build time (Chapter 4), how to distribute, store, and query it at scale (Chapter 5), how to correlate it with vulnerabilities and suppress the noise with VEX (Chapter 6), and how to distrust it appropriately (Chapter 7). This is the capstone. It stops asking "what is a good SBOM?" and asks the only question an executive sponsor actually cares about: **what capability does the money buy?** The thesis is blunt. "We generate SBOMs" is a checkbox that produces landfill — millions of JSON documents nobody queries, generated to satisfy a clause in a contract, delivering no security value and a false sense of one. The deliverable is not the documents. The deliverable is a **capability**: the ability to answer, for your entire fleet, in minutes, "are we affected, where, and in what environment," and to drive that answer to remediation and prove it. That capability is a distributed system you build and run, staffed by an owner, fed by a paved road, measured by metrics that resist gaming, and rolled out the way you roll out any tier-1 dependency. This chapter is how you build the program around the pipeline.

Learning goals — after this chapter you should be able to:

- Distinguish the **checkbox failure mode** ("we produce SBOMs") from the **capability goal** ("we have an SBOM-driven vulnerability-response and transparency function"), and explain why every component of Chapters 4–7 must be present for value.

- Frame the SBOM program around **three flows** — inbound (supplier SBOMs for *your* risk), internal (fleet SBOMs for *your* vuln management, the largest source of value), and outbound (SBOMs you ship to customers and regulators) — sharing one generation substrate but demanding different quality bars and access controls.
- Assign **ownership** with a concrete RACI: platform/security runs the platform, app teams own per-service quality *through the paved road* rather than by hand, and security/compliance/IR/ procurement consume.
- Sequence a **crawl-walk-run rollout** — generation+storage, then correlation, then VEX, then enforcement and delivery — using warn-then-enforce gating rather than a big-bang mandate.
- Execute the **flagship use case**: a Log4Shell-class rapid-response runbook driven entirely off the SBOM platform, and four supporting use cases (license, customer delivery, procurement intake, forensics).
- Choose **metrics that measure the capability, not vanity coverage**, govern the sensitive data SBOMs contain, and place your program on a four-level **maturity model** mapped to the chapters.

A boundary note. This chapter integrates the whole book, so it references the others constantly rather than re-deriving them. It leans on the paved-road and program-building material in Book 1, Chapter 9 — Supply Chain Security in Distributed Backend Systems and Chapter 10 — Building a Program; on the internal-registry substrate of Book 2, Chapter 8 — Vendoring, Mirroring, and Internal Registries and the update automation of Book 2, Chapter 9; on Book 4's build platform; on Book 5, Chapter 3 — Sigstore Architecture for signing; on Book 6, Chapter 6 — Admission Control and Policy Engines for enforcement; and on Book 8's governance chapters — Chapter 3 (Vendor and Third-Party Software Risk), Chapter 6 (Incident Response for Supply Chain Events), and Chapter 8 (Metrics, Audits, and Executive Reporting) — for the parts of the program that live outside engineering.

The checkbox and the capability

There is a specific, common, expensive way to fail at SBOMs, and almost every organization that starts does it. A regulation lands — the U.S. Executive Order 14028 self-attestation, an EU CRA obligation, an FDA premarket requirement, or simply a large customer's procurement

questionnaire (all surveyed in Chapter 1). Someone in security is told "we need SBOMs." They add `syft` to a handful of CI pipelines, dump the output into an S3 bucket, and report to leadership that the company "has SBOM coverage." The box is checked. The auditor is satisfied. And the organization has bought *nothing* — no faster incident response, no fleet visibility, no reduced risk — while believing it has bought safety. This is worse than doing nothing, because the false confidence displaces the real work.

The tell is simple: **nobody queries the SBOMs.** An SBOM is a document written for exactly one moment — when a new vulnerability lands and you need to know if you are exposed. If your SBOMs are not wired into a system that answers that question, they are not security artifacts; they are compliance exhaust. Chapter 5 made the point structurally: an SBOM is a normalized inventory record, worthless in isolation and valuable only in aggregate, queried across the fleet. Chapter 6 added that without VEX the aggregate query drowns you in false positives until you stop trusting it. Chapter 7 added that without quality gates the query returns clean on the components that will hurt you most. The program is what assembles all of these into a function that fires when it matters.

State the goal precisely, because the wording drives everything downstream. The goal is **an SBOM-driven vulnerability-response and transparency capability.** Unpack it:

- *SBOM-driven* — the inventory of record is derived from build-time SBOMs, not from a best-effort periodic scan or a spreadsheet a team updates by hand.
- *vulnerability-response* — the primary consumer is your own incident response, continuously and during crises: the Log4Shell capability. This is where the value is, and it points *inward*.
- *transparency* — the secondary consumers are customers, regulators, and auditors who need to see into your software, and suppliers whose software you need to see into. This points *outward* and *inward* respectively, and it is the part regulation forces but not the part that pays.
- *capability* — a running system with an owner, an SLO, and a budget, not a pile of files.

Everything in this chapter is in service of turning the checkbox into the capability. And the first move is to see that the capability is not one flow of documents but three.

Three flows: inbound, internal, outbound

An SBOM is a description of software, and software crosses your organizational boundary in both directions. That gives three distinct flows, and conflating them is the second-most-common program design error (the first being generating without consuming). Each flow has a different producer, a different consumer, a different quality bar, and a different access-control posture, and a mature program handles all three off one shared generation-and-storage substrate.

```
flowchart LR
    subgraph OUT["OUTSIDE YOUR ORG"]
        VENDOR["Vendors and OSS
upstream software"]
        CUST["Customers, regulators,
auditors"]
    end
    subgraph ORG["YOUR ORG"]
        STORE[("Central SBOM store
+ correlation engine")]
        FLEET["Your build platform
every service, every image"]
        IR["Security, IR,
vuln management"]
        COMP["Compliance,
procurement"]
    end

    VENDOR -->|"INBOUND
supplier SBOMs for
YOUR risk mgmt"| STORE
    FLEET -->|"INTERNAL
fleet SBOMs for
YOUR vuln mgmt"| STORE
    STORE -->|"OUTBOUND
SBOMs + VEX for
THEIR risk mgmt"| CUST
    STORE --> IR
    STORE --> COMP
    COMP -.->|"intake + evaluate"| VENDOR

    classDef store fill:#1e3a5f,stroke:#60a5fa,color:#fff;
    classDef ext fill:#3f3f46,stroke:#a1a1aa,color:#fff;
    class STORE store;
    class VENDOR,CUST ext;
```

Internal flow — the fleet. This is the flow that pays for the program. Every service you build produces an SBOM at build time on the paved road (Chapter 4), the SBOMs land in the central store (Chapter 5), and they are correlated against vulnerability feeds and VEX (Chapter 6). The consumer is *you*: your vulnerability-management team, your incident responders, your on-call during a Log4Shell. The quality bar is set by your own query needs — component-and-version coverage and valid purls above all, per Chapter 7's fitness-for-purpose framing — and you control it end to end because you generate these SBOMs yourself. This flow is where SBOMs stop being compliance and become a live operational tool used every week, not every audit. If you build nothing else, build this.

Inbound flow — suppliers. Software you did not build enters your estate: commercial products, open-source you consume as binaries, container base images, appliances, SaaS you self-host. Their risk is your risk, and their SBOMs — *if they provide them* — let you fold that software into the same fleet inventory. The hard truths of this flow are covered in depth in Book 8, Chapter 3 (Vendor and Third-Party Software Risk); operationally, three problems dominate. First, **absence**: many vendors still ship no SBOM, and your program needs a policy for the gap (require it in the contract, generate one yourself from the delivered artifact, or accept the blind spot explicitly and record it). Second, **quality**: a vendor's SBOM is subject to every failure mode in Chapter 7, and you did not control its generation, so you must score it on intake (sbomqs, ntia-conformance-checker) and treat a low score as a risk signal about the vendor, not just about the document. Third, **normalization**: their SBOM arrives in whatever format and identifier scheme they chose, and your store must ingest SPDX and CycloneDX alike and reconcile their purls and CPEs with yours (Chapter 5's normalization problem, now with an adversarially indifferent producer). Inbound SBOMs are lower-trust and lower-quality than internal ones, and the program must model that difference rather than dumping everything into one undifferentiated table.

Outbound flow — customers and regulators. SBOMs you *ship*: to a customer whose procurement demands one, to a regulator under CRA or FDA, to an auditor. Here the quality bar is highest because the document leaves your control and represents you — it must be format-conformant, carry the NTIA minimum elements, be signed for provenance (Book 5), and travel with a VEX document so the recipient does not open thousands of tickets against components you have already assessed as not-affected (Chapter 6 made this the difference between a useful delivery and a support-ticket generator). It also must be **redacted**: an internal SBOM reveals your architecture, and you do not hand your customers a map of your dependency graph without deciding what to omit. Same generation substrate as the internal flow, different — stricter, redacted, signed — treatment on the way out.

The design consequence: **one generation-and-storage substrate, three treatments.** You generate once, on the paved road, at the highest internal quality you can. Internal consumption reads it raw; outbound delivery redacts, conforms, signs, and attaches VEX; inbound ingestion is a separate lower-trust path into the same store. Teams that stand up

three separate tools for three flows end up with three inconsistent inventories and no single answer to "are we affected." The store is the point of convergence.

The operational pipeline

Chapters 4 through 7 each owned one stage of a pipeline. Operationalizing means running the whole pipeline as one system with the seams welded shut — every stage automatic, every handoff on the paved road, every artifact keyed to an immutable digest so it can be found again.

```
flowchart LR
    G["GENERATE
syft at build time
paved-road CI"] --> Q["QUALITY GATE
sbomqs score
fail below threshold"]
    Q --> S["SIGN + ATTEST
cosign, provenance
Book 5"]
    S --> A["ATTACH
OCI referrers
keyed to digest"]
    A --> I["INGEST
normalize, dedup
central store"]
    I --> C["CORRELATE
vulns + VEX
runtime truth"]
    C --> QY["QUERY + ALERT
fleet inventory
dashboards"]
    QY --> ACT["ACT
patch, update-bot,
incident response"]
    ACT -.->|"verify fix
rebuild"| G

    classDef gen fill:#14532d,stroke:#4ade80,color:#fff;
    classDef mid fill:#1e3a5f,stroke:#60a5fa,color:#fff;
    classDef act fill:#7c2d12,stroke:#fb923c,color:#fff;
    class G,Q gen;
    class S,A,I,C mid;
    class QY,ACT act;
```

Three properties make this a *pipeline* rather than a set of tools someone runs by hand.

- **Build-time generation, not scan-time.** The SBOM is produced by the build that produced the artifact, from the same inputs, so it describes exactly what shipped — the freshness property Chapter 7 insisted on. A scanner run later against a registry sees only what it can reverse from the binary and inherits every accuracy failure of after-the-fact analysis.
- **Digest as the join key.** Everything downstream — the signature, the SBOM attestation, the correlation record — is keyed to the artifact's content digest (`sha256:...`), the same key the OCI referrers API uses to attach the SBOM to the image (Chapter 5). This is what lets you answer "what is in the thing that is actually running" instead of "what is in the thing named `latest`," which is a different and usually wrong question.
- **Runtime truth, not just build truth.** The last correlation step joins the built inventory to what is *deployed*. An SBOM store that knows what you built but not what is running answers the archaeological question, not the operational one. Joining to the orchestrator (which digests are scheduled, in which clusters, in which environments) turns "we once built something with Log4j" into "these 14 running services in prod expose it." That join is the single highest-leverage integration in the whole platform, and it is the one most programs skip.

Building the program: ownership

A capability without an owner decays into a checkbox. The first organizational question is not "which tool" but "whose job." SBOMs cut across platform engineering, security, compliance, procurement, and every app team, and diffuse ownership is how programs die — everyone assumes someone else runs the store, nobody funds it, and it rots into a stale bucket. Assign it explicitly. The pattern that works at scale mirrors Book 1, Chapter 10's paved-road model: a central team *owns and operates the platform as a product*, and app teams *consume it* without having to become SBOM experts.

Responsibility	Platform/ Security (platform team)	App teams	Security IR / VulnMgmt	Compliance / Procurement
Run the SBOM store + correlation service	**R/A**	I	C	I
Provide paved-road generation + quality gate	**R/A**	C	I	I
Per-service SBOM quality (in practice)	**A** (via paved road)	**R** (fix flagged gaps)	I	I
Vulnerability triage + VEX authoring	C	C (app knowledge)	**R/A**	I
Drive remediation / patching	C	**R**	**A**	I
Outbound SBOM delivery to customers	C	I	I	**R/A**
Inbound supplier SBOM intake + scoring	C	I	C	**R/A**
Program metrics + executive reporting	C	I	C	**R/A**

(R = responsible, A = accountable, C = consulted, I = informed.)

The load-bearing idea in this table is the third row. **App teams are responsible for per-service SBOM quality, but only through the paved road** — not by writing SBOMs, running tools, or learning SPDX. The platform generates a quality SBOM automatically for every service on the standard build; the app team's residual responsibility is narrow:

when the quality gate flags a gap the platform *cannot* fix generically (a vendored C library the generator can't see, a shaded JAR, per Chapter 7's systematic-accuracy failures), the team that owns that code supplies the missing knowledge. Make app teams responsible for SBOM quality *without* the paved road and you have asked five hundred teams to each become mediocre SBOM engineers, and will get five hundred inconsistent, high-effort SBOMs. The paved road is what makes the RACI survivable.

The corollary is a funding reality Book 8, Chapter 8 will quantify: the platform team needs headcount to run the store as tier-1 infrastructure, and that budget competes with feature work. The argument that wins it is the flagship use case — one hour of a company-wide Log4Shell scramble avoided pays for the platform — so instrument the program to *show* that value, which brings us to metrics later.

Building the program: the paved-road integration

The technical heart of the program is making a quality SBOM a *free, inherited, unavoidable* side effect of building software. This is the paved-road pattern from Book 1, Chapters 9 and 10 applied to SBOMs: the golden path is so easy and so default that using it is less work than avoiding it, and the security property comes along for free because it is baked into the shared build, not bolted onto each service.

Concretely, the shared CI template — the reusable GitHub Actions workflow, the Tekton task, the internal build wrapper (Book 4) — that every service already inherits for building and pushing its image gains a few steps. Every service that builds the standard way gets them without touching its own pipeline:

```
# The shared, inherited build workflow — every service calls this, not
its own copy.
# Adding SBOM generation here gives it to the whole fleet at once.
jobs:
  build-and-attest:
    steps:
      - uses: ./build-image        # existing: produces IMAGE_DIGEST

      # 1. Generate at build time, from the built image, keyed to its
digest.
      - name: Generate SBOM
        run: syft "${IMAGE}@${IMAGE_DIGEST}" -o cyclonedx-
json=sbom.cdx.json

      # 2. Quality gate (Chapter 7). Fail the build below threshold.
      - name: Score SBOM quality
        run: |
          sbomqs score sbom.cdx.json --json > score.json
          score=$(jq '.files[0].avg_score' score.json)
          echo "SBOM quality score: ${score}"
          awk -v s="$score" 'BEGIN{exit !(s>=7.0)}' \

|| { echo "SBOM quality ${score} below threshold 7.0"; exit 1; }

      # 3. Sign + attest the SBOM as an in-toto attestation, keyed to
the digest (Book 5).
      - name: Attest SBOM
        run: |
          cosign attest --yes --predicate sbom.cdx.json \
            --type cyclonedx "${IMAGE}@${IMAGE_DIGEST}"

      # 4. Push SBOM to the central store, keyed to digest, for fleet-
wide query.
      - name: Ingest into SBOM platform
        run: |
          curl -sf -X POST "${SBOM_STORE}/v1/sboms" \
            -H "Authorization: Bearer ${OIDC_TOKEN}" \
            -F "digest=${IMAGE_DIGEST}" -F "sbom=@sbom.cdx.json"
```

Four properties of this arrangement matter more than the exact commands:

1. **Inherited, not copied.** The steps live in the shared template. When you improve SBOM generation — add a source-plus-binary diff (Chapter 4), tighten the quality threshold, switch generators — every service inherits the improvement on its next build. If instead each team had pasted these steps into its own pipeline, you would be filing

five hundred pull requests to change anything, and the fleet's SBOMs would drift into five hundred versions.

2. **The quality gate is in the pipeline (Chapter 7).** A build that produces a garbage SBOM fails, the same way a build that fails tests fails. This is what keeps the *distribution* of quality healthy rather than letting the long tail of unqueryable SBOMs accumulate silently. Start the threshold low (warn only), raise it as the fleet improves — the rollout discipline of the next section.
3. **It is signed and attached, keyed to the digest.** The SBOM is a signed in-toto attestation (Book 5, Chapter 3) attached to the image via OCI referrers, so provenance travels with it and the store can trust what it ingests. An unsigned SBOM in a bucket has no defense against tampering or against simple staleness.
4. **Shipping without an SBOM is made impossible by policy, not by request.** The final lever is admission control (Book 6, Chapter 6): a Kyverno or OPA/Gatekeeper policy in the deploy path that *refuses to admit an image that lacks a valid, signed SBOM attestation*. This is what converts "please generate SBOMs" from a request every busy team can defer into an invariant of the platform. You cannot deploy without one, so you have one, so the fleet inventory is complete by construction.

```
# Kyverno policy (Book 6, Ch 6): no SBOM attestation, no admission.
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-sbom-attestation
spec:
  validationFailureAction: Enforce      # start as Audit, promote to
Enforce (warn-then-enforce)
  rules:
    - name: check-sbom-attestation
      match:
        any:
          - resources: { kinds: [Pod] }
      verifyImages:
        - imageReferences: ["registry.internal/*"]
          attestations:
            - type: https://cyclonedx.org/bom
              conditions:
                - all:
                    - key: "{{ regex_match('(?i)cyclonedx', '{{ predica
teType }}') }}"
                      operator: Equals
                      value: true
```

This is the closed loop that defeats the checkbox failure: generation is inherited, quality is gated, provenance is signed, and admission enforces presence. No team has to care, and the inventory is complete anyway.

Rollout: crawl, walk, run

You cannot flip all of this on at once. A big-bang mandate — "every service must have a signed, quality-gated, VEX-annotated SBOM by Q3, enforced at admission" — fails the way every big-bang security mandate fails: it breaks builds the platform team didn't anticipate, generates a revolt, and gets rolled back, poisoning the well for the real rollout. Sequence it, and gate each stage warn-then-enforce, exactly as Book 1, Chapter 10 prescribes for any fleet-wide control. The sequence is not arbitrary; each stage is worthless without the one before and unusable without becoming the substrate for the one after.

```
flowchart TD
    L1["STAGE 1 — VISIBILITY
generate + store
Ch 4 + Ch 5
outcome: you can answer 'where is X'"]
    L2["STAGE 2 — VULN MGMT
correlate with vuln feeds
Ch 6
outcome: 'are we affected by CVE-Y'"]
    L3["STAGE 3 — DE-NOISE
add VEX + reachability
Ch 6 + Book 2 Ch 7
outcome: signal you can act on"]
    L4["STAGE 4 — ENFORCE + DELIVER
quality gate + admission + outbound
Ch 7 + Book 6 + compliance
outcome: complete inventory, external delivery"]

    L1 --> L2 --> L3 --> L4

    classDef s fill:#1e293b,stroke:#94a3b8,color:#fff;
    class L1,L2,L3,L4 s;
```

- **Stage 1 — visibility.** Turn on generation and storage in warn mode. No build fails; SBOMs are produced and ingested. The single deliverable is that you can query the store and get an answer to "where do we use component X." This alone is more than most orgs have, and it is enough to justify the next stage. Do not gate anything yet — you are measuring coverage and fixing generation bugs, not punishing teams.
- **Stage 2 — correlation.** Wire the store to vulnerability feeds (OSV, NVD, GHSA — Book 2, Chapter 5) so the inventory becomes a live "are we affected by CVE-Y" query. Now the platform is useful during an incident, and you have your first real value demonstration. It is also, without the next stage, unbearably noisy: every transitive dependency with an unreachable CVE lights up.
- **Stage 3 — VEX and reachability.** Add VEX (Chapter 6) so triaged not-affected findings are suppressed, and reachability (Book 2, Chapter 7) so exploitable findings float to the top. Without this stage teams learn to ignore the platform's alerts — the worst possible outcome, because an ignored alerting system is indistinguishable from no system during the incident that matters. De-noise before you make anyone accountable to the alerts.

- **Stage 4 — enforce and deliver.** Only now do you turn the quality gate from warn to enforce, turn the admission policy from Audit to Enforce, and stand up the outbound-delivery and inbound-intake flows for compliance and procurement. Enforcement is last because it is only fair once the paved road reliably produces a passing SBOM with near-zero team effort — otherwise you are punishing teams for a platform gap.

The discipline throughout is *warn, measure, fix the platform, then enforce.* Every gate ships in audit mode, runs long enough to surface the surprises (the team building firmware whose SBOM the generator can't see; the legacy service that doesn't use the shared build at all), and flips to enforce only when the dashboard shows the fleet already passing. Enforcement should be a formality that breaks almost nothing, because the warn period already drove the number to green.

Use cases realized

Maturity is measured in use cases served, not documents stored. Here are five, flagship first.

Flagship: rapid vulnerability response

This is the use case that justifies the program, and it is worth walking end to end because it exercises every stage of the pipeline under time pressure. The scenario is Log4Shell (CVE-2021-44228, disclosed December 2021, a trivially exploitable unauthenticated RCE in the near-ubiquitous `log4j-core`, described in detail in Book 1, Chapter 5). The lived experience of that incident, for organizations without an SBOM platform, was days-to-weeks of manual archaeology: every team grepping its own build files, no central answer, no way to know when you were done. The whole point of the program is to turn that scramble into a query.

```
flowchart TD
    CVE["1. NEW CRITICAL CVE
CVE-2021-44228 log4j-core
affects 2.0-beta9 .. 2.14.1"]
    Q["2. QUERY FLEET INVENTORY
pkg:maven/org.apache.logging.log4j/log4j-core
across every SBOM, keyed to running digests"]
    ID["3. IDENTIFY BLAST RADIUS
which services, which versions,
which environments (prod vs dev)"]
    PRI["4. PRIORITIZE
reachability + VEX + internet-facing
real exposure, not raw hit count"]
    FIX["5. REMEDIATE
update-bot PRs bump to 2.17.1
Book 2 Ch 9, rebuild via paved road"]
    VER["6. VERIFY
re-query inventory on new digests
hit count -> 0, prove closure"]

    CVE --> Q --> ID --> PRI --> FIX --> VER
    VER -.->|"residual: un-upgradable services"| MIT["7. MITIGATE + VEX
WAF rule, config flag;
record VEX not_affected/mitigated"]

    classDef hot fill:#7f1d1d,stroke:#f87171,color:#fff;
    classDef cool fill:#14532d,stroke:#4ade80,color:#fff;
    class CVE,ID hot;
    class VER,MIT cool;
```

Walk the runbook with the platform:

1. **The CVE lands.** `log4j-core`, purl `pkg:maven/org.apache.logging.log4j/log4j-core`, vulnerable range `>=2.0-beta9, <2.15.0` (later widened as the fix itself needed fixing — 2.15, then 2.16, then 2.17.1). Your correlation engine already pulls the advisory from OSV/GHSA the moment it publishes.
2. **Query the fleet.** One query against the store, matching that purl across every service's most-recent SBOM, joined to the orchestrator so it returns *running* digests, not just built ones:

```
sql -- Fleet inventory query: who is running a vulnerable log4j-core, right now, where? SELECT s.service, s.version, d.environment, c.version AS log4j_version FROM sbom_components c JOIN sboms s ON c.sbom_digest = s.digest JOIN deployments d ON d.image_digest = s.digest -- runtime truth WHERE c.purl_name = 'pkg:maven/
```

`org.apache.logging.log4j/log4j-core' AND semver_in_range(c.version, '>=2.0-beta9,<2.15.0') AND d.state = 'running' ORDER BY d.environment, s.service;`

The answer comes back in the time the query takes to run — minutes, not days. This single capability is the difference between the program and the checkbox, and it is why the internal flow is where the value lives.

1. **Identify the blast radius.** The result set *is* the blast radius: exact services, exact versions, exact environments. Note what the SBOM alone cannot tell you and Chapter 7 warned about — a service that statically shaded log4j into an uber-JAR may not appear if your generator missed it, which is precisely why the quality gate and the known-blind-spot catalog exist. The honest program hands IR the query result *and* the list of pipeline blind spots so responders know where to look manually.

2. **Prioritize.** Raw hit count is not a work queue. Layer on Chapter 6's VEX and Book 2, Chapter 7's reachability: a service that bundles log4j but never calls the vulnerable JNDI lookup path, or runs with `log4j2.formatMsgNoLookups=true`, is a lower priority than an internet-facing service that logs user-controlled input. Sort by real exposure — internet-facing × reachable × prod — so the responders spend the first hour on the services that can actually be popped.

3. **Remediate.** Drive fixes through the update automation of Book 2, Chapter 9: the platform opens dependency-bump PRs (bumping to 2.17.1) across the affected services, they rebuild on the paved road, and the paved road regenerates and re-ingests the SBOM automatically — closing the loop back to stage 1 of the pipeline.

4. **Verify.** Re-run the same query. As fixed services redeploy on new digests, the hit count falls toward zero, and you have something the 2021 scramble never had: *proof of closure*, a number you can watch reach zero and report to leadership as "done" with evidence rather than hope.

5. **Handle the residual.** Some services can't upgrade immediately (a frozen release, a vendor dependency). Mitigate them out-of-band (WAF rule, the config flag) and *record the mitigation as VEX* (`status: not_affected` with justification, or `affected` with a mitigation note) so the next query and the next auditor see that the residual is known and managed, not forgotten.

Every stage of the program earns its keep here: generation (the inventory exists), storage and query (the answer is fast), correlation (the CVE maps to components), VEX and reachability (the answer is prioritized), runtime join (the answer is about what's running), update automation (the fix ships), and verification (closure is proven). Remove any one and the runbook degrades to archaeology.

License compliance across the fleet

The same inventory answers a different question with a different consumer. Legal or an open-source program office needs to know the fleet's license exposure — where GPL-family or other copyleft/commercial-incompatible licenses appear, especially in software you distribute. Because the SBOM already carries per-component license fields (Book 1, Chapter 8 covers the license model in depth), this is another query against the same store:

```
SELECT s.service, c.name, c.version, c.license
FROM   sbom_components c JOIN sboms s ON c.sbom_digest = s.digest
WHERE  c.license SIMILAR TO '%(GPL|AGPL|SSPL)%'
  AND  s.distributed = true;      -- exposure matters most in shipped
software
```

The operational point is reuse: you built the store for vuln response, and license compliance falls out for near-zero marginal cost. A program that stood up a separate license scanner and a separate vuln scanner with separate inventories has two partial truths; the SBOM program has one inventory answering both.

Customer and regulator delivery

The outbound flow, realized. A customer's procurement team or a regulator (CRA, FDA) demands an SBOM for a product you ship. The delivery is not "email them the raw internal SBOM." It is a pipeline of its own: pull the signed SBOM for the exact shipped digest from the store, **redact** internal-only detail per your data-governance policy (below), **conform** it to the required format and the NTIA minimum elements (Chapter 1), attach the current **VEX** so the customer doesn't open a ticket for every unreachable CVE (Chapter 6), and deliver it signed (Book 5) so the recipient can verify provenance. The quality bar here is the strictest of the three flows because the artifact represents you to an

outside party and to a regulator; a malformed or unsigned outbound SBOM is a compliance finding waiting to happen.

Procurement and vendor SBOM intake

The inbound flow, realized, and the operational half of Book 8, Chapter 3. When you onboard a vendor or a new open-source binary dependency, request their SBOM as part of procurement, ingest it into the same store on the lower-trust inbound path, and **score it on arrival** (sbomqs, ntia-conformance-checker — Chapter 7). The score is a decision input two ways: a low-quality or absent SBOM is a data point about the vendor's own supply-chain maturity, and it tells you how much you can trust the resulting inventory rows. When the vendor ships no SBOM at all, the policy is explicit rather than silent: require it contractually going forward, generate one yourself from the delivered artifact (accepting that after-the-fact generation inherits Chapter 4's accuracy limits), or record the blind spot. The failure mode to avoid is a store that silently mixes high-trust internal rows with unlabeled low-trust vendor rows, so that a Log4Shell query returns an answer you cannot calibrate.

Forensics, audit, and M&A

The inventory is also a historical record, and three consumers want history. **Incident forensics** (Book 8, Chapter 6): after a compromise you need to know exactly what was in a given artifact at a given time — the store, keyed to immutable digests and retaining historical SBOMs, answers "what was running in prod on the day of the breach" without reconstruction. **Audit**: an auditor asking "show me you knew your components" gets a query result and a retention history rather than a promise. **M&A due diligence**: acquiring a company, you want its fleet's component and license exposure before you sign; if it runs an SBOM program you can ingest and query it, and if it does not, the absence is itself diligence signal. In every case the value is the same — the inventory of record, retained over time, keyed to what actually shipped.

Metrics and governance

Metrics that measure the capability, not the checkbox

Chapter 7 warned that "100% SBOM coverage" is a coverage metric masquerading as a visibility metric. The program's metrics must resist

that game. The governing distinction, developed fully in Book 8, Chapter 8, is **leading vs lagging**: leading indicators (coverage, quality, VEX presence) predict whether the capability *will work* when you need it; lagging indicators (time-to-inventory, time-to-remediate) measure whether it *did work*. A program that reports only leading metrics is claiming readiness it has never tested; one that reports only lagging metrics learns of gaps only during incidents. Track both.

Metric	Type	What it measures	The vanity trap it replaces
Quality-weighted coverage	Leading	% of running artifacts with a *current, quality-gated* SBOM (score ≥ threshold), not merely "an SBOM"	"100% have an SBOM" — counts landfill as coverage
SBOM quality distribution	Leading	The full histogram of sbomqs scores across the fleet, watching the tail	A single average that hides the unqueryable long tail
Runtime-join coverage	Leading	% of running digests joined to an SBOM (built truth = running truth)	"We built SBOMs" while inventory ≠ what's deployed
VEX coverage	Leading	% of open findings with a VEX status	Raw open-finding count that teams have learned to ignore
Inbound-SBOM coverage	Leading	% of vendors/third-party artifacts with an ingested, scored SBOM	Counting only what you built; ignoring supplier blind spots
MTTR-to-inventory	Lagging	Time from a new CVE to a fleet-wide "are we affected, where" answer	Not measured at all in checkbox programs
MTTR-to-remediate	Lagging	Time from identification to verified fix across affected services	Ticket-close counts detached from real closure

Two disciplines around this table. First, **quality-weight every coverage number** — counting a schema-perfect-but-empty SBOM (Chapter 7) as covered is precisely the vanity metric the book warns against; count only

SBOMs that would actually *answer the query*. Second, **treat MTTR-to-inventory as the headline capability metric.** It directly measures the Log4Shell capability, it justifies the platform's budget to the sponsor who signs off on headcount, and a checkbox program cannot produce it because it has never run the query. Rehearse it: periodically pick a random component, run the fleet query, and time it. That drill is to the SBOM program what a fire drill is to a building — the only way to know the capability is real before the day you need it.

Data governance: the SBOM is sensitive

An SBOM is a map of your software's internals — its dependencies, versions, and by inference its architecture, its build tooling, and sometimes internal component and service names. That map is useful to you and useful to an attacker, who can read your outbound SBOM to learn exactly which vulnerable versions to target. So the program needs a data-governance posture, not an open bucket:

- **Access control on the store.** Internal SBOMs are internal data. The store enforces authorization the way any sensitive datastore does; not every engineer needs to query the whole fleet's dependency graph, and the query logs are themselves a signal.
- **Internal vs external is a deliberate boundary.** Decide explicitly what leaves the org. The internal flow can carry everything; the outbound flow carries a curated subset.
- **Redaction for customer-facing SBOMs.** Strip internal-only components, internal service and path names, and infrastructure detail from outbound SBOMs, keeping what the customer legitimately needs (the third-party and open-source components whose risk they are assessing) and omitting the parts that are purely a map of your internals. The regulation asks for transparency into the software you ship, not a blueprint of your estate.

The tension is real: transparency is the point of an SBOM, and redaction cuts against it. Resolve it per-consumer rather than globally — maximal transparency internally where the SBOM is a tool, calibrated transparency externally where it is a disclosure — which is exactly why the three-flow model keeps the treatments separate.

Common pitfalls

The failure modes cluster, and naming them is cheaper than living them:

- **Sprawl without a query platform.** Generating SBOMs into a bucket with no ingestion, normalization, or query layer. You have paid the generation cost and captured none of the value, because the value is *only* in aggregate query (Chapter 5). This is the checkbox with extra storage bills.
- **Generating but never consuming.** The pipeline runs, the store fills, and no human or system ever queries it — no correlation, no alerting, no drill. The tell is that no one would notice if generation broke. If nobody queries, stop generating or start consuming; the middle is pure cost.
- **False confidence from poor quality.** Trusting a fleet of SBOMs that are schema-valid and substantively incomplete (Chapter 7). The Log4Shell query returns "clean" for a service that shaded log4j into an uber-JAR the generator never saw, and you stand down while still exposed. This is the most dangerous pitfall because it fails silently and specifically at the moment of need. The quality gate and the explicit blind-spot catalog exist to bound it.
- **Optimizing the outbound flow while ignoring the internal one.** Pouring effort into regulator-ready, signed, conformant outbound SBOMs — the compliance deliverable — while never standing up the internal vuln-management capability that actually reduces risk. This is the checkbox at its most seductive, because it produces impressive audit artifacts while leaving you exactly as slow to respond as before. The internal flow is where the value is; lead with it.
- **No VEX, so drowning in noise.** Correlation without VEX (Chapter 6) produces so many unreachable false positives that teams tune out the alerts, and a tuned-out alerting system is worse than none because it gives false assurance that someone is watching. De-noise before you hold anyone accountable to the signal.
- **No link to runtime truth.** An inventory of what you *built*, not what you *run*. During an incident you query it and get a list padded with services long since decommissioned and missing services deployed from images the store never saw. The build-to-runtime join is not optional; it is what makes the inventory operational.

Every one of these is a way of having the parts without the capability. The program is precisely the work of assembling the parts so the capability emerges.

The maturity model

Put it together as a ladder. Each level is a coherent capability plateau, mapped to the chapters that build it, and — critically — each level is *worth reaching and stopping at* if that is your budget, but only in order. Skipping a rung produces a program with expensive stage-4 outbound delivery sitting on a stage-1 inventory nobody can query.

```
flowchart TD
    L1["LEVEL 1 — GENERATE
SBOMs produced at build time
Ch 4
capability: documents exist"]
    L2["LEVEL 2 — STORE + QUERY
central normalized store
Ch 5
capability: 'where is component X'"]
    L3["LEVEL 3 — CORRELATE + VEX
vuln feeds + VEX + reachability
Ch 6, Book 2 Ch 7
capability: prioritized 'are we affected'"]
    L4["LEVEL 4 — ENFORCE + DELIVER + MEASURE
quality gate, admission, three flows, metrics
Ch 7, Book 6, Book 8 Ch 8
capability: complete, governed, provable program"]

    L1 --> L2 --> L3 --> L4

    classDef l1 fill:#3f3f46,stroke:#a1a1aa,color:#fff;
    classDef l2 fill:#1e3a5f,stroke:#60a5fa,color:#fff;
    classDef l3 fill:#134e4a,stroke:#2dd4bf,color:#fff;
    classDef l4 fill:#14532d,stroke:#4ade80,color:#fff;
    class L1 l1;
    class L2 l2;
    class L3 l3;
    class L4 l4;
```

Level	Capability	Key mechanisms	Chapters	Honest self-assessment
1 — Generate	SBOMs exist for built artifacts	Build-time generation (`syft`), OCI attach	Ch 4	"We have SBOMs" — the checkbox; zero query value if you stop here
2 — Store + Query	Answer "where is component X" fleet-wide	Central store, normalization, dedup, digest keys	Ch 5	First real value; you could survive a Log4Shell manually-ish
3 — Correlate + VEX	Prioritized "are we affected by CVE-Y"	Vuln feeds, VEX suppression, reachability, runtime join	Ch 6, Book 2 Ch 7	The Log4Shell capability is real and de-noised
4 — Enforce + Deliver + Measure	Complete inventory by construction; governed three-flow program; provable	Quality gate, admission control, outbound/ inbound flows, metrics, governance	Ch 7, Book 4, Book 5, Book 6, Book 8	A capability, not a checkbox; measured, funded, drilled

Most organizations that "have SBOMs" are at Level 1 and believe they are done. The distance from Level 1 to Level 3 is the distance from the checkbox to the capability, and it is almost entirely platform-and-process work — the documents were already there at Level 1. Level 4 is where the program becomes durable: complete by construction (admission enforcement), trustworthy (quality gate), governed (the three flows with their access controls), and provable (metrics and drills). Know your level, and climb it in order.

Distributed-systems lens

The reframing this whole chapter argues for is that **the SBOM platform is a tier-1 internal distributed system, and the documents are just its records.** Look at what you actually build to operationalize SBOMs

and it is unmistakably a distributed backend system of exactly the kind this reader builds for a living:

- **An ingestion pipeline** running at fleet scale — every build of every service emits an SBOM, which is a high-fan-in write path with the usual concerns: backpressure when a monorepo's thousand services all rebuild at once, idempotency so a retried CI job doesn't double-write, schema evolution as SPDX and CycloneDX versions drift.
- **A normalized datastore** — the reconciliation of SPDX and CycloneDX, of purls and CPEs, of internal and inbound sources, into one queryable inventory, which is a data-modeling and entity-resolution problem (Chapter 5) with the added twist that the identifiers are unreliable (Chapter 7).
- **A correlation service** joining that inventory against continuously-updated external feeds (OSV, NVD, GHSA) and internal VEX — a streaming join between your slowly-changing inventory and a fast-changing vulnerability stream, with the freshness and consistency questions any such join raises.
- **A query and alerting layer** that must answer the fleet-wide "are we affected" in minutes under incident load — a latency SLO on the one query that matters most, precisely when the system is under the most stress and attention.

And it lives at the center of a graph of integrations that are themselves the org's core infrastructure: joined to the **orchestrator** for runtime truth (what is running, where), to the **internal registry and proxy** (Book 2, Chapter 8) and the **build platform** (Book 4) for generation, to the **signing infrastructure** (Book 5) for provenance, and to the **admission controller** (Book 6) for enforcement. It is not a side project bolted onto security; it is a node in the same dependency graph as your service mesh and your CI system, with the same availability expectations during an incident.

The payoff is the thing that makes it worth building as real infrastructure rather than a spreadsheet: **one query answers "are we affected" for every service at once.** That property — fleet-wide, in minutes, with a proof of closure — does not exist at any smaller scale of effort. You cannot get it from per-team greps, from periodic scans, or from a compliance bucket. You get it only by building the platform. And that is the whole argument of the book in one sentence: the deliverable of an SBOM

program is not the bill of materials. It is the *capability the bill of materials makes possible* — retroactive, fleet-wide, provable transparency and response — and that capability is a distributed system you build, run, staff, measure, and drill like any other tier-1 dependency. The documents are the substrate. The capability is the product.

Key takeaways

- **The deliverable is a capability, not documents.** "We generate SBOMs" is a checkbox that produces landfill; the goal is "we can answer, fleet-wide, in minutes, are we affected, where, and in what environment — and prove the fix." If nobody queries the SBOMs, you have bought nothing but false confidence.
- **Value requires the whole book.** Generation (Ch 4) without storage/query (Ch 5) is sprawl; query without correlation (Ch 6) is inventory trivia; correlation without VEX is unactionable noise; all of it without quality gates (Ch 7) is false confidence. The program is the assembly.
- **Three flows, one substrate.** Internal (fleet vuln management — where the value is), inbound (supplier SBOMs, lower-trust, scored on intake — Book 8 Ch 3), and outbound (customer/regulator delivery, strictest bar, redacted and signed). Generate once; treat differently per flow.
- **Own it explicitly and pave the road.** A central platform team runs the store as a product; app teams get quality SBOMs for free from the inherited build; admission control (Book 6 Ch 6) makes shipping without a signed SBOM impossible, so the inventory is complete by construction.
- **Roll out crawl-walk-run, warn-then-enforce.** Visibility → correlation → VEX de-noising → enforce-and-deliver. Every gate ships in audit mode and flips to enforce only once the paved road already produces a passing SBOM with near-zero team effort.
- **Rapid vuln response is the flagship.** New CVE → fleet query joined to runtime truth → prioritize with reachability and VEX → drive fixes via update automation (Book 2 Ch 9) → re-query to prove closure. This turns a Log4Shell scramble from an archaeological dig into a database query.
- **Measure the capability, not vanity coverage.** Quality-weighted coverage, quality distribution, runtime-join coverage, VEX coverage,

and above all MTTR-to-inventory (rehearsed as a drill). Never let "100% have an SBOM" masquerade as visibility.

- **Govern the data.** SBOMs map your internals and help attackers too; access-control the store, keep the internal/external boundary deliberate, and redact outbound SBOMs to disclose the third-party risk your customer needs without handing over a blueprint of your estate.
- **The platform is a tier-1 distributed system.** An ingestion pipeline, a normalized datastore, a streaming correlation join, and a low-latency query layer, integrated with the orchestrator, the registry, the build platform, and the admission controller. Build it, staff it, and drill it like any other core dependency.

Further reading

- **CISA**, "Software Bill of Materials (SBOM)" resource hub and the SBOM community working-group outputs (Sharing & Exchanging, SBOM Types, Framing) — the evolving practice-and-process side of operationalization, beyond the document formats.
- **NTIA**, "The Minimum Elements for a Software Bill of Materials (SBOM)," 12 July 2021 — the outbound-delivery floor and the crucial concept of *known unknowns* for the blind-spot catalog.
- **OWASP Software Component Verification Standard (SCVS)** and its **BOM Maturity Model** — a vendor-neutral rubric that formalizes the maturity progression this chapter sketches.
- **U.S. Executive Order 14028** and the subsequent OMB M-22-18 / M-23-16 self-attestation guidance — the regulatory driver behind the outbound flow, and a cautionary source of checkbox pressure.
- **EU Cyber Resilience Act (CRA)** — SBOM and vulnerability-handling obligations for products with digital elements; a concrete forcing function for the outbound and internal flows both.
- **Dependency-Track** (OWASP) — a reference open-source implementation of the store-plus-correlate layers (Levels 2–3); read its architecture to see the ingestion/normalization/correlation split as a real system.
- **OpenVEX** and the **CSAF / CycloneDX VEX** profiles — the de-noising layer that makes correlation actionable (Chapter 6), and a required companion to any outbound delivery.

- **Sigstore / cosign** documentation on attestations and the **OCI referrers API** — how SBOMs are signed, attached to digests, and made trustworthy in transit (Book 5, Chapter 3).
- **Kyverno** and **OPA/Gatekeeper** image-verification documentation — the admission-control mechanism that enforces SBOM presence at deploy time (Book 6, Chapter 6).
- **The Log4Shell (CVE-2021-44228) retrospectives** — read several accounts of the December 2021 response specifically for the *inventory problem*: the organizations that suffered longest were the ones that could not answer "where do we run this," which is the exact gap this program closes.